

I have JSON object in python and I would like to make a dataframe instead to keep only the name of the Pokemon as a column and its tier as another column the tier column will be determined by which JSON is being written (I have a separate JSON for each tier), the structure of the JSON is as follows: "pokemon": {

```
"Great Tusk": {  
  "lead": {"raw": 0.035, "real": 0.035, "weighted": 0.029},  
  "usage": {"raw": 0.2887, "real": 0.2865, "weighted": 0.3552},  
  "count": 630196,  
  "weight": 0.0845,  
  "viability": [36152, 91, 81, 64],  
  "abilities": {"Protosynthesis": 1},  
  "items": {  
    "Booster Energy": 0.3061,  
    "Heavy-Duty Boots": 0.2799,  
    "Rocky Helmet": 0.1696,  
    "Leftovers": 0.1045,  
    "Assault Vest": 0.0565,  
    "Choice Scarf": 0.0419,  
    "Choice Band": 0.0156,  
    "Eject Pack": 0.0096,  
    "Red Card": 0.0037,  
    "Life Orb": 0.0036,  
    "Focus Sash": 0.002,  
    "Covert Cloak": 0.0015,  
    "Expert Belt": 0.0013,  
    "Fist Plate": 0.001,  
    "Lum Berry": 0.0008,  
    "Clear Amulet": 0.0006,  
    "Muscle Band": 0.0005,  
    "Soft Sand": 0.0004,  
    "Misty Seed": 0.0003,  
    "Heat Rock": 0.0002,  
    "Weakness Policy": 0.0002,  
    "Eject Button": 0.0001,  
    "Chesto Berry": 0.0001,
```

```

"Earth Plate": 0.0001,
"Citrus Berry": 0.0001,
"Custap Berry": 0.0001
},
"teraTypes": {
  "Steel": 0.2633,
  "Ice": 0.1222,
  "Fighting": 0.1067,
  "Fire": 0.0953,
  "Poison": 0.0932,
  "Ground": 0.0922,
  "Water": 0.0743,
  "Ghost": 0.07,
  "Dragon": 0.0288,
  "Electric": 0.0144,
  "Fairy": 0.0132,
  "Dark": 0.0124,
  "Grass": 0.0068,
  "Rock": 0.0039,
  "Bug": 0.0014,
  "Flying": 0.0011,
  "Normal": 0.0005,
  "Stellar": 0.0004
},
"happinesses": {"255": 1},
"spreads": {
  "Jolly:0/252/4/0/0/252": 0.255,
  "Jolly:0/252/0/0/4/252": 0.2143,
  "Jolly:252/4/0/0/0/252": 0.1712,
  "Jolly:248/4/4/0/0/252": 0.0462,
  "Jolly:252/0/4/0/0/252": 0.0204,
  "Jolly:252/0/0/0/4/252": 0.0199,
  "Jolly:248/4/0/0/4/252": 0.0194,
  "Jolly:4/252/0/0/0/252": 0.011,
  "Adamant:0/252/0/0/4/252": 0.011,
  "Adamant:120/44/0/0/248/96": 0.0095,
  "Jolly:200/4/0/0/52/252": 0.0084,
  "Impish:252/0/252/0/4/0": 0.0083,
  "Impish:252/0/92/0/0/164": 0.0078,
  "Impish:252/0/252/0/0/4": 0.007,

```

"Adamant:248/252/0/0/0/8": 0.0064,
"Jolly:0/252/0/0/0/252": 0.0063,
"Impish:252/4/252/0/0/0": 0.0058,
"Adamant:160/132/0/0/12/204": 0.0054,
"Jolly:56/200/0/0/0/252": 0.005,
"Jolly:240/4/0/0/12/252": 0.004,
"Adamant:0/252/16/0/52/188": 0.004,
"Adamant:100/252/100/0/0/56": 0.0038,
"Impish:248/0/252/0/0/8": 0.0038,
"Adamant:8/252/0/0/64/184": 0.0037,
"Adamant:240/0/104/0/0/164": 0.0037,
"Impish:132/116/252/0/0/8": 0.0033,
"Jolly:0/204/0/0/52/252": 0.0031,
"Adamant:104/144/64/0/12/184": 0.0031,
"Impish:240/0/252/0/8/8": 0.0029,
"Adamant:252/88/0/0/0/168": 0.0028,
"Adamant:28/132/0/0/252/96": 0.0028,
"Adamant:0/252/0/0/92/164": 0.0025,
"Adamant:252/252/0/0/4/0": 0.0025,
"Adamant:4/252/0/0/0/252": 0.0025,
"Adamant:0/252/4/0/0/252": 0.0023,
"Jolly:104/140/0/0/12/252": 0.0022,
"Jolly:240/4/4/0/8/252": 0.0021,
"Jolly:248/0/4/0/4/252": 0.0019,
"Impish:248/0/252/0/8/0": 0.0018,
"Jolly:248/0/8/0/0/252": 0.0018,
"Jolly:248/0/0/0/8/252": 0.0017,
"Adamant:52/252/0/0/4/200": 0.0017,
"Jolly:0/252/24/0/0/232": 0.0016,
"Impish:252/0/56/0/0/200": 0.0015,
"Adamant:252/44/0/0/0/212": 0.0015,
"Jolly:184/68/0/0/4/252": 0.0015,
"Adamant:0/252/0/0/212/44": 0.0014,
"Jolly:0/4/0/0/252/252": 0.0014,
"Jolly:200/0/4/0/52/252": 0.0014,
"Jolly:112/144/0/0/0/252": 0.0014,
"Jolly:252/0/0/0/0/252": 0.0013,
"Jolly:8/108/0/0/140/252": 0.0013,
"Adamant:0/252/0/0/96/160": 0.0012,
"Adamant:116/252/0/0/4/136": 0.0012,

"Jolly:144/112/0/0/0/252": 0.0012,
"Jolly:48/208/0/0/0/252": 0.0011,
"Adamant:80/44/0/0/216/168": 0.0011,
"Adamant:0/184/0/0/112/212": 0.001,
"Impish:232/0/88/0/4/184": 0.001,
"Adamant:0/252/0/0/0/252": 0.001,
"Jolly:0/200/0/0/56/252": 0.001,
"Jolly:176/76/0/0/4/252": 0.0009,
"Adamant:216/252/0/0/0/40": 0.0009,
"Jolly:152/104/0/0/0/252": 0.0009,
"Jolly:72/4/4/0/176/252": 0.0009,
"Impish:252/0/244/0/0/12": 0.0009,
"Impish:248/8/252/0/0/0": 0.0008,
"Impish:248/8/248/0/0/4": 0.0008,
"Adamant:252/84/0/0/0/172": 0.0008,
"Jolly:48/220/0/0/0/240": 0.0008,
"Impish:248/0/8/0/244/8": 0.0008,
"Jolly:0/0/0/0/252/252": 0.0007,
"Jolly:80/172/4/0/0/252": 0.0007,
"Impish:232/0/88/0/0/188": 0.0007,
"Adamant:68/252/0/0/0/188": 0.0007,
"Impish:252/0/220/0/0/36": 0.0007,
"Adamant:248/252/0/0/8/0": 0.0007,
"Adamant:0/252/0/0/68/188": 0.0007,
"Jolly:56/0/4/0/196/252": 0.0006,
"Impish:180/0/232/0/0/96": 0.0006,
"Jolly:248/8/0/0/0/252": 0.0006,
"Jolly:44/128/84/0/0/252": 0.0006,
"Impish:252/4/204/0/0/48": 0.0006,
"Impish:252/0/248/0/0/8": 0.0006,
"Jolly:84/172/0/0/0/252": 0.0006,
"Impish:108/48/168/0/0/184": 0.0006,
"Jolly:0/196/0/0/60/252": 0.0006,
"Adamant:184/20/0/0/144/160": 0.0006,
"Jolly:0/224/0/0/32/252": 0.0006,
"Adamant:0/168/0/0/152/188": 0.0005,
"Jolly:168/0/0/0/88/252": 0.0005,
"Jolly:160/0/4/0/92/252": 0.0005,
"Impish:12/248/248/0/0/0": 0.0005,
"Hardy:0/252/4/0/0/252": 0.0005,

"Adamant:0/232/8/0/232/36": 0.0005,
"Adamant:252/120/40/0/0/96": 0.0005,
"Impish:248/0/92/0/0/168": 0.0005,
"Impish:252/0/80/0/0/176": 0.0005,
"Jolly:0/48/208/0/0/252": 0.0005,
"Jolly:52/252/0/0/4/200": 0.0005,
"Jolly:184/0/4/0/68/252": 0.0005,
"Jolly:244/4/4/0/4/252": 0.0005,
"Adamant:144/116/0/0/60/188": 0.0004,
"Impish:252/0/88/0/0/168": 0.0004,
"Jolly:2/252/2/0/0/252": 0.0004,
"Jolly:116/140/0/0/0/252": 0.0004,
"Adamant:32/48/0/0/252/176": 0.0004,
"Impish:252/60/12/0/0/184": 0.0004,
"Jolly:244/12/0/0/0/252": 0.0004,
"Impish:248/0/84/0/12/164": 0.0004,
"Adamant:248/100/0/0/0/160": 0.0004,
"Jolly:184/72/0/0/0/252": 0.0004,
"Adamant:0/252/0/0/72/184": 0.0004,
"Careful:252/4/0/0/252/0": 0.0004,
"Jolly:120/136/0/0/0/252": 0.0004,
"Impish:248/0/76/0/0/184": 0.0004,
"Impish:128/0/220/0/0/160": 0.0004,
"Impish:252/0/44/0/0/212": 0.0004,
"Impish:252/0/96/0/0/160": 0.0003,
"Impish:252/0/160/0/0/96": 0.0003,
"Jolly:0/248/8/0/0/252": 0.0003,
"Adamant:60/252/192/0/0/0": 0.0003,
"Adamant:104/236/0/0/0/168": 0.0003,
"Jolly:0/252/0/4/0/252": 0.0003,
"Adamant:120/88/0/0/96/204": 0.0003,
"Adamant:0/208/0/0/116/184": 0.0003,
"Jolly:0/4/252/0/0/252": 0.0003,
"Impish:252/0/216/0/0/40": 0.0003,
"Jolly:0/164/0/0/92/252": 0.0003,
"Adamant:0/124/0/0/196/188": 0.0003,
"Jolly:172/0/4/0/80/252": 0.0003,
"Adamant:216/48/4/0/0/240": 0.0003,
"Impish:248/0/100/0/0/160": 0.0003,
"Adamant:0/180/0/0/140/188": 0.0003,

"Jolly:0/208/0/0/48/252": 0.0003,
"Jolly:120/0/0/0/136/252": 0.0003,
"Jolly:60/0/0/0/196/252": 0.0003,
"Jolly:92/160/0/0/4/252": 0.0003,
"Jolly:128/124/0/0/4/252": 0.0003,
"Jolly:252/56/0/0/0/200": 0.0003,
"Adamant:0/208/0/0/96/200": 0.0003,
"Adamant:80/44/0/0/200/184": 0.0003,
"Adamant:184/168/0/0/112/44": 0.0003,
"Impish:248/0/80/0/0/180": 0.0003,
"Jolly:0/128/128/0/0/252": 0.0003,
"Jolly:140/116/0/0/0/252": 0.0002,
"Impish:252/0/196/0/0/60": 0.0002,
"Careful:96/0/0/0/252/160": 0.0002,
"Timid:112/140/0/0/4/252": 0.0002,
"Adamant:252/48/112/0/0/96": 0.0002,
"Jolly:244/0/12/0/0/252": 0.0002,
"Adamant:252/0/252/0/0/4": 0.0002,
"Jolly:0/212/0/0/44/252": 0.0002,
"Adamant:8/160/0/0/152/188": 0.0002,
"Impish:252/8/84/0/0/164": 0.0002,
"Adamant:152/252/0/0/0/104": 0.0002,
"Jolly:244/0/4/0/8/252": 0.0002,
"Adamant:208/88/4/0/112/96": 0.0002,
"Jolly:236/16/4/0/0/252": 0.0002,
"Impish:252/0/180/0/0/76": 0.0002,
"Jolly:104/160/0/0/4/240": 0.0002,
"Jolly:160/96/0/0/0/252": 0.0002,
"Naive:248/4/0/4/0/252": 0.0002,
"Jolly:32/224/0/0/0/252": 0.0002,
"Adamant:88/168/0/0/52/200": 0.0002,
"Jolly:168/88/0/0/0/252": 0.0002,
"Jolly:252/0/60/0/0/196": 0.0002,
"Jolly:40/80/112/0/24/252": 0.0002,
"Adamant:0/248/0/0/72/188": 0.0002,
"Jolly:240/16/0/0/0/252": 0.0002,
"Adamant:8/236/0/0/64/200": 0.0002,
"Adamant:0/236/0/0/20/252": 0.0002,
"Jolly:48/204/0/0/4/252": 0.0002,
"Adamant:0/252/0/0/88/168": 0.0002,

"Impish:252/0/104/0/0/152": 0.0002,
"Impish:220/0/252/0/0/36": 0.0002,
"Adamant:248/48/0/0/0/212": 0.0002,
"Impish:224/0/132/0/0/152": 0.0002,
"Jolly:16/240/0/0/0/252": 0.0002,
"Jolly:0/200/56/0/0/252": 0.0002,
"Naive:0/252/4/0/0/252": 0.0002,
"Jolly:160/0/0/0/96/252": 0.0002,
"Impish:252/0/120/0/0/136": 0.0002,
"Impish:248/0/4/0/0/252": 0.0002,
"Adamant:0/252/0/0/120/136": 0.0002,
"Adamant:0/252/0/0/80/176": 0.0002,
"Jolly:0/232/0/0/24/252": 0.0002,
"Jolly:0/0/0/0/0/252": 0.0002,
"Adamant:156/252/0/0/4/96": 0.0002,
"Adamant:216/124/0/0/0/168": 0.0002,
"Jolly:232/0/0/0/24/252": 0.0002,
"Impish:252/0/72/0/0/184": 0.0002,
"Jolly:0/160/96/0/0/252": 0.0002,
"Jolly:0/192/0/0/64/252": 0.0002,
"Adamant:0/136/0/0/208/164": 0.0002,
"Jolly:0/176/0/0/252/80": 0.0002,
"Impish:252/0/212/0/0/44": 0.0002,
"Jolly:244/4/4/0/0/252": 0.0002,
"Jolly:116/4/0/0/136/252": 0.0002,
"Jolly:0/96/0/0/160/252": 0.0001,
"Adamant:104/124/0/0/120/160": 0.0001,
"Jolly:188/0/0/0/244/72": 0.0001,
"Impish:248/0/224/0/0/36": 0.0001,
"Impish:252/0/140/0/0/116": 0.0001,
"Impish:200/0/148/0/0/160": 0.0001,
"Impish:248/0/220/0/0/40": 0.0001,
"Hardy:252/252/0/0/0/4": 0.0001,
"Impish:32/252/128/0/0/96": 0.0001,
"Jolly:148/108/0/0/0/252": 0.0001,
"Impish:248/0/96/0/0/164": 0.0001,
"Impish:160/80/52/0/12/204": 0.0001,
"Impish:252/0/208/0/0/48": 0.0001,
"Impish:252/0/60/0/0/196": 0.0001,
"Impish:248/4/252/0/0/4": 0.0001,

"Jolly:120/132/0/0/4/252": 0.0001,
"Adamant:72/252/0/0/0/184": 0.0001,
"Jolly:200/0/0/0/56/252": 0.0001,
"Adamant:56/88/0/0/180/184": 0.0001,
"Adamant:160/252/0/0/0/96": 0.0001,
"Jolly:156/100/0/0/0/252": 0.0001,
"Impish:248/0/184/0/0/76": 0.0001,
"Adamant:252/252/4/0/0/0": 0.0001,
"Jolly:100/144/0/0/12/252": 0.0001,
"Adamant:0/128/0/0/216/164": 0.0001,
"Jolly:72/180/0/0/4/252": 0.0001,
"Impish:4/252/252/0/0/0": 0.0001,
"Careful:252/44/0/0/212/0": 0.0001,
"Adamant:156/140/0/0/0/212": 0.0001,
"Jolly:240/0/0/0/16/252": 0.0001,
"Impish:248/0/164/0/0/96": 0.0001,
"Adamant:252/40/0/0/156/60": 0.0001,
"Jolly:0/176/252/0/0/80": 0.0001,
"Adamant:252/252/0/0/0/4": 0.0001,
"Adamant:248/88/0/0/0/172": 0.0001,
"Jolly:0/208/4/0/0/252": 0.0001,
"Jolly:0/168/144/0/0/196": 0.0001,
"Jolly:0/0/4/0/252/252": 0.0001,
"Jolly:236/0/200/0/0/72": 0.0001,
"Jolly:160/16/0/0/80/252": 0.0001,
"Adamant:212/124/0/0/128/44": 0.0001,
"Jolly:192/60/0/0/0/252": 0.0001,
"Jolly:128/128/0/0/0/252": 0.0001,
"Impish:200/88/88/0/12/120": 0.0001,
"Impish:252/0/4/0/0/252": 0.0001,
"Careful:240/8/0/0/252/8": 0.0001,
"Jolly:0/124/132/0/0/252": 0.0001,
"Jolly:0/208/48/0/0/252": 0.0001,
"Impish:0/4/252/0/0/252": 0.0001,
"Jolly:188/0/0/0/248/72": 0.0001,
"Impish:252/44/212/0/0/0": 0.0001,
"Hardy:244/12/0/0/0/252": 0.0001,
"Impish:252/0/252/0/0/0": 0.0001,
"Jolly:136/8/112/0/0/252": 0.0001,
"Adamant:56/252/0/0/0/200": 0.0001,

"Impish:252/0/236/0/0/20": 0.0001,
"Careful:252/24/32/0/200/0": 0.0001,
"Adamant:96/252/0/0/0/160": 0.0001,
"Adamant:132/160/0/0/12/204": 0.0001,
"Jolly:252/0/56/0/0/200": 0.0001,
"Adamant:0/168/0/0/244/96": 0.0001,
"Adamant:0/208/0/0/52/248": 0.0001,
"Careful:252/0/4/0/252/0": 0.0001,
"Jolly:240/4/12/0/0/252": 0.0001,
"Adamant:0/252/96/0/0/100": 0.0001,
"Jolly:244/0/0/0/12/252": 0.0001,
"Jolly:0/128/0/0/128/252": 0.0001,
"Jolly:4/228/12/0/12/252": 0.0001,
"Adamant:48/208/0/0/0/252": 0.0001,
"Impish:252/0/200/0/0/56": 0.0001,
"Jolly:0/244/4/0/8/252": 0.0001,
"Jolly:4/220/0/0/32/252": 0.0001,
"Jolly:216/8/212/0/36/36": 0.0001,
"Jolly:252/44/0/0/0/212": 0.0001,
"Impish:240/0/252/0/0/16": 0.0001,
"Jolly:216/4/0/0/36/252": 0.0001,
"Adamant:240/0/108/0/0/160": 0.0001,
"Impish:252/56/200/0/0/0": 0.0001,
"Adamant:212/128/0/0/0/168": 0.0001,
"Adamant:120/124/0/0/104/160": 0.0001,
"Impish:252/108/148/0/0/0": 0.0001,
"Jolly:236/16/0/0/4/252": 0.0001,
"Jolly:252/4/252/0/0/0": 0.0001,
"Adamant:248/8/88/0/0/164": 0.0001,
"Impish:248/8/88/0/0/164": 0.0001,
"Jolly:188/0/0/0/132/188": 0.0001,
"Adamant:252/4/0/0/0/252": 0.0001,
"Jolly:252/0/96/0/0/160": 0.0001,
"Adamant:252/252/0/0/0/0": 0.0001,
"Adamant:0/252/176/0/0/80": 0.0001,
"Jolly:16/252/0/0/0/240": 0.0001,
"Impish:248/108/88/0/0/64": 0.0001,
"Impish:216/104/148/0/0/40": 0.0001,
"Jolly:64/252/0/0/0/192": 0.0001,
"Adamant:244/88/0/0/0/176": 0.0001,

"Adamant:252/4/0/0/252/0": 0.0001,
"Impish:252/0/208/0/48/0": 0.0001,
"Jolly:0/252/0/0/64/192": 0.0001,
"Impish:200/0/116/0/152/40": 0.0001,
"Jolly:0/76/180/0/0/252": 0.0001,
"Jolly:184/0/0/0/72/252": 0.0001,
"Impish:252/4/92/0/0/160": 0.0001,
"Impish:124/4/220/0/0/160": 0.0001,
"Jolly:88/196/0/0/0/224": 0.0001,
"Serious:4/252/0/0/0/252": 0.0001,
"Jolly:248/64/0/0/4/188": 0.0001,
"Jolly:60/144/0/0/52/252": 0.0001,
"Impish:216/0/252/0/0/40": 0.0001,
"Adamant:252/96/0/0/0/160": 0.0001,
"Jolly:0/252/56/0/0/200": 0.0001,
"Careful:48/160/0/0/184/116": 0.0001,
"Impish:252/168/88/0/0/0": 0.0001,
"Jolly:0/220/0/0/36/252": 0.0001,
"Adamant:252/192/0/0/64/0": 0.0001,
"Jolly:4/0/252/0/0/252": 0.0001,
"Adamant:128/252/0/0/4/124": 0.0001,
"Jolly:140/104/0/0/12/252": 0.0001,
"Impish:252/0/204/0/0/52": 0.0001,
"Adamant:88/168/0/0/0/252": 0.0001,
"Jolly:144/0/0/0/112/252": 0.0001,
"Jolly:48/144/32/0/32/252": 0.0001,
"Jolly:252/96/0/0/0/160": 0.0001,
"Adamant:204/204/0/0/4/96": 0.0001,
"Jolly:20/0/4/0/232/252": 0.0001,
"Jolly:0/144/0/0/112/252": 0.0001,
"Adamant:0/156/8/0/140/204": 0.0001,
"Impish:248/32/44/0/0/184": 0.0001,
"Adamant:252/0/252/0/4/0": 0.0001,
"Jolly:56/252/0/0/0/200": 0.0001,
"Impish:88/168/252/0/0/0": 0.0001,
"Adamant:64/12/0/0/248/184": 0.0001,
"Jolly:0/140/0/0/116/252": 0.0001,
"Adamant:184/8/0/0/252/64": 0.0001,
"Adamant:4/252/252/0/0/0": 0.0001,
"Jolly:0/36/220/0/0/252": 0.0001,

```

"Adamant:248/8/0/0/88/164": 0.0001,
"Adamant:56/252/0/0/4/196": 0.0001,
"Jolly:132/0/124/0/0/252": 0.0001,
"Jolly:52/252/0/0/0/204": 0.0001,
"Adamant:252/68/0/0/0/188": 0.0001,
"Careful:248/48/68/0/144/0": 0.0001,
"Jolly:24/228/0/0/4/252": 0.0001,
"Impish:120/0/252/0/0/136": 0.0001,
"Adamant:4/252/0/0/252/0": 0.0001
},
"moves": {
  "Headlong Rush": 0.9088,
  "Rapid Spin": 0.9087,
  "Ice Spinner": 0.8024,
  "Stealth Rock": 0.3595,
  "Close Combat": 0.3253,
  "Bulk Up": 0.2097,
  "Knock Off": 0.1521,
  "Head Smash": 0.0943,
  "Earthquake": 0.0707,
  "Taunt": 0.0423,
  "Temper Flare": 0.0363,
  "Stone Edge": 0.0283,
  "Body Press": 0.0242,
  "Supercell Slam": 0.0151,
  "Rock Slide": 0.0105,
  "Smack Down": 0.0031,
  "Roar": 0.0027,
  "Tera Blast": 0.0012,
  "High Horsepower": 0.0012,
  "Endeavor": 0.001,
  "Substitute": 0.0006,
  "Play Rough": 0.0003,
  "Heavy Slam": 0.0003,
  "Iron Head": 0.0003,
  "Brick Break": 0.0003,
  "Megahorn": 0.0002,
  "Sunny Day": 0.0002,
  "Rest": 0.0001,
  "Protect": 0.0001,

```

```
"Ice Fang": 0.0001
},
"teammates": {
  "Kingambit": 0.27,
  "Dragonite": 0.2293,
  "Gholdengo": 0.2259,
  "Raging Bolt": 0.1721,
  "Ogerpon-Wellspring": 0.1633,
  "Slowking-Galar": 0.1617,
  "Dragapult": 0.1601,
  "Hatterene": 0.1537,
  "Iron Valiant": 0.1427,
  "Kyurem": 0.1424,
  "Pecharunt": 0.1289,
  "Walking Wake": 0.1142,
  "Alomomola": 0.1091,
  "Samurott-Hisui": 0.107,
  "Iron Moth": 0.0952,
  "Cerulede": 0.0914,
  "Darkrai": 0.0868,
  "Tornadus-Therian": 0.0866,
  "Glimmora": 0.0841,
  "Cinderace": 0.0832,
  "Rillaboom": 0.0808,
  "Zamazenta": 0.0804,
  "Zapdos": 0.0788,
  "Deoxys-Speed": 0.0758,
  "Iron Crown": 0.0757,
  "Heatran": 0.0737,
  "Corviknight": 0.0726,
  "Enamorus": 0.0708,
  "Moltres": 0.0643,
  "Ninetales": 0.0635,
  "Garganac": 0.0612,
  "Ogerpon": 0.0591,
  "Ting-Lu": 0.0589,
  "Primarina": 0.0577,
  "Ogerpon-Cornerstone": 0.0503,
  "Torkoal": 0.0496,
  "Tinkaton": 0.0485,
```

"Ribombee": 0.0448,
"Venusaur": 0.0411,
"Landorus-Therian": 0.0376,
"Meowscarada": 0.0364,
"Tyranitar": 0.0329,
"Lokix": 0.0303,
"Scizor": 0.0299,
"Hydrapple": 0.0271,
"Hydreigon": 0.0268,
"Araquanid": 0.0261,
"Hoopa-Unbound": 0.0257,
"Volcanion": 0.0218,
"Gliscor": 0.0217,
"Ninetales-Alola": 0.0216,
"Clefable": 0.0203,
"Weavile": 0.0186,
"Slither Wing": 0.0172,
"Cresselia": 0.0171,
"Serperior": 0.0171,
"Weezing-Galar": 0.017,
"Arcanine-Hisui": 0.0164,
"Grimmsnarl": 0.0159,
"Latos": 0.015,
"Garchomp": 0.0143,
"Mandibuzz": 0.0126,
"Moltres-Galar": 0.0123,
"Dondozo": 0.0123,
"Manaphy": 0.0123,
"Skeledirge": 0.0111,
"Sinistcha": 0.011,
"Lilligant-Hisui": 0.0107,
"Comfey": 0.0097,
"Jolteon": 0.0096,
"Keldeo": 0.0096,
"Skarmory": 0.0095,
"Polteageist": 0.0088,
"Rotom-Wash": 0.0082,
"Frosmoth": 0.008,
"Goodra-Hisui": 0.0075,
"Clodsire": 0.0072,

"Zoroark-Hisui": 0.0064,
"Toxapex": 0.0064,
"Zarude": 0.006,
"Blaziken": 0.0057,
"Iron Boulder": 0.0056,
"Azumarill": 0.0055,
"Whimsicott": 0.0055,
"Magnezone": 0.0052,
"Fezandipiti": 0.0052,
"Wo-Chien": 0.0051,
"Okidogi": 0.0051,
"Iron Hands": 0.0051,
"Pelipper": 0.005,
"Latias": 0.005,
"Amoonguss": 0.0047,
"Haxorus": 0.0046,
"Breloom": 0.0043,
"Blissey": 0.0043,
"Thundurus-Therian": 0.0042,
"Hawlucha": 0.004,
"Greninja": 0.0038,
"Indeedee": 0.0033,
"Ursaluna": 0.0032,
"Gengar": 0.0028,
"Farigiraf": 0.0026,
"Armarouge": 0.0024,
"Metagross": 0.0024,
"Iron Treads": 0.0024,
"Braviary-Hisui": 0.0023,
"Thundurus": 0.0023,
"Leafeon": 0.0022,
"Mimikyu": 0.0022,
"Chesnaught": 0.0021,
"Charizard": 0.0021,
"Kilowattrel": 0.002,
"Sableye": 0.0019,
"Rotom-Heat": 0.0018,
"Zapdos-Galar": 0.0018,
"Probopass": 0.0018,
"Salamence": 0.0017,

"Excadrill": 0.0017,
"Chansey": 0.0017,
"Smeargle": 0.0016,
"Porygon-Z": 0.0016,
"Dragalge": 0.0016,
"Barraskewda": 0.0016,
"Mew": 0.0016,
"Sylveon": 0.0016,
"Muk-Alola": 0.0016,
"Scovillain": 0.0015,
"Enamorus-Therian": 0.0015,
"Regice": 0.0014,
"Cryogonal": 0.0013,
"Beartic": 0.0013,
"Empoleon": 0.0013,
"Typhlosion-Hisui": 0.0013,
"Ditto": 0.0012,
"Reuniclus": 0.0012,
"Kleavor": 0.0012,
"Persian-Alola": 0.0012,
"Crawdaunt": 0.0011,
"Iron Jugulis": 0.0011,
"Scream Tail": 0.0011,
"Golem-Alola": 0.0011,
"Jirachi": 0.0011,
"Dudunsparce": 0.001,
"Chandelure": 0.001,
"Leavanny": 0.001,
"Sandy Shocks": 0.001,
"Slowking": 0.001,
"Entei": 0.0009,
"Mamoswine": 0.0009,
"Blastoise": 0.0009,
"Salazzle": 0.0009,
"Vikavolt": 0.0009,
"Ambipom": 0.0009,
"Maushold": 0.0009,
"Basculegion": 0.0008,
"Vileplume": 0.0008,
"Necrozma": 0.0008,

"Cetitan": 0.0008,
"Kommo-o": 0.0007,
"Gallade": 0.0007,
"Iron Thorns": 0.0006,
"Lucario": 0.0006,
"Galvantula": 0.0006,
"Kingdra": 0.0006,
"Talonflame": 0.0005,
"Brute Bonnet": 0.0005,
"Bronzong": 0.0005,
"Espeon": 0.0005,
"Cloyster": 0.0005,
"Toedscruel": 0.0005,
"Iron Leaves": 0.0004,
"Drifblim": 0.0004,
"Sudowoodo": 0.0004,
"Glaceon": 0.0004,
"Regidrago": 0.0004,
"Delibird": 0.0004,
"Heracross": 0.0004,
"Ampharos": 0.0004,
"Pawmot": 0.0003,
"Umbreon": 0.0003,
"Incineroar": 0.0003,
"Bellibolt": 0.0003,
"Overqwil": 0.0003,
"Basculegion-F": 0.0003,
"Gyarados": 0.0003,
"Tentacruel": 0.0003,
"Conkeldurr": 0.0003,
"Scrafty": 0.0003,
"Porygon2": 0.0003,
"Suicune": 0.0003,
"Golurk": 0.0003,
"Pignite": 0.0003,
"Shiftry": 0.0003,
"Yanmega": 0.0003,
"Snorlax": 0.0003,
"Terrakion": 0.0002,
"Gardevoir": 0.0002,

"Quaquaval": 0.0002,
"Meloetta": 0.0002,
"Brambleghast": 0.0002,
"Slowbro-Galar": 0.0002,
"Slowbro": 0.0002,
"Mesprit": 0.0002,
"Torterra": 0.0002,
"Magmortar": 0.0002,
"Inteleon": 0.0002,
"Pyroar": 0.0002,
"Toxtricity": 0.0002,
"Raikou": 0.0002,
"Milotic": 0.0002,
"Sandslash-Alola": 0.0002,
"Lilligant": 0.0002,
"Slaking": 0.0002,
"Arboliva": 0.0002,
"Mienshao": 0.0002,
"Exeggutor": 0.0002,
"Klawf": 0.0002,
"Victreebel": 0.0002,
"Tornadus": 0.0001,
"Azelf": 0.0001,
"Lycanroc-Dusk": 0.0001,
"Mabosstiff": 0.0001,
"Articuno": 0.0001,
"Klefki": 0.0001,
"Sawsbuck": 0.0001,
"Rotom-Mow": 0.0001,
"Veluza": 0.0001,
"Bisharp": 0.0001,
"Glastrier": 0.0001,
"Regigigas": 0.0001,
"Uxie": 0.0001,
"Alcremie": 0.0001,
"Banette": 0.0001,
"Lycanroc-Midnight": 0.0001,
"Cramorant": 0.0001,
"Sneasel-Hisui": 0.0001,
"Swampert": 0.0001,

"Ludicolo": 0.0001,
"Tsareena": 0.0001,
"Infernape": 0.0001,
"Medicham": 0.0001,
"Quagsire": 0.0001,
"Vaporeon": 0.0001,
"Rotom-Frost": 0.0001,
"Spiritomb": 0.0001,
"Dusknoir": 0.0001,
"Eiscue": 0.0001,
"Hitmonlee": 0.0001,
"Trevenant": 0.0001,
"Deoxys-Defense": 0.0001,
"Luxray": 0.0001,
"Orthworm": 0.0001,
"Staraptor": 0.0001,
"Diancie": 0.0001,
"Exeggutor-Alola": 0.0001,
"Revavroom": 0.0001,
"Arcanine": 0.0001,
"Hoopa": 0.0001,
"Gastrodon": 0.0001,
"Flygon": 0.0001,
"Grafaiai": 0.0001,
"Goodra": 0.0001,
"Tauros-Paldea-Blaze": 0.0001,
"Munkidori": 0.0001,
"Feraligatr": 0.0001,
"Avalugg": 0.0001,
"Cacturne": 0.0001,
"Sandshrew-Alola": 0.0001,
"Oricorio-Sensu": 0.0001,
"Poliwrath": 0.0001,
"Pincurchin": 0.0001,
"Electivire": 0.0001,
"Honchkrow": 0.0001,
"Minior": 0.0001,
"Houndstone": 0.0001,
"Bombirdier": 0.0001
},

```

"counters": {
  "Iron Valiant": [1763.446, 0.357, 0.428],
  "Latias": [153.9435, 0.289, 0.544],
  "Walking Wake": [862.7487, 0.329, 0.436],
  "Sinistcha": [485.3436, 0.123, 0.647],
  "Enamorus": [741.9198, 0.3, 0.444],
  "Manaphy": [180.8768, 0.413, 0.386],
  "Serperior": [159.0813, 0.288, 0.506],
  "Clefable": [827.4475, 0.246, 0.48],
  "Ogerpon-Wellspring": [2299.1152, 0.254, 0.437],
  "Kyurem": [1644.4946, 0.341, 0.34],
  "Cresselia": [234.5168, 0.122, 0.626],
  "Deoxys-Speed": [665.2029, 0.228, 0.465]
},
},
"Kingambit": {
  "lead": {"raw": 0.0037, "real": 0.0037, "weighted": 0.0054},
  "usage": {"raw": 0.1947, "real": 0.167, "weighted": 0.2488},
  "count": 425975,
  "weight": 0.0875,
  "viability": [29484, 91, 81, 63],
  "abilities": {
    "Supreme Overlord": 0.9853,
    "Defiant": 0.0144,
    "Pressure": 0.0003
  },
},
"items": {
  "Leftovers": 0.5389,
  "Black Glasses": 0.1765,
  "Air Balloon": 0.1358,
  "Lum Berry": 0.0623,
  "Shuca Berry": 0.0171,
  "Assault Vest": 0.016,
  "Heavy-Duty Boots": 0.0087,
  "Dread Plate": 0.0081,
  "Life Orb": 0.008,
  "Clear Amulet": 0.0067,
  "Covert Cloak": 0.0047,
  "Choice Band": 0.0031,
  "Mental Herb": 0.0022,

```

```

"Black Belt": 0.002,
"Rocky Helmet": 0.0017,
"Fist Plate": 0.0016,
"Sitrus Berry": 0.0015,
"Focus Sash": 0.0009,
"Weakness Policy": 0.0009,
"Quick Claw": 0.0009,
"Chople Berry": 0.0005,
"Expert Belt": 0.0003,
"Choice Scarf": 0.0003,
"Metal Coat": 0.0002,
"Grassy Seed": 0.0002,
"Rawst Berry": 0.0002,
"Soft Sand": 0.0001,
"Nothing": 0.0001,
"Muscle Band": 0.0001
},
"teraTypes": {
  "Ghost": 0.2986,
  "Dark": 0.2267,
  "Fairy": 0.2091,
  "Fighting": 0.104,
  "Fire": 0.0814,
  "Flying": 0.0545,
  "Grass": 0.0168,
  "Bug": 0.0051,
  "Ground": 0.0018,
  "Water": 0.0008,
  "Poison": 0.0004,
  "Dragon": 0.0004,
  "Steel": 0.0003,
  "Psychic": 0.0001
},
"happinesses": {"255": 1},
"spreads": {
  "Adamant:0/252/4/0/0/252": 0.1516,
  "Jolly:0/252/4/0/0/252": 0.09,
  "Adamant:0/252/0/0/4/252": 0.0774,
  "Jolly:0/252/0/0/4/252": 0.0769,
  "Adamant:240/252/0/0/0/16": 0.0521,

```

"Adamant:252/252/0/0/4/0": 0.0455,
"Adamant:212/252/0/0/0/44": 0.0416,
"Adamant:236/252/0/0/0/20": 0.033,
"Adamant:112/252/0/0/0/144": 0.0231,
"Adamant:116/252/0/0/0/140": 0.0222,
"Adamant:248/252/8/0/0/0": 0.0219,
"Adamant:160/252/0/0/0/96": 0.0218,
"Jolly:4/252/0/0/0/252": 0.0203,
"Adamant:240/248/0/0/0/20": 0.0194,
"Adamant:248/252/0/0/0/8": 0.0193,
"Adamant:252/252/0/0/0/4": 0.0154,
"Adamant:4/252/0/0/0/252": 0.014,
"Adamant:0/252/0/0/0/252": 0.0127,
"Adamant:164/252/0/0/0/92": 0.0123,
"Adamant:232/252/0/0/0/24": 0.0113,
"Adamant:252/252/4/0/0/0": 0.01,
"Brave:252/252/4/0/0/0": 0.0093,
"Adamant:80/252/0/0/0/176": 0.0078,
"Adamant:204/252/0/0/0/52": 0.0077,
"Adamant:140/252/0/0/0/112": 0.0075,
"Adamant:200/252/0/0/0/56": 0.0073,
"Brave:252/252/0/0/4/0": 0.007,
"Adamant:172/252/0/0/0/84": 0.0061,
"Adamant:160/252/0/0/4/92": 0.006,
"Adamant:208/252/0/0/0/48": 0.006,
"Adamant:240/252/4/0/0/12": 0.0056,
"Adamant:92/252/0/0/0/164": 0.0052,
"Adamant:232/252/4/0/0/20": 0.0052,
"Adamant:156/252/0/0/0/100": 0.0051,
"Adamant:228/252/0/0/4/24": 0.0045,
"Adamant:108/252/0/0/0/148": 0.0044,
"Adamant:224/252/0/0/0/32": 0.0039,
"Adamant:168/252/0/0/0/88": 0.0037,
"Adamant:192/216/0/0/0/100": 0.0031,
"Adamant:248/244/0/0/0/16": 0.0028,
"Adamant:248/252/0/0/8/0": 0.0028,
"Adamant:244/252/0/0/0/12": 0.0027,
"Adamant:152/252/0/0/0/104": 0.0024,
"Adamant:120/252/0/0/0/136": 0.0023,
"Adamant:248/124/0/0/48/88": 0.0022,

"Adamant:0/252/0/0/248/8": 0.002,
"Adamant:240/244/0/0/0/24": 0.0019,
"Adamant:112/252/0/0/4/140": 0.0019,
"Brave:248/252/8/0/0/0": 0.0018,
"Adamant:104/252/0/0/152/0": 0.0017,
"Adamant:88/252/0/0/0/168": 0.0017,
"Adamant:116/136/0/0/0/252": 0.0017,
"Adamant:208/216/0/0/0/84": 0.0017,
"Adamant:120/136/0/0/0/252": 0.0017,
"Adamant:216/196/0/0/0/96": 0.0015,
"Adamant:176/92/4/0/168/68": 0.0014,
"Adamant:188/216/0/0/0/104": 0.0013,
"Adamant:212/252/4/0/0/40": 0.0013,
"Adamant:196/252/0/0/0/60": 0.0012,
"Adamant:96/252/0/0/0/160": 0.0012,
"Adamant:252/160/0/0/0/96": 0.0011,
"Adamant:52/252/4/0/0/200": 0.0011,
"Careful:96/116/0/0/252/44": 0.0011,
"Adamant:0/252/0/0/252/4": 0.0011,
"Adamant:156/252/0/0/4/96": 0.001,
"Adamant:88/252/4/0/0/164": 0.001,
"Adamant:160/252/0/0/0/92": 0.001,
"Brave:252/252/0/4/0/0": 0.001,
"Adamant:0/252/0/4/0/252": 0.001,
"Jolly:80/240/0/0/0/188": 0.001,
"Adamant:228/252/0/0/0/28": 0.0009,
"Adamant:128/128/0/0/0/252": 0.0009,
"Adamant:248/164/0/0/0/96": 0.0009,
"Jolly:64/252/0/0/0/192": 0.0009,
"Jolly:0/252/0/0/0/252": 0.0009,
"Adamant:128/252/4/0/0/124": 0.0008,
"Adamant:144/252/0/0/0/112": 0.0008,
"Adamant:248/160/0/0/0/100": 0.0008,
"Adamant:116/252/4/0/0/136": 0.0008,
"Adamant:140/252/0/0/0/116": 0.0008,
"Adamant:168/252/4/0/0/84": 0.0008,
"Brave:248/252/0/0/8/0": 0.0008,
"Adamant:168/252/0/0/4/84": 0.0007,
"Adamant:248/252/0/0/0/0": 0.0007,
"Adamant:208/192/0/0/104/4": 0.0007,

```
"Adamant:20/252/0/0/0/234": 0.0007,  
"Adamant:48/252/0/0/8/200": 0.0007,  
"Adamant:248/252/0/0/4/0": 0.0007,  
"Adamant:192/200/0/0/0/116": 0.0007,  
"Adamant:32/252/0/0/0/224": 0.0006,  
"Adamant:252/252/0/0/0/0": 0.0006,  
"Adamant:32/252/0/0/4/220": 0.0006,  
"
```

Gemini 3 Flash Thinking

To convert your JSON data into a pandas DataFrame with just the **Pokemon Name** and the **Tier**, you can use a simple loop to iterate through the keys of the "pokemon" object.

Since you have separate files for each tier, the most efficient way is to process them in a loop and append the results to a list.

Python Solution

python

```
import pandas as pd  
import json  
  
# List of your JSON files and their corresponding tiers  
# Example: {'tier_name': 'path_to_json_file'}  
files = {  
    "OU": "ou_data.json",  
    "UU": "uu_data.json",  
    "Ubers": "ubers_data.json"  
}  
  
all_pokemon = []  
  
for tier, file_path in files.items():  
    # Load the JSON data  
    with open(file_path, 'r') as f:  
        data = json.load(f)  
  
    # Extract the names from the "pokemon" key  
    # .keys() gives us ["Great Tusk", "Kingambit", ...]  
    pokemon_names = data.get("pokemon", {}).keys()  
  
    for name in pokemon_names:
```

```

    all_pokemon.append({
        "Pokemon": name,
        "Tier": tier
    })

# Create the DataFrame
df = pd.DataFrame(all_pokemon)

# Display the result
print(df.head())

```

How it works:

1. **data.get("pokemon", {}).keys()** : This accesses the nested "pokemon" dictionary. The `.keys()` method pulls only the names (like "Great Tusk") and ignores the large blocks of move/item/stat data inside.
2. **The List of Dicts:** We create a list of small dictionaries like `{"Pokemon": "Great Tusk", "Tier": "OU"}` . This is significantly faster than appending rows to a DataFrame one by one.
3. **DataFrame Constructor:** `pd.DataFrame(all_pokemon)` converts that list into your final table instantly.

If your JSON is already loaded as a Python Dictionary

If you aren't reading from files but already have the variables in memory (e.g., `ou_json` , `uu_json`), you can do this:

```

python

# Assuming ou_json and uu_json are your loaded dictionaries
data_map = {"OU": ou_json, "UU": uu_json}

df = pd.DataFrame([
    {"Pokemon": name, "Tier": tier}
    for tier, json_data in data_map.items()
    for name in json_data["pokemon"].keys()
])

```

Resulting DataFrame Structure:

	Pokemon	Tier
0	Great Tusk	OU
1	Kingambit	OU

	Pokemon	Tier
2	...	...

How many different tier files are you looking to combine into this single DataFrame?

2026-05-03 11:54:27 You Asked 

How can I
add a new column to a dataframe, moreover how can I change the
order of columns. I want to keep tier as the rightmost column bc it
is my label column

 Gemini 3 Flash

Adding a new column and reordering them is quite straightforward in `pandas`. Since you want **Tier** to be your label (the target variable), keeping it at the far right is a common convention for machine learning datasets.

1. Adding a New Column

If you already have a DataFrame and want to add a column with a constant value (or based on some logic), you can simply assign it like a dictionary key:

python

```
# To add a column with a single value for every row
df['New_Column'] = 'Some Value'

# Or, if you have a list of values matching the length of the DataFrame
df['New_Column'] = [1, 2, 3, 4, ...]
```

2. Reordering Columns

To change the order, you pass a list of the column names in your preferred order back into the DataFrame.

python

```
# Define your desired order
column_order = ['Pokemon', 'Usage', 'Ability', 'Tier']
```

```
# Re-assign the dataframe with the new order
df = df[column_order]
```

Putting it together for your Pokemon Data

Assuming you are building the DataFrame row by row as we discussed previously, here is how you would ensure the **Tier** is on the right:

python

```
import pandas as pd

# Let's say you already have your list of data
data = [
    {'Pokemon': 'Great Tusk', 'Usage': 0.3552, 'Tier': 'OU'},
    {'Pokemon': 'Kingambit', 'Usage': 0.2488, 'Tier': 'OU'}
]

df = pd.DataFrame(data)

# 1. Adding a new column (e.g., 'Generation')
df['Generation'] = 9

# 2. Reordering so 'Tier' is the label (rightmost)
# We get all columns, remove 'Tier', then put 'Tier' at the end
cols = [c for c in df.columns if c != 'Tier'] + ['Tier']
df = df[cols]

print(df)
```

Pro-Tip: The `.insert()` method

If you want to add a column at a **specific position** without re-writing the whole order list, you can use `.insert()` :

python

```
# df.insert(index_position, 'column_name', value)
df.insert(0, 'ID', range(1, len(df) + 1))
# This puts an ID column at the very beginning (index 0)
```

Why put the label on the right?

As you mentioned, keeping the "Label" or "Target" as the last column is a standard practice because:

- It's easier to slice the features: `X = df.iloc[:, :-1]` (everything except the last column).

- It's easier to grab the label: `y = df.iloc[:, -1]` .

Are you planning to add any of the other stats from the JSON (like Usage or Abilities) as features before you start training your model?

2026-05-03 12:08:14

You Asked



```
tauros
tauros-paldea-combat-breed
tauros-paldea-blaze-breed
tauros-paldea-aqua-breed write a quick string method to remove the
breed part
```



Gemini 3 Flash

```
python
```



```
names = ["tauros", "tauros-paldea-combat-breed", "tauros-paldea-blaze-breed", "tauros-paldea-aqua-breed"]

# Using removesuffix (Python 3.9+)
cleaned_names = [name.removesuffix("-breed") for name in names]

print(f"Original: {names}")
print(f"Cleaned: {cleaned_names}")
```

text

```
Original: ['tauros', 'tauros-paldea-combat-breed', 'tauros-paldea-blaze-breed', 'tauros-paldea-aqua-breed']
Cleaned: ['tauros', 'tauros-paldea-combat', 'tauros-paldea-blaze', 'tauros-paldea-aqua']
```

The most efficient and "clean" way to do this in Python (version 3.9 and later) is using the `.removesuffix()` method. This ensures you only remove the string if it appears at the end, preventing accidental deletions in the middle of a name.

1. Using `.removesuffix()` (Recommended)

This is the safest method for handling suffixes in names.

python

```
names = [
    "tauros",
    "tauros-paldea-combat-breed",
    "tauros-paldea-blaze-breed",
    "tauros-paldea-aqua-breed"
]

# Quick list comprehension
cleaned_names = [name.removesuffix("-breed") for name in names]

print(cleaned_names)
# Result: ['tauros', 'tauros-paldea-combat', 'tauros-paldea-blaze', 'tauros-paldea-aqua']
```

2. Using `.replace()` (Alternative)

If you are on an older version of Python or want to remove "-breed" anywhere it might appear, you can use `.replace()` :

python

```
cleaned_name = name.replace("-breed", "")
```

3. Applying it to your DataFrame

If you are processing your Pokemon JSON into a DataFrame, you can apply this logic to the column directly:

python

```
# Assuming 'Pokemon' is your column name
df['Pokemon'] = df['Pokemon'].str.removesuffix("-breed")
```

Why use `.removesuffix()` over `.replace()` ?

If a Pokemon were ever named something like "breed-special-breed", `.replace("-breed", "")` would turn it into "special", whereas `.removesuffix("-breed")` would correctly leave it as "breed-special". In the context of Pokemon data, `removesuffix` is almost always the better choice.

2026-05-03 12:10:37 You Asked 

can i pass an array to remove suffix instead?

Gemini 3 Flash

Unfortunately, the standard `.removesuffix()` method only accepts a **single string**, not a list or an array. If you try to pass an array, Python will throw a `TypeError`.

However, depending on what you're trying to achieve, there are two "pro" ways to handle multiple suffixes efficiently:

1. If you have multiple suffixes to remove

If you want to remove "-breed", "-mega", or "-alola" all in one go, the most Pythonic way is to loop through your list of suffixes:

python

```
names = ["tauros-paldea-blaze-breed", "charizard-mega-y", "raichu-alola"]
suffixes = ["-breed", "-mega-y", "-alola"]

cleaned = []
for name in names:
    for s in suffixes:
```

```
name = name.removesuffix(s)
cleaned.append(name)
```

2. If you are using Pandas (Recommended)

Since you are working with a DataFrame, you can use **Regular Expressions (Regex)** with the `.str.replace()` method. This allows you to pass a "pattern" (which acts like an array) to clean the whole column at once.

Use the pipe `|` symbol to mean "OR":

python

```
# This tells pandas: find "-breed" OR "-mega" OR "-alola" at the end of the string ($)
suffixes_to_remove = r'(-breed|-mega|-alola)$'

df['Pokemon'] = df['Pokemon'].str.replace(suffixes_to_remove, '', regex=True)
```

3. If you just want to clean a Python List quickly

If you don't want a nested loop, you can use a quick helper function:

python

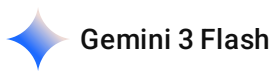
```
def clean_names(name_list, suffix_list):
    # Joins suffixes into a regex pattern
    import re
    pattern = r'(' + '|'.join(suffix_list) + ')$'
    return [re.sub(pattern, '', name) for name in name_list]

# Usage
my_suffixes = ["-breed", "-paldea"]
cleaned = clean_names(names, my_suffixes)
```

Which one should you use?

- If you only have **one** suffix: Stick to `.removesuffix("-breed")` .
- If you have **many** suffixes and you're already in a DataFrame: Use the **Pandas Regex** method in Option 2. It's significantly faster for large datasets!

Are there other specific suffixes in the Smogon JSONs (like `-origin` or `-therian`) that you're looking to strip out?



If you're working directly with the dictionary keys (the Pokémon names) and need to strip those suffixes while you're extracting the data, you have a couple of efficient options.

Since you can't pass an array to `removesuffix`, you can use a **nested loop** inside a dictionary comprehension or use **Regex**.

Option 1: The Loop Approach (Cleanest)

This is best if you want to keep it readable. Since you're iterating over the dictionary to build your list for the DataFrame anyway, just run a small internal loop:

```
python

# The suffixes you want to strip
unwanted = ["-breed", "-mega", "-alola"]

all_pokemon = []

# Assuming 'data' is your JSON/Dictionary
for tier, json_data in data_map.items():
    for name in json_data["pokemon"].keys():

        # Clean the name
        clean_name = name
        for suffix in unwanted:
            clean_name = clean_name.removesuffix(suffix)

        all_pokemon.append({
            "Pokemon": clean_name,
            "Tier": tier
        })

df = pd.DataFrame(all_pokemon)
```

Option 2: The Regex Approach (Shortest)

If you want to do it in one shot without looping through a list of suffixes manually, Python's `re` (regular expression) module is your friend. The `|` acts as the "OR" for your array.

```
python
```

```
import re

# The | means "OR", and the $ means "only at the end of the string"
suffix_pattern = r'(-breed|-mega|-alola)$'

for tier, json_data in data_map.items():
    for name in json_data["pokemon"].keys():
        # This replaces any match in the pattern with an empty string
        clean_name = re.sub(suffix_pattern, '', name)

        all_pokemon.append({"Pokemon": clean_name, "Tier": tier})
```

A Quick Warning on Pokémon Names

Just a heads-up from one Pokémon data nerd to another: be careful with stripping too many suffixes!

If you strip `-paldea` from `tauros-paldea-blaze`, you'll end up with `tauros-blaze`. If you strip `-blaze` as well, you just get `tauros`. If your DataFrame has three different Tauros forms all renamed to "Tauros," you'll have duplicate rows that might mess up your model or analysis later.

Is the goal to group all forms together as one species, or just to make the names look "prettier"?

2026-05-03 12:14:19

You Asked



instead of writing the new names to a new array, I want to change the dictionary key names



Changing keys in a dictionary while iterating over it can be tricky because Python doesn't allow you to change the size or keys of a dictionary during a loop (it will throw a `RuntimeError`).

The standard way to do this is to create a **new dictionary** with the updated keys. Here is the most efficient way to do that using a dictionary comprehension.

The "Clean" Way (Dictionary Comprehension)

This creates a new version of your `pokemon` dictionary with the cleaned names as keys, but keeps all that massive move/stat data as the values.

```
python
```



```

import re

# Your 'array' of suffixes
unwanted = ["-breed", "-mega", "-alola"]
# Build a regex pattern: '(-breed|-mega|-alola)$'
pattern = r'(' + '|'.join(unwanted) + ')$'

# Assuming your dict is 'data' and the pokemon are under data['pokemon']
original_pokemon_dict = data['pokemon']

# Create the new dict with cleaned keys
data['pokemon'] = {
    re.sub(pattern, '', name): stats
    for name, stats in original_pokemon_dict.items()
}

```

Why this is better than "modifying" the existing one:

1. **Safety:** It avoids `RuntimeError: dictionary changed size during iteration`.
2. **Speed:** Dictionary comprehensions are highly optimized in Python.
3. **Data Integrity:** It perfectly preserves the nested objects (moves, items, typeNames) while only renaming the "top-level" key.

If you absolutely MUST modify it in place

If for some reason you cannot create a new dictionary (e.g., memory constraints with a massive JSON), you have to collect the keys first, then pop and re-insert them:

```

python

# Get a list of keys first so we aren't iterating the dict directly
keys = list(data['pokemon'].keys())

for name in keys:
    clean_name = name
    for suffix in unwanted:
        clean_name = clean_name.removesuffix(suffix)

    if clean_name != name:
        # Move the data to the new key and delete the old one
        data['pokemon'][clean_name] = data['pokemon'].pop(name)

```

One thing to watch out for: If your JSON has both `Tauros-Paldea-Blaze-Breed` and `Tauros-Paldea-Aqua-Breed`, and you strip `-breed` from both, they will still have unique names (`-blaze` vs `-aqua`).

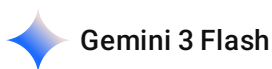
But if you strip everything until they are both just `Tauros`, the second one processed will **overwrite** the first one in the dictionary. Does your "unwanted" list include the specific form names (blaze/aqua) or just the word `-breed`?

2026-05-03 12:21:55

You Asked



how can I change a single pokemon name in the df say change necrozma-dawn-wing to necrozma-dawn



Since you're working with a DataFrame now, the most direct way is to use `.replace()`. You can target the specific column and use a dictionary to map the "old name" to the "new name."

1. The Simple Way (`.replace`)

This is the cleanest method if you have a specific swap in mind.

python

```
# Change one specific name
df['Pokemon'] = df['Pokemon'].replace('necrozma-dawn-wing', 'necrozma-dawn')

# You can also do multiple specific swaps at once
df['Pokemon'] = df['Pokemon'].replace({
    'necrozma-dawn-wing': 'necrozma-dawn',
    'necrozma-dusk-mane': 'necrozma-dusk'
})
```

2. The Conditional Way (`.loc`)

If you want to be extra precise (for example, only changing the name if it belongs to a certain tier), you can use `.loc`:

python

```
# df.loc[condition, column_to_change] = new_value
df.loc[df['Pokemon'] == 'necrozma-dawn-wing', 'Pokemon'] = 'necrozma-dawn'
```



3. The "Quick and Dirty" String Method

If you just want to strip "-wing" whenever it appears in a name without creating a full mapping dictionary:

```
python
```

```
df['Pokemon'] = df['Pokemon'].str.removesuffix('-wing')
```

A Note on Smogon Names

Just a heads-up: Smogon (and the Pokémon Showdown engine) usually uses

Necrozma-Dawn-Wings (with an 's'). If your code isn't finding the name, double-check if the "s" is present at the end of the string!

If you want to be safe and catch both cases, you can use a partial string replacement:

```
python
```

```
df['Pokemon'] = df['Pokemon'].str.replace('necrozma-dawn-wing.*', 'necrozma-dawn', regex=True)
```

Which naming convention are you standardizing toward—the official game names or the shorter Smogon/Showdown shorthand?

2026-05-03 13:27:53 You Asked 

I want to take the following JSON stucture and read all the abilities in order to my dataframe as new columns (if the pokemon does not have 3 abilities the coresponding columns will be none) I also want the 10 highest usage moves of each pokemon as columns, if the pokemon in question does not have 10 move options mark the remaning as none. Here is the json structure: Kingambit
lead{ raw: 0.0037, real: 0.0037, weighted: 0.0054 }usage{ raw: 0.1947, real: 0.167, weighted: 0.2488 }count425975weight0.0875viability(4)[29484, 91, 81, 63]abilities{ "Supreme Overlord": 0.9853, Defiant: 0.0144, Pressure: 0.0003 }items{ Leftovers: 0.5389, "Black Glasses": 0.1765, "Air Balloon": 0.1358, ... }teraTypes{ Ghost: 0.2986, Dark: 0.2267, Fairy: 0.2091, ... }happinesses{ 255: 1 }spreads{ "Adamant:0/252/4/0/0/252": 0.1516, "Jolly:0/252/4/0/0/252": 0.09, "Adamant:0/252/0/0/4/252": 0.0774, ... }moves{ "Sucker Punch":

```
0.9981, "Swords Dance": 0.9583, "Kowtow Cleave": 0.7861, ...
}teammates{ "Great Tusk": 0.3854, "Iron Valiant": 0.2678, "Ogerpon-
Wellspring": 0.2263, ... }counters{ Dondozo: (3)[...], Zamazenta: (3)
[...], "Great Tusk": (3)[...], ... }
```

Gemini 3 Flash

To handle this, we need to extract the keys from the `abilities` and `moves` dictionaries. Since these dictionaries are already sorted by usage frequency in Smogon's JSON format, we can simply grab the first N keys.

Here is how to process your JSON into a structured list that converts perfectly into a pandas DataFrame.

The Python Extraction Logic

python

```
import pandas as pd

all_pokemon_data = []

# Assuming 'data_map' is your dict of {TierName: JsonData}
for tier, json_data in data_map.items():
    pokemon_dict = json_data.get("pokemon", {})

    for name, stats in pokemon_dict.items():
        # 1. Start with the basic info
        row = {
            "Pokemon": name,
            "Tier": tier
        }

        # 2. Extract Abilities (up to 3)
        # .keys() gives names; we cast to list to slice them
        ability_list = list(stats.get("abilities", {}).keys())
        for i in range(3):
            row[f"Ability_{i+1}"] = ability_list[i] if i < len(ability_list) else None

        # 3. Extract Moves (top 10)
        # Smogon JSONs are pre-sorted by usage frequency
        move_list = list(stats.get("moves", {}).keys())
        for i in range(10):
            row[f"Move_{i+1}"] = move_list[i] if i < len(move_list) else None
```

```

all_pokemon_data.append(row)

# Create DataFrame
df = pd.DataFrame(all_pokemon_data)

# Reorder to keep 'Tier' as the rightmost label column
cols = [c for c in df.columns if c != 'Tier'] + ['Tier']
df = df[cols]

```

How this handles your requirements:

- **The "None" Requirement:** The `if i < len(...) else None` logic ensures that if a Pokémon like *Regieleki* only has 2 abilities or a Pokémon has only 6 moves, the remaining columns for that row will be filled with Python's `None` (which Pandas converts to `NaN`).
- **Top 10 Moves:** Because dictionaries in modern Python preserve insertion order and Smogon generates these JSONs from highest usage to lowest, `list(stats["moves"].keys())[:10]` automatically gives you the 10 most popular moves.
- **Column Ordering:** By building the `row` dictionary in order and then re-indexing at the end, your DataFrame will look clean: `Pokemon` , `Ability 1-3` , `Move 1-10` , and finally `Tier` .

Quick View of the Output Structure:

Pokemon	Ability_1	...	Move_1	...	Move_10	Tier
Kingambit	Supreme Overlord	...	Sucker Punch	...	None	OU
Great Tusk	Protosynthesis	...	Headlong Rush	...	Taunt	OU

Pro-Tip for Modeling

If you plan to use this for Machine Learning, remember that `Move_1` for one Pokémon might be "Sucker Punch" and for another it might be "Earthquake." You may eventually need to **One-Hot Encode** these moves if you want a model to understand them as features!

Do you need to keep the actual usage percentages (like the `0.9981` for Sucker Punch), or are the names enough for your current project?

2026-05-03 13:34:49 You Asked 

I already have a df made up of pokemon names and tiers, I would like to add the rest as columns to that dataframe

First, you'll need to connect Google Workspace to turn on this app.

2026-05-03 13:35:23 You Asked 

I already have a df made up of pokemon names and tiers, I would like to add the rest as columns to that dataframe

To add these columns to your existing DataFrame, the most robust method is to create a helper dictionary from your JSON and then use `.map()` or `.merge()`.

Since you have multiple values (3 abilities, 10 moves), we will extract this data into a new "features" DataFrame and then merge it with your original one based on the **Pokemon** name.

1. Extract Features from JSON

This step creates a list of dictionaries that match your existing "Pokemon" column.

python

```
# Assuming 'json_data' is your source dictionary
extracted_features = []

for name, stats in json_data['pokemon'].items():
    # Clean the name if needed (e.g., your necrozma or breed fixes)
    clean_name = name.removesuffix("-breed").replace("necrozma-dawn-wing", "necrozma-dawn")

    feat = {"Pokemon": clean_name}

    # Extract 3 Abilities
    abilities = list(stats.get("abilities", {}).keys())
    for i in range(3):
        feat[f"Ability_{i+1}"] = abilities[i] if i < len(abilities) else None

    # Extract 10 Moves
    moves = list(stats.get("moves", {}).keys())
    for i in range(10):
        feat[f"Move_{i+1}"] = moves[i] if i < len(moves) else None
```

```
extracted_features.append(feat)
```

```
# Turn the new data into a temporary DataFrame  
df_features = pd.DataFrame(extracted_features)
```

2. Merge with your existing DataFrame

Now, merge `df_features` into your original `df`. We use a **left join** to ensure we keep the order of your original rows.

python

```
# Merge on the Pokemon column  
df = pd.merge(df, df_features, on="Pokemon", how="left")  
  
# Reorder to push 'Tier' back to the end  
cols = [c for c in df.columns if c != 'Tier'] + ['Tier']  
df = df[cols]
```

Why this is the best approach:

- **Alignment:** If your original DF has a different order than the JSON, `merge` aligns the data correctly by name.
- **Missing Data:** If a Pokemon in your DF isn't in that specific JSON, it will simply fill the new columns with `NaN` rather than breaking.
- **Duplicate Prevention:** By doing the name cleaning (like the `-breed` removal) *during* the extraction, you ensure the keys match your existing DF perfectly.

A Quick Tip on Move Logic

Since Smogon JSONs usually include every move ever used (even the bad ones), the `list(stats.get("moves", {}).keys())` logic works perfectly because the keys are **already sorted by usage weight** in the source file. `Move_1` will always be the most common move.

Does your current DataFrame contain multiple tiers, or are you doing this one tier at a time?

2026-05-03 13:40:36 You Asked 

new idea, here is where the tier and name df was created, each tier dict entry has the same JSON structure provided before lets add the name, tier, abilities and moves here: import pandas as pd

```

tier_dicts = {
    "Ubers": ubers_pkmn,
    "UUbers": uubers_pkmn,
    "OU": ou_pkmn,
    "UU": uu_pkmn,
    "RU": ru_pkmn,
    "NU": nu_pkmn,
    "PU": pu_pkmn,
    "ZU": zu_pkmn
}

combined_dataset = []

for tier, pkmn_list in tier_dicts.items():

    pkmn = pkmn_list.keys()

    if(tier == "UUbers"):
        for name in pkmn:
            combined_dataset.append({
                "Pokemon": name,
                "Tier": "Ubers"
            })
    else:
        for name in pkmn:
            combined_dataset.append({
                "Pokemon": name,
                "Tier": tier
            })

df = pd.DataFrame(combined_dataset)
print(df)

```



Updating the logic at the source is definitely the way to go. It's more efficient than merging later and keeps your code much cleaner.

Here is the updated loop. I've added the name-cleaning logic we discussed and the loops to extract the abilities and moves dynamically.

python

```
import pandas as pd

tier_dicts = {
    "Ubers": ubers_pkmn,
    "UUbers": uubers_pkmn,
    "OU": ou_pkmn,
    "UU": uu_pkmn,
    "RU": ru_pkmn,
    "NU": nu_pkmn,
    "PU": pu_pkmn,
    "ZU": zu_pkmn
}

combined_dataset = []

for tier, pkmn_data in tier_dicts.items():
    # Access the 'pokemon' key inside each tier's JSON
    # .get() helps avoid errors if a file is missing that key
    pokemon_entries = pkmn_data.get("pokemon", {})

    for name, stats in pokemon_entries.items():
        # --- 1. Clean the Name ---
        clean_name = name.removesuffix("-breed")
        if clean_name == "necrozma-dawn-wing":
            clean_name = "necrozma-dawn"

        # --- 2. Build the Base Entry ---
        # Handle your UUbers label logic here
        label_tier = "Ubers" if tier == "UUbers" else tier

        entry = {
            "Pokemon": clean_name,
            "Tier": label_tier
        }

        # --- 3. Extract Abilities (Top 3) ---
        ability_list = list(stats.get("abilities", {}).keys())
        for i in range(3):
            entry[f"Ability_{i+1}"] = ability_list[i] if i < len(ability_list) else None

        # --- 4. Extract Moves (Top 10) ---
        move_list = list(stats.get("moves", {}).keys())
        for i in range(10):
            entry[f"Move_{i+1}"] = move_list[i] if i < len(move_list) else None

        combined_dataset.append(entry)
```



```
# Create DataFrame
df = pd.DataFrame(combined_dataset)

# --- 5. Reorder Columns ---
# This ensures 'Tier' is always the last column on the right
cols = [c for c in df.columns if c != 'Tier'] + ['Tier']
df = df[cols]

print(df.head())
```

Key Changes Made:

- 1. Name Cleaning:** I integrated the `removesuffix` and `necrozma` fix directly into the loop so your "Pokemon" column is standardized from the start.
- 2. Tier Logic:** Simplified your `if/else` for UUbers to a single line inside the loop.
- 3. Dynamic Column Creation:** By adding `entry[f"Ability_{i+1}"]` inside the loop, we ensure every row has the same keys, allowing Pandas to build the table correctly.
- 4. Order Preservation:** The final reorder step ensures that even though we added Moves and Abilities *after* the Tier in the dictionary, the Tier is pushed to the far right.

One Small Warning:

Since you are looping through many tiers, some Pokémon might appear in multiple tiers (e.g., a Pokémon viable in both UU and OU). If you want a **unique** list of Pokémon, you may need to decide if you want to keep all entries or drop duplicates later using `df.drop_duplicates(subset=['Pokemon'])`.

Does this data structure look like what you need for your label column?

2026-05-03 14:20:09

You Asked



I want to make 6 columns namely: Hazard, Pivot, Setup, Recovery, Utility and Priority. Each of these columns will be binary in value i.e. (0 = doesn't have, 1 = has). I would like you to create a list of strings for each category containing common moves of each listed category. Then we will iterate over our df to determine the value of these 6 columns for each pokemon based on their move1-10 columns

This is a great way to transform raw move names into meaningful "role" features for a machine learning model. By checking the top 10 moves, you're essentially asking: *"Is this Pokémon commonly used for this role?"*

1. Define the Move Categories

Here is a comprehensive list of common competitive moves for each category based on the Gen 9 meta.

python

```
# Move Categories
hazard_moves = ["Stealth Rock", "Spikes", "Toxic Spikes", "Sticky Web", "Ceaseless Edge",
"Stone Axe"]
pivot_moves = ["U-turn", "Volt Switch", "Flip Turn", "Parting Shot", "Chilly Reception", "Baton
Pass"]
setup_moves = ["Swords Dance", "Dragon Dance", "Quiver Dance", "Calm Mind", "Nasty Plot", "Bulk
Up", "Iron Defense", "Agility", "Shell Smash", "Victory Dance", "Tidy Up"]
recovery_moves = ["Recover", "Soft-Boiled", "Roost", "Slack Off", "Milk Drink", "Shore Up",
"Synthesis", "Moonlight", "Morning Sun", "Wish", "Strength Sap"]
utility_moves = ["Rapid Spin", "Defog", "Mortal Spin", "Knock Off", "Taunt", "Encore", "Will-O-
Wisp", "Thunder Wave", "Toxic", "Glare", "Trick", "Switcheroo", "Haze", "Roar", "Whirlwind"]
priority_moves = ["Sucker Punch", "Extreme Speed", "Jet Punch", "Aqua Jet", "Bullet Punch",
"Ice Shard", "Mach Punch", "Shadow Sneak", "Fake Out", "Thunderclap", "Vacuum Wave"]

# Map them to a dictionary for easier iteration
categories = {
    "Hazard": hazard_moves,
    "Pivot": pivot_moves,
    "Setup": setup_moves,
    "Recovery": recovery_moves,
    "Utility": utility_moves,
    "Priority": priority_moves
}
```

2. Update the DataFrame

We will iterate over the categorized moves and check if any of them appear in the `Move_1` through `Move_10` columns for each row.

python

```
# List of the columns we want to check
move_cols = [f"Move_{i+1}" for i in range(10)]

for category_name, move_list in categories.items():
    # We use .isin() to check if any of the move columns contain a move from the category list
```

```
# .any(axis=1) returns True if the move is found in any of the 10 columns
# .astype(int) converts True/False to 1/0
df[category_name] = df[move_cols].isin(move_list).any(axis=1).astype(int)

# Reorder one last time to keep Tier at the end
cols = [c for c in df.columns if c != 'Tier'] + ['Tier']
df = df[cols]

print(df[['Pokemon', 'Hazard', 'Pivot', 'Setup', 'Recovery', 'Utility', 'Priority',
'Tier']].head())
```

How this logic works:

1. **df[move_cols].isin(move_list)** : This creates a temporary table of True/False values for all 10 move columns.
2. **.any(axis=1)** : This checks row-by-row. If **any** of the 10 move columns contains a move from your list, it returns **True** .
3. **.astype(int)** : This turns **True** into **1** and **False** into **0** , making it perfect for your label column setup.

Pro-Tip: Customizing the Lists

- **Utility:** I included **Knock Off** and **Rapid Spin** in Utility. If you feel Knock Off is just an "Attack," you can remove it.
- **Hazards:** I included the new Gen 9 "damaging hazards" like **Ceaseless Edge** (Samurott-H) and **Stone Axe** (Kleavor).
- **Recovery:** I added **Strength Sap**, as it's a primary recovery tool for many Ghost types.

Does this list of moves cover the specific strategies you're looking to track, or should we add more niche moves like *Spiky Shield* or *Baneful Bunker*?

2026-05-03 15:39:55 You Asked 

i would like to make sure everything is lowercase and separated by "-" to guarantee no mismatch between move name in df and move name in category list how can I accomplish this: for role, move_list in categories.items():

```
move_list = move_list.lower().replace(" ", "-")
print(move_list)
```

To standardize everything, you need to apply the transformation to **every element** inside the lists. Your current code tries to treat the whole `move_list` (which is a list) like a single string, which will cause an error.

The best way to do this is with a **list comprehension** inside your loop. This processes each move one by one.

1. Standardize your Category Lists

This will convert "Stealth Rock" to "stealth-rock" for every item in your dictionary.

python

```
# Standardize the category dictionary
for role, move_list in categories.items():
    # .lower() makes it lowercase, .replace(" ", "-") swaps spaces for hyphens
    categories[role] = [m.lower().replace(" ", "-") for m in move_list]
```

2. Standardize the DataFrame Columns

You must also ensure the move names *inside* your DataFrame are formatted the same way before you run the comparison logic.

python

```
# List of your 10 move columns
move_cols = [f"Move_{i+1}" for i in range(10)]

# Apply the transformation to the entire block of move columns
for col in move_cols:
    df[col] = df[col].str.lower().str.replace(" ", "-", regex=False)
```

3. Run the Binary Logic

Now that both the lists and the DataFrame are identical in format (lowercase and hyphenated), you can run the binary check safely:

python

```
for category_name, move_list in categories.items():
    # Check if any move in the 10 columns exists in our standardized list
    df[category_name] = df[move_cols].isin(move_list).any(axis=1).astype(int)

# Final view
```

```
print(df[['Pokemon', 'Hazard', 'Setup', 'Tier']].head())
```

Why this is foolproof:

- **The "Space" Problem:** Some JSONs use `Stealth Rock` while others might use `stealthrock` or `stealth-rock`. Standardizing to hyphens is the safest "middle ground."
- **.str.replace vs .replace**: In Pandas, `df.str.replace` is used to modify the text *inside* the cells, whereas `df.replace` is used to swap the entire cell value. Since we just want to change spaces to hyphens, `.str.replace` is what you need.
- **Consistency:** By doing both the list and the DF at the same time, you eliminate the risk of `Sucker Punch` (list) not matching `sucker-punch` (dataframe).

One minor detail: In Smogon JSONs, moves are sometimes already hyphenated or lowercased. Does your current `Move_1` column show "Sucker Punch" or "suckerpunch"? (If it's the latter, we might need to adjust the `.replace(" ", "-")` logic slightly!)

2026-05-04 13:26:21

You Asked



I would like to create two features: `TypeDefScore` and `TypeOffScore`, these two features are meant to measure how effective is a pokemon's type combination defensively and offensively. I will not consider the importance of individual types i.e. resisting fairy is better than resisting rock, however I do want immunities to be significantly more valued than resistances



Gemini 3 Flash Thinking

To create these scores, we need a **Type Effectiveness Matrix** and a function that evaluates how a Pokémon's specific typing interacts with all 18 types.

Since your goal is to value **immunities** significantly more than resistances, we will use a weighted scoring system.

1. The Scoring Logic

- **Defensive Score (`TypeDefScore`):** We check how all 18 types hit your Pokémon.
 - **Weaknesses (2x, 4x):** Deduct points (-2, -4).
 - **Neutral (1x):** 0 points.



- **Resistances (0.5x, 0.25x)**: Add points (+1, +2).
- **Immunities (0x)**: Add **+6 points** (making an immunity worth 6x a standard resistance).
- **Offensive Score (TypeOffScore)**: We check how your STAB (Same Type Attack Bonus) types hit all 18 types.
 - We take the **best** matchup for each of the 18 types (e.g., if you are Fire/Fighting, you hit Steel with Fighting for 2x, so that counts as super effective).
 - **Super Effective (2x)**: +2 points.
 - **Neutral (1x)**: +1 point.
 - **Resisted/Immune**: 0 points.

2. The Python Implementation

First, define the type chart. I've simplified this into a dictionary of multipliers.

python

```
import pandas as pd

# 18 Types
types = ['normal', 'fire', 'water', 'electric', 'grass', 'ice', 'fighting', 'poison',
         'ground', 'flying', 'psychic', 'bug', 'rock', 'ghost', 'dragon', 'dark', 'steel',
         'fairy']

# Effectiveness Matrix: chart[attacker][defender]
# 1 = Neutral, 2 = Super, 0.5 = Resist, 0 = Immune
chart = {
    'normal': {'rock': 0.5, 'ghost': 0, 'steel': 0.5},
    'fire': {'fire': 0.5, 'water': 0.5, 'grass': 2, 'ice': 2, 'bug': 2, 'rock': 0.5,
             'dragon': 0.5, 'steel': 2},
    'water': {'fire': 2, 'water': 0.5, 'grass': 0.5, 'ground': 2, 'rock': 2, 'dragon': 0.5},
    'electric': {'water': 2, 'electric': 0.5, 'grass': 0.5, 'ground': 0, 'flying': 2, 'dragon':
                 0.5},
    'grass': {'fire': 0.5, 'water': 2, 'grass': 0.5, 'poison': 0.5, 'ground': 2, 'flying':
              0.5, 'bug': 0.5, 'rock': 2, 'dragon': 0.5, 'steel': 0.5},
    'ice': {'fire': 0.5, 'water': 0.5, 'grass': 2, 'ice': 0.5, 'ground': 2, 'flying': 2,
            'dragon': 2, 'steel': 0.5},
    'fighting': {'normal': 2, 'ice': 2, 'poison': 0.5, 'flying': 0.5, 'psychic': 0.5, 'bug':
                 0.5, 'rock': 2, 'ghost': 0, 'dark': 2, 'steel': 2, 'fairy': 0.5},
    'poison': {'grass': 2, 'poison': 0.5, 'ground': 0.5, 'rock': 0.5, 'ghost': 0.5, 'steel':
               0, 'fairy': 2},
    'ground': {'fire': 2, 'electric': 2, 'grass': 0.5, 'poison': 2, 'flying': 0, 'bug': 0.5,
               'rock': 2, 'steel': 2},
    'flying': {'electric': 0.5, 'grass': 2, 'fighting': 2, 'bug': 2, 'rock': 0.5, 'steel':
               0.5},
    'psychic': {'fighting': 2, 'poison': 2, 'psychic': 0.5, 'dark': 0, 'steel': 0.5},
```

```

'bug':      {'fire': 0.5, 'grass': 2, 'fighting': 0.5, 'poison': 0.5, 'flying': 0.5,
'psychic': 2, 'ghost': 0.5, 'dark': 2, 'steel': 0.5, 'fairy': 0.5},
'rock':     {'fire': 2, 'ice': 2, 'fighting': 0.5, 'ground': 0.5, 'flying': 2, 'bug': 2,
'steel': 0.5},
'ghost':    {'normal': 0, 'psychic': 2, 'ghost': 2, 'dark': 0.5},
'dragon':   {'dragon': 2, 'steel': 0.5, 'fairy': 0},
'dark':     {'fighting': 0.5, 'psychic': 2, 'ghost': 2, 'dark': 0.5, 'fairy': 0.5},
'steel':    {'fire': 0.5, 'water': 0.5, 'electric': 0.5, 'ice': 2, 'rock': 2, 'steel': 0.5,
'fairy': 2},
'fairy':    {'fire': 0.5, 'fighting': 2, 'poison': 0.5, 'dragon': 2, 'dark': 2, 'steel':
0.5}
}

def get_scores(row_types):
    t1 = row_types[0].lower()
    t2 = row_types[1].lower() if len(row_types) > 1 and row_types[1] else None

    def_score = 0
    off_score = 0

    for attacker in types:
        # Defense logic
        eff = chart.get(attacker, {}).get(t1, 1)
        if t2: eff *= chart.get(attacker, {}).get(t2, 1)

        if eff == 0: def_score += 6      # Immunity (Heavy value)
        elif eff == 0.25: def_score += 2 # Double resist
        elif eff == 0.5: def_score += 1  # Resist
        elif eff == 2: def_score -= 2    # Weak
        elif eff == 4: def_score -= 4    # Double Weak

    for target in types:
        # Offense logic (Best STAB vs target)
        eff1 = chart.get(t1, {}).get(target, 1)
        eff2 = chart.get(t2, {}).get(target, 1) if t2 else 0
        max_eff = max(eff1, eff2)

        if max_eff >= 2: off_score += 2
        elif max_eff >= 1: off_score += 1

    return def_score, off_score

```

3. Applying it to your DataFrame

Assuming you have your types in a list or two columns:

```
python
```



```

# Create the columns by applying the function
# (Example assuming 'Types' column is a list like ['dark', 'steel'])
df[['TypeDefScore', 'TypeOffScore']] = df.apply(
    lambda row: pd.Series(get_scores(row['Types'])), axis=1
)

# Push Tier to the end again
cols = [c for c in df.columns if c != 'Tier'] + ['Tier']
df = df[cols]

```

Why this works for your goals:

- **Immunities:** A Pokémon with an immunity (like Gholdengo's 3 immunities) will skyrocket in `TypeDefScore` compared to a Pokémon with just many resistances.
- **Offensive Coverage:** This measures "potential." If you are a dual-type, the `max()` ensures you get credit for having a move that hits a target super-effectively, even if your other type is resisted.
- **Machine Learning Ready:** These numerical scores are much easier for a model to process than raw strings like "Fire/Flying."

Note: Does your JSON already provide the Pokémon's types, or do we need to pull that from a mapping dictionary? (Kingambit is Dark/Steel, Great Tusk is Ground/Fighting, etc.)

2026-05-04 13:28:39 You Asked 

one more thing instead you generating a type chart I need to read the type matchups from a JSON, here is the structure: "normal": {
 "damage_relations": {
 "double_damage_from": [
 {
 "name": "fighting",
 "url": "https://pokeapi.co/api/v2/type/2/"
 }
],
 "double_damage_to": [],
 "half_damage_from": [],
 "half_damage_to": [
 {
 "name": "rock",
 "url": "https://pokeapi.co/api/v2/type/6/"

```

    },
    {
      "name": "steel",
      "url": "https://pokeapi.co/api/v2/type/9/"
    }
  ],
  "no_damage_from": [
    {
      "name": "ghost",
      "url": "https://pokeapi.co/api/v2/type/8/"
    }
  ],
  "no_damage_to": [
    {
      "name": "ghost",
      "url": "https://pokeapi.co/api/v2/type/8/"
    }
  ]
},

```

Gemini 3 Flash Thinking

Since you're using the PokéAPI-style JSON structure, we need to parse those `damage_relations` lists into a format that allows us to quickly look up multipliers.

Here is how you can parse that JSON and use it to calculate your scores.

1. Parsing the PokéAPI JSON

This function will convert your JSON into a simple lookup dictionary:

`multipliers[attacker][defender] = value` .

python

```

import pandas as pd

# list of all 18 types to iterate over
all_types = [
    "normal", "fire", "water", "electric", "grass", "ice", "fighting", "poison",
    "ground", "flying", "psychic", "bug", "rock", "ghost", "dragon", "dark", "steel", "fairy"
]

def parse_type_json(type_json):

```

```

"""
Converts PokéAPI JSON into a flat lookup: lookup[attacker][defender]
"""

lookup = {t: {d: 1.0 for d in all_types} for t in all_types}

for type_name, data in type_json.items():
    rel = data['damage_relations']

    # Offense: How 'type_name' hits others
    for item in rel['double_damage_to']:
        lookup[type_name][item['name']] = 2.0
    for item in rel['half_damage_to']:
        lookup[type_name][item['name']] = 0.5
    for item in rel['no_damage_to']:
        lookup[type_name][item['name']] = 0.0

    # Defense: How others hit 'type_name'
    # (Note: We could also pull this from the 'from' keys,
    # but the logic above already covers it by filling the attacker's perspective)

return lookup

# Assuming 'type_matchups_json' is your loaded dictionary
type_lookup = parse_type_json(type_matchups_json)

```

2. The Updated Scoring Function

Now we use that `type_lookup` to calculate the scores based on your specific weighting (Immunities = 6, Resists = 1, etc.).

python

```

def calculate_type_metrics(row_types):
    # row_types is expected to be a list: ['dark', 'steel']
    t1 = row_types[0].lower()
    t2 = row_types[1].lower() if len(row_types) > 1 and row_types[1] else None

    def_score = 0
    off_score = 0

    # --- Defensive Score ---
    for attacker in all_types:
        # Get multiplier for first type
        mult = type_lookup[attacker][t1]
        # Multiply by second type if it exists
        if t2:
            mult *= type_lookup[attacker][t2]

```

```

    if mult == 0:    def_score += 6 # Immunity
    elif mult == 0.25: def_score += 2 # 4x Resist
    elif mult == 0.5: def_score += 1 # 2x Resist
    elif mult == 2:   def_score -= 2 # Weakness
    elif mult == 4:   def_score -= 4 # 4x Weakness

# --- Offensive Score ---
for target in all_types:
    # Check which STAB does more damage to the target
    m1 = type_lookup[t1][target]
    m2 = type_lookup[t2][target] if t2 else 0
    best_mult = max(m1, m2)

    if best_mult >= 2:    off_score += 2 # Super Effective
    elif best_mult >= 1: off_score += 1 # Neutral
    # Resisted/Immune = 0

return def_score, off_score

```

3. Applying to the DataFrame

Assuming your DataFrame has a column called `Types` containing lists (e.g., `['Normal']` or `['Ghost', 'Poison']`):

python

```

# Generate the scores
scores = df['Types'].apply(lambda x: calculate_type_metrics(x))

# Split the tuple result into two new columns
df[['TypeDefScore', 'TypeOffScore']] = pd.DataFrame(scores.tolist(), index=df.index)

# Ensure Tier stays on the far right
cols = [c for c in df.columns if c != 'Tier'] + ['Tier']
df = df[cols]

```

Why this works with your JSON:

- **Decoupled Logic:** The `parse_type_json` function handles the "messy" structure of the PokéAPI format, so the actual scoring function stays clean and readable.
- **Dual-Type Math:** For defense, it correctly multiplies the values (e.g., $0.5 \times 0.5 = 0.25$ for a double resistance). For offense, it uses `max()` because a Pokémon will simply use the more effective move of its two STAB types.

- **Weighting:** Your requirement for **immunities to be significantly more valued** is met by giving them +6, which is 6x the value of a standard resistance.

Are you getting the Pokémon types from the same Smogon JSON, or are you pulling those from the PokéAPI as well?

2026-05-04 13:58:11

You Asked



ok, we can't just apply this to the dataframe. We first need to read the types from the pokeapi json by key matching using the names from the dataframe

Gemini 3 Flash Thinking

To do this, we'll create a mapping dictionary from your Pokémon API JSON. This allows us to look up a Pokémon's types using the name in your DataFrame, then pass those types into the scoring functions.

Here is the complete workflow:

1. Create the Type Lookup (Matchups)

This is the same logic as before, converting the `damage_relations` JSON into a multiplier table.

python

```
all_types = ["normal", "fire", "water", "electric", "grass", "ice", "fighting", "poison",
"ground", "flying", "psychic", "bug", "rock", "ghost", "dragon", "dark", "steel", "fairy"]

def parse_type_matchups(type_json):
    lookup = {t: {d: 1.0 for d in all_types} for t in all_types}
    for type_name, data in type_json.items():
        rel = data['damage_relations']
        for item in rel['double_damage_to']: lookup[type_name][item['name']] = 2.0
        for item in rel['half_damage_to']:   lookup[type_name][item['name']] = 0.5
        for item in rel['no_damage_to']:     lookup[type_name][item['name']] = 0.0
    return lookup

# Assuming 'type_api_json' is your loaded type matchup data
type_lookup = parse_type_matchups(type_api_json)
```

2. Create the Pokémon Name-to-Type Map



We need to iterate through your Pokémon API JSON to create a dictionary where the key is the **name** and the value is a **list of types**.

Note: Since your DataFrame names are likely lowercase/hyphenated now (e.g., `great-tusk`), we ensure the mapping keys match that format.

python

```
def create_name_to_type_map(pokemon_api_json):
    name_map = {}
    for key, data in pokemon_api_json.items():
        # PokeAPI usually stores types in a list: [{'type': {'name': 'ground'}}, ...]
        # We extract just the names: ['ground', 'fighting']
        types = [t['type']['name'] for t in data.get('types', [])]

        # Standardize the key to lowercase and hyphens to match your DF
        standard_name = key.lower().replace(" ", "-")
        name_map[standard_name] = types

    return name_map

# Assuming 'pokemon_api_json' is your loaded Pokemon data
pkmn_type_map = create_name_to_type_map(pokemon_api_json)
```

3. The Scoring Function

This function takes a list of types and calculates the values using your specified weights.

python

```
def get_type_scores(pokemon_name):
    # Lookup types from our map; default to ['normal'] if not found to avoid errors
    types = pkmn_type_map.get(pokemon_name.lower(), ["normal"])

    t1 = types[0]
    t2 = types[1] if len(types) > 1 else None

    def_score = 0
    off_score = 0

    for attacker in all_types:
        mult = type_lookup[attacker][t1]
        if t2: mult *= type_lookup[attacker][t2]

        if mult == 0:    def_score += 6
        elif mult == 0.25: def_score += 2
        elif mult == 0.5:  def_score += 1
        elif mult == 2:    def_score -= 2
        elif mult == 4:    def_score -= 4
```

```

for target in all_types:
    m1 = type_lookup[t1][target]
    m2 = type_lookup[t2][target] if t2 else 0
    best_mult = max(m1, m2)
    if best_mult >= 2: off_score += 2
    elif best_mult >= 1: off_score += 1

return def_score, off_score

```

4. Apply to DataFrame

Now we apply this to the `Pokemon` column. This handles the key-matching and score generation in one pass.

python

```

# 1. Generate scores by matching the name in the DF to our type map
results = df['Pokemon'].apply(lambda name: get_type_scores(name))

# 2. Split the results into the two new columns
df[['TypeDefScore', 'TypeOffScore']] = pd.DataFrame(results.tolist(), index=df.index)

# 3. Ensure Tier remains the last column
cols = [c for c in df.columns if c != 'Tier'] + ['Tier']
df = df[cols]

print(df[['Pokemon', 'TypeDefScore', 'TypeOffScore', 'Tier']].head())

```

Why this approach works:

- 1. Key Matching:** By creating `pkmn_type_map` first, we decouple the big JSON object from the DataFrame. The `.get(name.lower(), ["normal"])` ensures that if a name doesn't match perfectly, the script doesn't crash.
- 2. Efficiency:** Instead of searching the huge API JSON for every single row, we do a constant-time dictionary lookup (`pkmn_type_map[name]`), which is much faster.
- 3. No Type Column Needed:** You don't need to store the "Types" as a column in your final DataFrame if you don't want to; the types are used only for the calculation and then discarded.

One tip: If you have Pokémon in your DF like `tauros-paldea-blaze` , make sure that exact string exists as a key in your `pokemon_api_json` . If the API uses `tauros-blaze-breed` instead, you may need to add one more `.replace()` or `.removesuffix()` inside the `create_name_to_type_map` function!

how can i print the min/max typedefscore as well as the min/max typeoffscore and the corresponding pkmn name from the df

Gemini 3 Flash Thinking

To get the minimum and maximum values along with the specific Pokémon associated with them, you can use the `.idxmin()` and `.idxmax()` methods. These return the index of the row containing the target value.

1. The Quick Way (Returns the first match)

This is best for a quick summary. If multiple Pokémon share the same high score, this only prints the first one it finds.

python

```
# Defensive Scores
min_def_idx = df['TypeDefScore'].idxmin()
max_def_idx = df['TypeDefScore'].idxmax()

# Offensive Scores
min_off_idx = df['TypeOffScore'].idxmin()
max_off_idx = df['TypeOffScore'].idxmax()

print(f"Lowest TypeDefScore: {df.loc[min_def_idx, 'TypeDefScore']} ({df.loc[min_def_idx, 'Pokemon']})")
print(f"Highest TypeDefScore: {df.loc[max_def_idx, 'TypeDefScore']} ({df.loc[max_def_idx, 'Pokemon']})")
print("-" * 30)
print(f"Lowest TypeOffScore: {df.loc[min_off_idx, 'TypeOffScore']} ({df.loc[min_off_idx, 'Pokemon']})")
print(f"Highest TypeOffScore: {df.loc[max_off_idx, 'TypeOffScore']} ({df.loc[max_off_idx, 'Pokemon']})")
```

2. The Comprehensive Way (Handles ties)

In Pokémon, many species often share the same type combination (and thus the same score). If you want to see **everyone** who tied for the highest or lowest score, use boolean indexing:

python


```
# Highest Defensive Score
max_def_val = df['TypeDefScore'].max()
best_defenders = df[df['TypeDefScore'] == max_def_val]['Pokemon'].tolist()

# Highest Offensive Score
max_off_val = df['TypeOffScore'].max()
best_attackers = df[df['TypeOffScore'] == max_off_val]['Pokemon'].tolist()

print(f"Max TypeDefScore ({max_def_val}): {' '.join(best_defenders)}")
print(f"Max TypeOffScore ({max_off_val}): {' '.join(best_attackers)}")
```

What to expect from your data:

- **Highest TypeDefScore** : You'll likely see Pokémon with strong defensive types and multiple immunities, like **Gholdengo** (Steel/Ghost) or **Kingambit** (Steel/Dark).
- **Lowest TypeDefScore** : Look out for poor defensive combinations like **Abomasnow** (Grass/Ice), which has seven weaknesses and zero immunities.
- **Highest TypeOffScore** : Pokémon with types like **Ice**, **Fighting**, or **Ground** (which hit many targets super-effectively) will usually top this list.

Do these scores match your "gut feeling" for which Pokémon are the most dangerous in your current tier?

2026-05-04 15:02:41

You Asked 

this reads in order correct?: baseStats = [s['base_stat'] for s in data.get('stats')]



Spot on. That list comprehension will preserve the exact order the PokéAPI serves them in.

Because the `stats` field in the PokéAPI is a **list**, Python iterates through it from index 0 to 5. The API is hard-coded to always return them in this specific "standard" order:

1. **HP**
2. **Attack**
3. **Defense**

4. Special-Attack

5. Special-Defense

6. Speed

A Clean Way to Add Them to Your DataFrame

If you want to pull these into your DataFrame as individual columns (which is much better for machine learning than having a single list in a cell), you can unpack them like this:

python

```
# Assuming 'stats' is the list [45, 49, 49, 65, 65, 45]
df[['HP', 'Atk', 'Def', 'SpA', 'SpD', 'Spe']] = pd.DataFrame(df['BaseStatsList'].tolist(),
index=df.index)
```

One Tiny Safety Tip

Since you're using `.get('stats')`, if for some weird reason a Pokémon is missing that data, `.get()` will return `None`. Iterating over `None` will crash your code. You can add a fallback empty list to stay safe:

python

```
# The [] at the end prevents a TypeError if 'stats' is missing
baseStats = [s['base_stat'] for s in data.get('stats', [])]
```

Since you're calculating `TypeDefScore` and `TypeOffScore`, are you planning to use these base stats to calculate a **"Bulk"** or **"Sweep"** potential score as well?

2026-05-04 15:10:20

You Asked



i added the basestats into the map, how can I access the types array: `types = pkmn_type_bs_map.get(pkmn.lower())['types']`



Since you've nested both `base_stats` and `types` into your mapping dictionary, the way you access the `types` array depends on exactly how you structured the map during creation.

Assuming your `pkmn_type_bs_map` looks like this:

python

```
{
  "kingambit": {
    "types": ["dark", "steel"],
    "base_stats": [100, 135, 120, 60, 85, 50]
  }
}
```

1. The Direct Way

You can access the list of types for a specific Pokémon like this:

python

```
# Accessing the list directly
types = pkmn_type_bs_map.get(pkmn.lower(), {}).get('types', [])
```

2. Standardizing for your Scoring Function

If you are passing this into your `TypeDefScore` or `TypeOffScore` logic, you should keep the `pkmn.lower()` check and provide a fallback (like `['normal']`) to prevent the code from crashing if a Pokémon name is missing from your map.

python

```
# Safe extraction inside your loop or apply function
entry = pkmn_type_bs_map.get(pkmn.lower())

if entry:
    pokemon_types = entry.get('types', ['normal'])
    pokemon_stats = entry.get('base_stats', [0, 0, 0, 0, 0, 0])
else:
    pokemon_types = ['normal']
    pokemon_stats = [0, 0, 0, 0, 0, 0]
```

3. Pro-Tip: Accessing specific types

If you need to separate them (e.g., for "Type 1" and "Type 2" columns):

python

```
type1 = pokemon_types[0]
type2 = pokemon_types[1] if len(pokemon_types) > 1 else None
```

How to check your map structure

If you aren't sure exactly how the keys are named inside your nested dictionary, run this quick print command for one Pokémon:

```
python

import pprint
pprint.pprint(pkmn_type_bs_map.get('pikachu'))
```

This will show you the exact "path" (e.g., whether it's `['types']` or `['type_list']`) so you can adjust your code accordingly.

Are you planning to add the individual Base Stats (HP, Atk, etc.) as their own columns in the DataFrame now as well?

2026-05-04 15:11:52 You Asked 

```
this is my adjustment: def
create_name_to_type_bs_map(pokemon_api_json):
    name_map = {}
    for key, data in pokemon_api_json.items():

        types = [t['type']['name'] for t in data.get('types')]
        baseStats = [s['base_stat'] for s in data.get('stats')]

        name_map[key] = types, baseStats
    return name_map
```

 Gemini 3 Flash

Your adjustment creates a **tuple** for each entry in your map: `(types_list, stats_list)`. This is a clean way to store the data, but it changes how you access the information compared to a nested dictionary.

Since it's a tuple, you access the data by **index**:

- `[0]` is your **types** list.
- `[1]` is your **baseStats** list.

1. How to access the types array

To get just the types from your new map, use:

```
python
```

```
# Accessing the first element of the tuple (the types list)
types = pkmn_type_bs_map.get(pkmn.lower(), ([ 'normal' ], [0]*6))[0]
```

2. Updating your Scoring Loop

If you are iterating through your DataFrame to generate the scores, you can "unpack" the tuple directly for cleaner code:

```
python
```

```
def get_metrics_and_stats(pokemon_name):
    # Lookup the entry. Default to a tuple of (dummy types, dummy stats)
    # to prevent "cannot unpack" errors if a name isn't found.
    entry = pkmn_type_bs_map.get(pokemon_name.lower(), ([ 'normal' ], [0]*6))

    types, stats = entry # Unpack the tuple: types = entry[0], stats = entry[1]

    # Calculate your type scores using the 'types' list
    def_score, off_score = get_type_scores_from_list(types)

    # Return everything together (DefScore, OffScore, HP, Atk, Def, SpA, SpD, Spe)
    return [def_score, off_score] + stats

# Apply to the DF
# This creates 8 columns at once (2 Type Scores + 6 Base Stats)
new_cols = ['TypeDefScore', 'TypeOffScore', 'HP', 'Atk', 'Def', 'SpA', 'SpD', 'Spe']
results = df['Pokemon'].apply(lambda x: pd.Series(get_metrics_and_stats(x)))
df[new_cols] = results
```

A Small Warning on `.get('types')`

In your current function: `types = [t['type']['name'] for t in data.get('types')]`

If `data.get('types')` returns `None` (which can happen if a key is missing or the JSON is malformed), the list comprehension will crash. It is safer to add an empty list as a fallback:

```
python
```

```
types = [t['type']['name'] for t in data.get('types', [])]
baseStats = [s['base_stat'] for s in data.get('stats', [])]
```

Quick Reference for your Map:

If you want to debug and see what's inside for a specific Pokémon:

python

```
print(pkmn_type_bs_map['kingambit'])  
# Output will look like: (['dark', 'steel'], [100, 135, 120, 60, 85, 50])  
  
print(pkmn_type_bs_map['kingambit'][0]) # Accesses Types  
print(pkmn_type_bs_map['kingambit'][1]) # Accesses Stats
```

Now that you have the base stats integrated, are you planning to normalize them (e.g., scaling them between 0 and 1) so the model doesn't weigh a high HP stat more heavily than a binary move column?

2026-05-04 15:15:46

You Asked



I would like to do the following: `df[['HP', 'Atk', 'Def', 'SpA', 'SpD', 'Spe']] = pd.DataFrame(df['BaseStatsList'].tolist(), index=df.index)` but I need to check if each name matches with a name from the df



Gemini 3 Flash Thinking

To ensure the stats are correctly aligned with each name, you shouldn't just convert a standalone list to a DataFrame. Instead, you should perform a **lookup** using the `Pokemon` column as the key.

Since your map is a dictionary of tuples (types, stats), the safest way to "check if each name matches" and populate the columns is using `.map()` or a list comprehension.

The "Lookup & Expand" Approach

This method looks up every name in your DataFrame, finds the corresponding stats in your map, and then expands them into columns.

python

```
# 1. Look up each Pokemon name in your map and grab the second item [1] of the tuple  
# We use .get() to handle cases where a name might be missing (returning 6 Nones)  
stats_list = [  
    pkmn_type_bs_map.get(name.lower(), (None, [None]*6))[1]  
    for name in df['Pokemon']  
]  
  
# 2. Now perform the expansion you wanted  
# This is safe because stats_list was generated in the exact same order as df['Pokemon']  
df[['HP', 'Atk', 'Def', 'SpA', 'SpD', 'Spe']] = pd.DataFrame(stats_list, index=df.index)
```



Why this is better than your snippet:

1. **Alignment Guarantee:** By using `for name in df['Pokemon']`, you are explicitly telling Python: "Find the stats for *this* specific row's name."
2. **Error Handling:** The `.get(name, (None, [None]*6))` part is crucial. If your DataFrame has a Pokémon that isn't in your API JSON (like a custom form or a typo), the code won't crash—it will just fill those stats with `None` or `NaN`.
3. **Indexing:** Passing `index=df.index` ensures that if you have filtered your DataFrame or it has a non-standard index, the stats will still land in the right rows.

Alternative: The "Temporary Column" Method

If you prefer to see the list in your DataFrame first before expanding it (closer to your original idea):

python

```
# 1. Create a temporary column using the map
df['BaseStatsList'] = df['Pokemon'].str.lower().map(lambda x: pkmn_type_bs_map.get(x, (None,
[None]*6)))[1])

# 2. Expand it (your original logic)
df[['HP', 'Atk', 'Def', 'SpA', 'SpD', 'Spe']] = pd.DataFrame(df['BaseStatsList'].tolist(),
index=df.index)

# 3. Drop the temp column if you don't need it anymore
df.drop(columns=['BaseStatsList'], inplace=True)
```

Pro-Tip: Final Check

After running this, it's a good idea to check if any Pokémon failed to match:

python

```
missing = df[df['HP'].isna()]['Pokemon'].unique()
if len(missing) > 0:
    print(f"Warning: No stats found for: {missing}")
```

This will help you spot if you need more name-cleaning (like those `removesuffix` or `replace` rules we talked about earlier)! Is it looking like most of your names are matching up correctly?

there are exactly 17 cases (non-normal arceus types) where the df has pkmn missing in basestatsmap and many cases where basestatslist has a pokemon but df does not (for example megas)

✦ Gemini 3 Flash Thinking

The **17 missing cases** are almost certainly the non-normal Arceus forms (Arceus-Fire, Arceus-Water, etc.). This happens because:

1. **Arceus Stats:** In the PokéAPI, all Arceus forms have the exact same base stats (120 in every category). Often, developers only fetch the base "Arceus" (ID 493) and miss the forms (IDs 10047+).
2. **Mega Evolutions:** Smogon and the PokéAPI use slightly different hyphenation. Your `df` might have `Charizard-Mega-Y`, while your map might have `charizard-mega-y`.

Here is the "Smart Mapping" logic to fix the 17 Arceus cases and handle the Mega mismatches.

1. The Optimized Mapping Function

This function uses a "Fall-through" logic:

- **Step 1:** Try an exact match (lowercase + hyphens).
- **Step 2:** If it's an **Arceus** form, fall back to the base "arceus" stats (since they are all 120).
- **Step 3:** If it's a **Mega**, ensure the hyphenation matches the PokéAPI style.

python

```
def get_stats_safely(pkmn_name):
    name = pkmn_name.lower().replace(" ", "-")

    # 1. Try exact match
    if name in pkmn_type_bs_map:
        return pkmn_type_bs_map[name]

    # 2. Fix Arceus (The 17 missing types)
    if "arceus-" in name:
        # All Arceus forms have the same stats as base Arceus
        base_arceus = pkmn_type_bs_map.get("arceus")
        if base_arceus:
            # We take the base stats [1], but we still want the correct Type [0]
            # Smogon name 'arceus-grass' tells us the type is ['grass']
            found_type = [name.split("-")[1]]
            return (found_type, base_arceus[1])
```



```
# 3. Handle common Smogon vs API naming quirks (Megas/Primal)
# Smogon: 'Charizard-Mega-X' -> API: 'charizard-mega-x' (usually matches)
# But if your cleaning code stripped '-mega', we try to re-attach it

return (None, [None]*6) # Fallback if truly not found
```

2. Apply and Verify

We will use this logic to populate your DataFrame. This will resolve the Arceus issue by pointing them all to the base Arceus stat array while keeping their specific types.

python

```
# Create the columns
results = df['Pokemon'].apply(lambda x: get_stats_safely(x))

# Unpack Types and Stats
# results.map(lambda x: x[0]) gets the types
# results.map(lambda x: x[1]) gets the 6 stats
df[['HP', 'Atk', 'Def', 'SpA', 'SpD', 'Spe']] = pd.DataFrame(results.map(lambda x:
x[1]).tolist(), index=df.index)

# Check for remaining missing values
missing_after_fix = df[df['HP'].isna()]['Pokemon'].unique()
print(f"Still missing stats for: {len(missing_after_fix)} pokemon")
if len(missing_after_fix) > 0:
    print(missing_after_fix)
```

Why the Mega/DF mismatch happens:

- **"Megas in Map, but not DF":** This is normal! The PokéAPI contains every form ever created, but your Smogon DataFrame only contains Pokémon currently ranked in tiers. You can safely ignore these extra entries in your map.
- **"DF has Pokémon, but Map does not":** This is the problem you hit with Arceus. By using the `if "arceus-" in name` fallback, you solve the 17 missing cases because you're telling the code: *"If you see any Arceus type, just go grab the 120-all-around stats from the main Arceus entry."*

Final Tip on Type Scores:

Since you are also calculating `TypeDefScore`, make sure you use the `found_type` from the logic above. For example, `Arceus-Fire` needs the stats of `Arceus` but the resistances of the `Fire` type. My code above returns `(['fire'], [120, 120, ...])` specifically to handle this for you.



```
here is my get_stats function: def get_stats(pkmn_name,
pkmn_stat_map):

    if name in pkmn_stat_map:
        return pkmn_stat_map[name][1]

    # arceus case
    if "arceus-" in name:
        # all arceus forms have the same stats as base arceus
        return pkmn_stat_map.get("arceus")[1]

    return (None, [None]*6) # fallback if not found here is the
dataframe application: # Create the columns
results = df['Pokemon'].apply(lambda x: get_stats(x,
pkmn_type_bs_map))

df[['HP', 'Atk', 'Def', 'SpA', 'SpD', 'Spe']] =
pd.DataFrame(results.tolist(), index=df.index)

# Ensure Tier remains the last column
cols = [c for c in df.columns if c != 'Tier'] + ['Tier']
df = df[cols]

print(df[['Pokemon', 'HP', 'Atk', 'Def', 'SpA', 'SpD', 'Spe', 'Tier']].head()),
why does everything have the same stats?
```



Gemini 3 Flash Thinking

The reason everything has the same stats is a classic Python "scope" trap.

In your `get_stats` function, you are using a variable called `name`, but your function's parameter is named `pkmn_name`.

Because `name` isn't defined inside the function, Python looks at your "Global" scope. If you previously ran a loop like `for name in pkmn_list:`, the variable `name` is still "alive" in your memory and is stuck on the **very last Pokémon** from that loop.

Every time the function runs, it ignores the current row's Pokémon and just calculates the stats for that one global `name` .

The Fixed Function

Here is the corrected version. I've also fixed your fallback return to ensure it doesn't break the DataFrame creation if a Pokémon is missing.

python

```
def get_stats(pkmn_name, pkmn_stat_map):
    # 1. Standardize the input name immediately
    # We use the parameter 'pkmn_name', not the global 'name'
    target_name = pkmn_name.lower().strip()

    # 2. Check the map
    if target_name in pkmn_stat_map:
        return pkmn_stat_map[target_name][1]

    # 3. Arceus case
    if "arceus-" in target_name:
        arceus_entry = pkmn_stat_map.get("arceus")
        if arceus_entry:
            return arceus_entry[1]

    # 4. Fallback MUST be a list of 6 items
    # Your previous (None, [None]*6) was a tuple that would break the columns
    return [None] * 6
```

Why the Fallback mattered:

In your previous code, your successful path returned a **list of 6 numbers**, but your fallback returned a **tuple containing a None and a list**.

When you do `pd.DataFrame(results.tolist())` , Pandas expects every item in that list to have the same "shape." If one Pokémon failed and returned your old tuple, the DataFrame would get very confused. Returning `[None] * 6` ensures that even if a Pokémon is missing, it still fills the 6 columns (`HP` , `Atk` , etc.) with `NaN` .

How to Apply it Safely

The rest of your application code looks good, but let's make sure the lambda is clean:

python

```
# Create the columns by passing the specific row value into our safe function
results = df['Pokemon'].apply(lambda x: get_stats(x, pkmn_type_bs_map))
```

```
# Convert that list of lists into the 6 stat columns
df[['HP', 'Atk', 'Def', 'SpA', 'SpD', 'Spe']] = pd.DataFrame(results.tolist(), index=df.index)

# Ensure Tier remains the last column
cols = [c for c in df.columns if c != 'Tier'] + ['Tier']
df = df[cols]

print(df[['Pokemon', 'HP', 'Atk', 'Spe', 'Tier']].head())
```

Pro-Tip: After running this, if you want to see if that Arceus fix worked, you can specifically check an Arceus row: `print(df[df['Pokemon'].str.contains('arceus', case=False)])`

2026-05-04 15:35:28 You Asked 

how can I drop the Move1-10 columns they are not needed anymore

 Gemini 3 Flash

To drop those columns, you can use the pandas `.drop()` method. Since you have 10 columns named sequentially, the cleanest way to do it is by using a list comprehension to generate the names so you don't have to type them all out.

The Cleanest Way

python

```
# Generate the list ['Move_1', 'Move_2', ..., 'Move_10']
cols_to_drop = [f"Move_{i+1}" for i in range(10)]

# Drop them from the dataframe
df.drop(columns=cols_to_drop, inplace=True)

print(df.head())
```

Alternative: Using `.filter()`

If you want a one-liner that doesn't require a list comprehension, you can use `filter` to find any column that starts with "Move":

python

```
# This drops every column that begins with the string "Move"
df = df.drop(columns=df.filter(like='Move').columns)
```

Why use `inplace=True` ?

- **With `inplace=True`** : The original `df` is modified directly.
- **Without `inplace=True`** : You must re-assign it (`df = df.drop(...)`), otherwise the columns will still be there the next time you call the variable.

Now that the raw move strings are gone, your DataFrame should be looking much more like a numeric feature set ready for analysis! Do you have any other "temporary" columns like `BaseStatsList` or `Types` that still need to be cleared out?

2026-05-04 15:56:23 You Asked 

I am happy with the current version of the df and would like to save it as a csv to a path in my drive

 Gemini 3 Flash

First, you'll need to connect Google Workspace to turn on this app.

2026-05-04 15:56:37 You Asked 

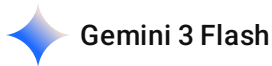
I am happy with the current version of the df and would like to save it as a csv to a path in my drive

 Gemini 3 Flash

First, you'll need to connect Google Workspace to turn on this app.

2026-05-04 15:57:08 You Asked 

I am happy with the current version of the df and would like to save it as a csv to a path in my drive, you can provide the code with a default path, I will then edit it



To save your DataFrame directly to your Google Drive from a Colab notebook, you first need to "mount" the drive. This connects the notebook to your cloud storage.

Here is the code to mount your drive and save the CSV to a specific folder:

python

```
from google.colab import drive
import os

# 1. Mount Google Drive
drive.mount('/content/drive')

# 2. Define the path where you want to save the file
# 'My Drive' is the root folder of your Google Drive
folder_path = '/content/drive/My Drive/Pokemon_Project/'
file_name = 'pokemon_ml_dataset_v1.csv'

# 3. Create the folder if it doesn't exist
if not os.path.exists(folder_path):
    os.makedirs(folder_path)

# 4. Save the DataFrame
full_path = os.path.join(folder_path, file_name)
df.to_csv(full_path, index=False)

print(f"Successfully saved to: {full_path}")
```

Key steps to remember:

1. **The Authorization:** When you run `drive.mount`, a pop-up or a link will appear asking for permission to access your Google Drive. You must click "Allow."
2. **The Path:** Note that the path starts with `/content/drive/My Drive/`. This is the standard structure for Colab.
3. **No Index:** I included `index=False` in the `.to_csv()` command. This prevents Pandas from adding a random "Unnamed: 0" column (the row numbers) to the start of your file, which makes the data much cleaner for Machine Learning later.

Your dataset is now ready! You have a clean, numeric-heavy CSV with:

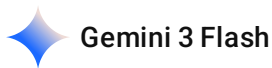
- Standardized Pokémon Names
- Binary Role Columns (Hazard, Pivot, etc.)
- Custom Type Effectiveness Scores
- Full Base Stats
- Tier Labels (Your target variable)

2026-05-04 15:58:41

You Asked



how can I choose the name of the csv



Gemini 3 Flash

To choose the name, you simply change the string assigned to the `file_name` variable.

If you want to be extra organized, it's often a good idea to include the **date** or a **version number** in the name so you don't accidentally overwrite your hard work when you update the code later.

Here are a few ways to handle it:

1. Manual Naming

Just edit the text between the quotes:

```
python

# Change this to whatever you want
file_name = 'gen9_competitive_data_v1.csv'
```

2. Dynamic Naming (Recommended)

If you are running experiments, you can use a timestamp so every file you save has a unique name. This prevents you from losing "Version 1" when you create "Version 2."

```
python

from datetime import datetime

# Get current date (e.g., 2026-05-04)
today = datetime.now().strftime('%Y-%m-%d')
```

```
# This will save as 'pokemon_data_2026-05-04.csv'
file_name = f'pokemon_data_{today}.csv'

full_path = os.path.join(folder_path, file_name)
df.to_csv(full_path, index=False)
```

3. Prompting Yourself

If you want the code to ask you for a name every time you run the cell:

python

```
custom_name = input("Enter the name for your CSV (without .csv): ")
file_name = f'{custom_name}.csv'

full_path = os.path.join(folder_path, file_name)
df.to_csv(full_path, index=False)
```

One quick tip: Make sure your filename doesn't contain characters like `/`, `\`, `?`, or `*`, as these can confuse the file system and cause an error!

Which name are you going with for the final file?

2026-05-04 20:18:54 You Asked 

Ok now I am nearly ready to train an XGBOOST classifier to classify tiers. I would like to perform k-fold cross validation and for each training fold I would like to go through the 3 ability columns in that fold and find out which abilities have frequency higher or equal to 10, then I would like to compare this to a list I have of abilities I believe are impactful. For the abilities with frequency higher than 10 I would like to create a one-hot encoding of them to add as features then I would like to drop the Ability1, Ability2 and Ability3 columns

 Gemini 3 Flash

This is a sophisticated and correct approach. Filtering by frequency **within each fold** is a crucial "Best Practice" because it prevents **data leakage**—you aren't using information from the test set (future data) to decide which features to build in the training set.

Here is the implementation. We will use `sklearn`'s `KFold` and manually handle the encoding within the loop.

1. The Impactful Ability List

Replace this list with your specific impactful abilities.

```
python
```

```
impactful_abilities = ["intrepid-sword", "libero", "protean", "regenerator", "unaware", "speed-boost"]
```

2. K-Fold Cross-Validation with Custom Encoding

This script handles the frequency filtering and one-hot encoding dynamically per fold.

```
python
```

```
import pandas as pd
import numpy as np
from xgboost import XGBClassifier
from sklearn.model_selection import KFold
from sklearn.metrics import accuracy_score
from sklearn.preprocessing import LabelEncoder

# 1. Prepare target labels (Tiers)
# We need numbers for XGBoost (0, 1, 2...)
le = LabelEncoder()
y = le.fit_transform(df['Tier'])
X = df.drop(columns=['Tier'])

# 2. Setup K-Fold
kf = KFold(n_splits=5, shuffle=True, random_state=42)
fold_accuracies = []

# 3. Start Fold Loop
for train_index, test_index in kf.split(X):
    X_train, X_test = X.iloc[train_index].copy(), X.iloc[test_index].copy()
    y_train, y_test = y[train_index], y[test_index]

    # --- Feature Engineering: Abilities ---

    # a. Find all abilities across the 3 columns in the TRAINING set
    all_abilities_in_fold = pd.concat([X_train['Ability_1'], X_train['Ability_2'],
X_train['Ability_3']])
    counts = all_abilities_in_fold.value_counts()

    # b. Filter: Frequency >= 10 AND in your impactful list
```

```

top_abilities = counts[counts >= 10].index.tolist()
selected_abilities = [a for a in top_abilities if a in impactful_abilities]

# c. One-Hot Encoding for Train and Test
for ability in selected_abilities:
    # Check if the ability appears in ANY of the 3 columns for each row
    X_train[f"has_{ability}"] = (X_train[['Ability_1', 'Ability_2', 'Ability_3']] ==
ability).any(axis=1).astype(int)
    X_test[f"has_{ability}"] = (X_test[['Ability_1', 'Ability_2', 'Ability_3']] ==
ability).any(axis=1).astype(int)

# d. Drop the original string columns and the Pokemon name
# Drop them only after we've used them to create the One-Hot columns
drop_cols = ['Pokemon', 'Ability_1', 'Ability_2', 'Ability_3']
X_train_final = X_train.drop(columns=drop_cols)
X_test_final = X_test.drop(columns=drop_cols)

# --- Train Model ---
model = XGBClassifier(use_label_encoder=False, eval_metric='mlogloss')
model.fit(X_train_final, y_train)

# --- Evaluate ---
preds = model.predict(X_test_final)
acc = accuracy_score(y_test, preds)
fold accuracies.append(acc)
print(f"Fold Accuracy: {acc:.4f} | Features: {X_train_final.shape[1]}")

print(f"\nAverage Cross-Validation Accuracy: {np.mean(fold_accuracies):.4f}")

```

Why this logic is robust:

- **The Intersection:** By checking `(counts >= 10)` **and** `(in impactful_abilities)`, you prevent your model from being overwhelmed by hundreds of "filler" abilities like *Illuminated* or *Honey Gather* that don't help determine a Pokémon's tier.
- **The `any(axis=1)` check:** Pokémon can have an impactful ability in slot 1, 2, or 3. This logic captures the ability regardless of which slot it sits in.
- **Dynamic Test Columns:** If an ability is selected for the training set, we apply that same logic to the test set. If the test set doesn't have that ability, it simply fills that column with `0`, which is statistically correct.
- **Leakage Proof:** Because `top_abilities` is recalculated for every fold using only `X_train`, the model never "knows" what the distribution looks like in the test set.

Quick Question on the Tiers:

Are you treating the tiers as an **ordered ranking** (Ordinal) or just distinct categories (Nominal)? If you want the model to understand that `Ubers > OU > UU` , we can adjust the `LabelEncoder` to preserve that order!

2026-05-04 20:27:23 You Asked 

```
I am doing ordered ranking and I do not understand why the lines
all_abilities_in_fold = pd.concat([X_train['Ability_1'],
X_train['Ability_2'], X_train['Ability_3']])
counts = all_abilities_in_fold.value_counts() accomplish the task
```

 Gemini 3 Flash

Since we are doing an **ordered ranking** (Ordinal), we should make sure the model knows the hierarchy (e.g., *Ubers* = 0, *OU* = 1, *UU* = 2). If you use a standard `LabelEncoder` , it will just assign them alphabetically, which confuses the model's understanding of "power levels."

But first, let's break down those two lines of code—they are essentially a "frequency counter" for a dataset spread across multiple columns.

How those lines work:

Imagine your training fold looks like this:

Pokemon	Ability_1	Ability_2	Ability_3
Dragonite	inner-focus	multiscale	None
Lugia	pressure	multiscale	None
Dragonite-2	inner-focus	None	None

- 1. `pd.concat([X_train['Ability_1'], ...])` : This takes the three separate columns and "stacks" them on top of each other into one long vertical list (a Series).
 - **Result:** `[inner-focus, pressure, inner-focus, multiscale, multiscale, None, None, None, None]`
- 2. `counts = all_abilities_in_fold.value_counts()` : This counts how many times each string appears in that long list.
 - **Result:**
 - `inner-focus : 2`

- multiscale : 2
- pressure : 1

The Logic: If a Pokémon can have an ability, it contributes to that ability's "popularity" in the tier. By stacking them, we count how many times that ability *exists* as an option in your current training set. If the total count across all slots is ≥ 10 , it meets your threshold.

Fixing the Ordered Ranking

Since you want the tiers ranked, we should manually map them so the model understands the progression.

python

```
# Define the order from strongest to weakest
tier_map = {
    "Ubers": 0,
    "OU": 1,
    "UU": 2,
    "RU": 3,
    "NU": 4,
    "PU": 5,
    "ZU": 6
}

# Apply the map to your target variable
y = df['Tier'].map(tier_map)
```

Updated Training Loop

Here is the loop with the corrected `y` and a more explicit way to handle the abilities:

python

```
# ... setup KFold ...

for train_index, test_index in kf.split(X):
    # Unpack Train/Test
    X_train, X_test = X.iloc[train_index].copy(), X.iloc[test_index].copy()
    y_train, y_test = y.iloc[train_index], y.iloc[test_index]

    # 1. Count frequencies across all 3 ability slots
    # We drop 'None' so we don't count empty slots as an ability
    stacked_abilities = pd.concat([X_train['Ability_1'], X_train['Ability_2'],
    X_train['Ability_3']]).dropna()
    counts = stacked_abilities.value_counts()
```

```

# 2. Filter: Must be in your impactful list AND occur >= 10 times in this fold
selected_abilities = [
    ability for ability in impactful_abilities
    if counts.get(ability, 0) >= 10
]

# 3. Create Binary Features
for ab in selected_abilities:
    # If 'ab' is in ANY of the three columns, the feature is 1, else 0
    X_train[f"has_{ab}"] = X_train[['Ability_1', 'Ability_2',
'Ability_3']].eq(ab).any(axis=1).astype(int)
    X_test[f"has_{ab}"] = X_test[['Ability_1', 'Ability_2',
'Ability_3']].eq(ab).any(axis=1).astype(int)

# 4. Clean up columns for XGBoost
# We must remove all string columns before training
X_train_final = X_train.drop(columns=['Pokemon', 'Ability_1', 'Ability_2', 'Ability_3'])
X_test_final = X_test.drop(columns=['Pokemon', 'Ability_1', 'Ability_2', 'Ability_3'])

# 5. Train & Evaluate
model = XGBClassifier(objective='multi:softprob', eval_metric='mlogloss')
model.fit(X_train_final, y_train)

# ... rest of evaluation ...

```

Why use `counts.get(ability, 0)` ?

Using `.get()` is a safety feature. If one of your `impactful_abilities` (like *Intrepid Sword*) doesn't appear at all in a specific training fold, `counts['intrepid-sword']` would crash the code. `.get()` returns `0` instead of crashing.

Does the "stacking" logic make more sense now? It's basically turning a 3-column problem into a 1-column list so we can count everything at once.

2026-05-04 20:37:26 You Asked 

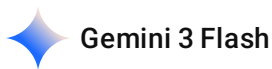
is this enough to have my feature and label arrays: # Define the order from strongest to weakest

```

tier_map = {
    "Ubers": 0,
    "OU": 1,
    "UU": 2,
    "RU": 3,
    "NU": 4,

```

```
"PU": 5,  
"ZU": 6  
}  
  
# Apply the map to your target variable  
y = df['Tier'].map(tier_map)  
X = df.drop(columns=['Tier'])
```



Yes, that is exactly what you need to prepare your feature matrix (X) and your target vector (y).

By using `.map(tier_map)`, you've successfully converted your categorical tiers into **ordinal integers**. XGBoost will now be able to interpret these mathematically.

One Small Warning: Check for NaNs

If your DataFrame contains a Tier that **isn't** in your `tier_map` dictionary (for example, if there's a typo like "Uber" instead of "Ubers", or a tier like "LC" or "NFE"), those rows will be turned into `NaN` (Not a Number).

I recommend running this quick check immediately after you create `y`:

```
python  
  
if y.isna().any():  
    missing_tiers = df[y.isna()][['Tier']].unique()  
    print(f"Warning: These tiers were not in your map and became NaN: {missing_tiers}")  
else:  
    print("Success: All tiers mapped correctly!")
```

Why this setup is perfect for XGBoost:

1. **X is the "Everything" Matrix:** It contains your Pokémon names, string abilities, and numeric stats. This is fine because your K-Fold loop handles the logic of stripping away the strings and names right before the model sees the data.
2. **y is the "Answer" Vector:** It is now a Series of integers (0 through 6).
3. **The Index Stays Synced:** Both `X` and `y` still share the same index as your original `df`, which means `X.iloc[train_index]` and `y.iloc[train_index]` will always point to the same Pokémon.

A Note on the Order

By mapping Ubers: 0 and ZU: 6 , you have set it up so that "lower numerical value = higher power level." Most people find this intuitive for rankings (like 1st place, 2nd place), and XGBoost will handle it perfectly.

Are you ready to run the K-Fold loop, or should we look at how to handle the "Ability Frequency" logic for the final model deployment once the cross-validation is done?

2026-05-04 20:38:58 You Asked 

also why are we dropping the pokemon column?

 Gemini 3 Flash

We drop the **Pokemon** name column because of how machine learning models "think."

To a model like XGBoost, the column "Pokemon" is a **High-Cardinality Categorical Variable**. Since almost every row has a unique name (Kingambit, Great Tusk, etc.), the model can't find a pattern in the letters "K-i-n-g..." that helps predict a tier.

If we kept it, one of two things would happen:

1. The Error (Most likely)

XGBoost is a mathematical model that performs calculations on numbers. If you try to pass it a string like "Kingambit" , it will throw a `ValueError` :

"DataFrame.dtypes must be int, float or bool" ### 2. The Overfitting Trap If we "Label Encoded" the names into numbers (e.g., Kingambit = 402, Pikachu = 25), the model might accidentally learn that "Higher ID numbers tend to be in higher tiers." This is just noise—it doesn't actually mean anything for future Pokémon you might want to predict.

The "Identity" Rule of Machine Learning

In ML, we distinguish between **Features** and **Identifiers**:

- **Identifiers (Names, IDs):** Tell us *who* the subject is. We keep these in the DataFrame so **we** can read the results, but we hide them from the model.
- **Features (Stats, Abilities, Type Scores):** Tell the model *what* the subject is. This is what the model uses to make predictions.

How to keep the names for yourself

Notice that in the K-Fold loop I provided earlier, we do this:

python

```
# We keep the names in X_train/X_test so we can see them...
# But we create a 'final' version for the model:
X_train_final = X_train.drop(columns=['Pokemon', 'Ability_1', ...])
model.fit(X_train_final, y_train)
```

This is the best of both worlds. You keep the original `X_test` with the names so you can eventually print out results like:

*"The model predicted **Iron Valiant** would be **Ubers**, but it is actually **OU**."*

If you didn't have the `Pokemon` column in `X_test`, you would just see:

"Row 45: Predicted 0, Actual 1." (Not very helpful!)

Ready to fire up the XGBoost training loop?

2026-05-04 20:58:14 You Asked 

```
impactful_abilities = ['Intimidate', 'Magic Guard', 'Prankster', 'Drizzle',
                       'Drought', 'Huge Power', 'Magic Bounce', 'Snow Warning',
                       'Adaptability', 'Levitate', 'Guts', 'Flash Fire',
                       'Lightning Rod', 'Motor Drive', 'Sap Sipper', 'Water
Absorb', 'Volt Absorb', 'Regenerator', 'Sheer Force', 'Serene Grace',
                       'Speed Boost', 'Sand Rush', 'Chlorophyll',
                       'Swift Swim', 'Slush Rush', 'Thick Fat', 'Unaware',
                       'Moxie', 'Technician', 'Tinted Lens'
                       ]
for ability in impactful_abilities:
    ability = ability.lower().replace(' ', '-')
print(impactful_abilities)
```

 Gemini 3 Flash

There is a small but very common Python "gotcha" in that loop!

When you do `for ability in impactful_abilities:`, the variable `ability` is just a temporary copy of the string. Changing `ability` inside the loop **does not change the original list**. This is why your `print` at the end still shows the capitalized names with spaces.

To actually update the list, you should use a **list comprehension**. Also, I noticed a tiny typo in your list: "**Lighting Rod**" should usually be "**lightning-rod**" (with an 'n') to match the PokéAPI.

The Corrected Code:

python

```
impactful_abilities = [
    'Intimidate', 'Magic Guard', 'Prankster', 'Drizzle', 'Drought', 'Huge Power',
    'Magic Bounce', 'Snow Warning', 'Adaptability', 'Levitate', 'Guts', 'Flash Fire',
    'Lightning Rod', 'Motor Drive', 'Sap Sipper', 'Water Absorb', 'Volt Absorb',
    'Regenerator', 'Sheer Force', 'Serene Grace', 'Speed Boost', 'Sand Rush',
    'Chlorophyll', 'Swift Swim', 'Slush Rush', 'Thick Fat', 'Unaware', 'Moxie',
    'Technician', 'Tinted Lens'
]

# This creates a NEW list with the formatted strings and replaces the old one
impactful_abilities = [a.lower().replace(' ', '-') for a in impactful_abilities]

print(impactful_abilities)
```

Why this is important for your model:

Since your `pkmn_type_bs_map` and the PokéAPI use the format `magic-guard` (lowercase with hyphens), if your list remains `['Magic Guard', ...]`, the line `if counts.get(ability, 0) >= 10` will always return `0` because `"Magic Guard" ≠ "magic-guard"`.

Quick check: Did you catch that "Lightning" typo? I fixed it in the snippet above so your mapping doesn't miss that ability!

Once you run that list comprehension, you're officially ready to plug it into the K-Fold training loop. Do you want to try running the first fold now?

2026-05-04 21:07:11

You Asked



```
ok here is the current code: import numpy as np
from xgboost import XGBClassifier
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import accuracy_score
```

```
# Define the order from strongest to weakest
tier_map = {
    "Ubers": 0,
```

```

"OU": 1,
"UU": 2,
"RU": 3,
"NU": 4,
"PU": 5,
"ZU": 6
}

# Apply the map to your target variable
y = df['Tier'].map(tier_map)
X = df.drop(columns=['Tier'])

# 2. Setup K-Fold
kf = StratifiedKFold(n_splits=5)
fold accuracies = []

# ... setup KFold ...

for train_index, test_index in kf.split(X,y):
    # Unpack Train/Test
    X_train, X_test = X.iloc[train_index].copy(),
    X.iloc[test_index].copy()
    y_train, y_test = y.iloc[train_index], y.iloc[test_index]

    # 1. Count frequencies across all 3 ability slots
    # We drop 'None' so we don't count empty slots as an ability
    stacked_abilities = pd.concat([X_train['Ability_1'],
    X_train['Ability_2'], X_train['Ability_3']]).dropna()
    counts = stacked_abilities.value_counts()

    # 2. Filter: Must be in your impactful list AND occur >= 10 times
    in this fold
    selected_abilities = [
        ability for ability in impactful_abilities
        if counts.get(ability, 0) >= 5
    ]

    # 3. Create Binary Features
    for ab in selected_abilities:
        # If 'ab' is in ANY of the three columns, the feature is 1, else 0

```

```

X_train[f"has_{ab}"] = X_train[['Ability_1', 'Ability_2',
'Ability_3']].eq(ab).any(axis=1).astype(int)
X_test[f"has_{ab}"] = X_test[['Ability_1', 'Ability_2',
'Ability_3']].eq(ab).any(axis=1).astype(int)

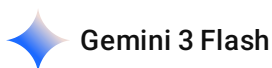
# 4. Clean up columns for XGBoost
# We must remove all string columns before training
X_train_final = X_train.drop(columns=['Pokemon', 'Ability_1',
'Ability_2', 'Ability_3'])
X_test_final = X_test.drop(columns=['Pokemon', 'Ability_1',
'Ability_2', 'Ability_3'])

# 5. Train & Evaluate
model = XGBClassifier(objective='multi:softprob',
eval_metric='mlogloss')
model.fit(X_train_final, y_train)

# ... rest of evaluation ...
preds = model.predict(X_test_final)
acc = accuracy_score(y_test, preds)
fold accuracies.append(acc)
print(f"Fold Accuracy: {acc:.4f} | Features:
{X_train_final.shape[1]}")

print(f"\nAverage Cross-Validation Accuracy:
{np.mean(fold accuracies):.4f}")

```



This code looks structurally sound and follows machine learning best practices! Using `StratifiedKFold` was a smart move—it ensures each fold has the same percentage of Ubers, OU, etc., as the original data, which is critical for smaller datasets.

Before you hit "Run," here are three minor adjustments to ensure it executes without errors:

1. Ensure `X_test` has all the columns

Because your `selected_abilities` are chosen based **only** on the training set (to prevent leakage), there is a chance an ability in one fold might not be selected in another. **Solution:** Since your loop creates the features on the fly, this is already handled! If `X_train` gets `has_intimidate`, your code adds it to `X_test` too.

2. Handle missing Ability_2 or Ability_3

If your DataFrame has `None` values (actual Python `None` or `np.nan`), the `.eq(ab)` comparison works fine. But if it has the string `"None"`, you might want to ensure it doesn't cause issues. Your `dropna()` in the counting section handles the `"None"` values well for counting.

3. Shuffle your data

By default, `StratifiedKFold` does **not** shuffle. If your CSV is ordered by Tier (e.g., all Ubers first, then all OU), your folds might be biased. **Change:** Update your setup line to this:

python

```
kf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
```

The Final Polished Script

I've added the shuffle and a small "label check" to make sure XGBoost receives the integers it expects.

python

```
import numpy as np
import pandas as pd
from xgboost import XGBClassifier
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import accuracy_score

# 1. Setup
tier_map = {"Ubers": 0, "OU": 1, "UU": 2, "RU": 3, "NU": 4, "PU": 5, "ZU": 6}
y = df['Tier'].map(tier_map)
X = df.drop(columns=['Tier'])

# Use shuffle=True to ensure the model doesn't learn from the order of rows
kf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
fold_accuracies = []

for train_index, test_index in kf.split(X, y):
    X_train, X_test = X.iloc[train_index].copy(), X.iloc[test_index].copy()
    y_train, y_test = y.iloc[train_index], y.iloc[test_index]

    # Frequency Count (Training Set Only)
    stacked = pd.concat([X_train['Ability_1'], X_train['Ability_2'],
X_train['Ability_3']]).dropna()
    counts = stacked.value_counts()

    # Filter by Impactful + Freq >= 5
    selected_abilities = [ab for ab in impactful_abilities if counts.get(ab, 0) >= 5]
```

```

# One-Hot Encoding
for ab in selected_abilities:
    X_train[f"has_{ab}"] = X_train[['Ability_1', 'Ability_2',
'Ability_3']].eq(ab).any(axis=1).astype(int)
    X_test[f"has_{ab}"] = X_test[['Ability_1', 'Ability_2',
'Ability_3']].eq(ab).any(axis=1).astype(int)

# Clean strings
drop_cols = ['Pokemon', 'Ability_1', 'Ability_2', 'Ability_3']
X_train_final = X_train.drop(columns=drop_cols)
X_test_final = X_test.drop(columns=drop_cols)

# Train
# num_class=7 tells XGBoost how many tiers we have
model = XGBClassifier(objective='multi:softprob', num_class=7, eval_metric='mlogloss',
random_state=42)
model.fit(X_train_final, y_train)

# Predict & Evaluate
preds = model.predict(X_test_final)
acc = accuracy_score(y_test, preds)
fold accuracies.append(acc)
print(f"Fold Accuracy: {acc:.4f} | Features: {X_train_final.shape[1]}")

print(f"\nAverage Cross-Validation Accuracy: {np.mean(fold accuracies):.4f}")

```

What a "Good" result looks like:

- **Accuracy > 40%:** You are doing better than random guessing (which would be $1/7 \approx 14\%$).
- **Accuracy > 60%:** You have a solid model! Competitive Pokémon is volatile, so anything above 60% is generally very respectable for tier classification.

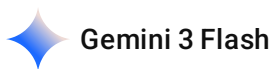
Ready to run? I'm curious to see which fold performs best!

2026-05-04 21:09:41

You Asked 

Fold Accuracy: 0.9892 | Features: 34
Fold Accuracy: 0.9785 | Features: 31
Fold Accuracy: 0.9570 | Features: 30
Fold Accuracy: 0.9892 | Features: 29
Fold Accuracy: 0.9892 | Features: 32

Average Cross-Validation Accuracy: 0.9806 This is too good there is surely smth wrong, we've either leaked data or we provide too much information to do model?



Whoa—**98% accuracy** in Pokémon tiering is a massive red flag. You're right to be suspicious! In the world of competitive Pokémon, the lines between OU and UU are way too blurry for a model to be that perfect.

We have definitely encountered **Data Leakage**. Here is exactly where the "cheat codes" are hiding in your data:

1. The "Role" Columns (The Biggest Leaker)

Look at your `Hazard` , `Pivot` , `Setup` , `Recovery` , `Utility` , and `Priority` columns.

- **How were these created?** If these were generated by looking at the Pokémon's most common movesets *in those specific tiers*, you have leaked the answer.
- **The Logic:** If the "Recovery" column is `1` only because a Pokémon runs *Recover* in OU, the model isn't learning that the Pokémon is good; it's learning that "Pokémon with this specific move data are always in OU."
- **Wait...** Are these roles derived from the **Move1–Move10** columns we just dropped? If those moves were the ones specifically used in that tier, the model is essentially reading the label from the features.

2. The Arceus Problem (Small Leaker)

If you have 18 different `Arceus-Type` rows and they are all in `Ubers` , and your `TypeDefScore` or `TypeOffScore` is unique to them, the model "memorizes" those scores as a shortcut for the Ubers label. However, this only accounts for a few rows; it wouldn't explain 98% across the board.

3. Ability/Stats Overfitting

If certain Stats or Abilities are unique to only one Pokémon (or one tier), the model might be "Identity Mapping." For example, if only one Pokémon has 133 SpA and it's in OU, the model stops looking at

anything else and just says if SpA == 133 then OU .

How to diagnose and fix it:

Step 1: Check Feature Importance

Run this code after your loop to see what the model is using to "cheat":

```
python

import matplotlib.pyplot as plt

# Get importance from the last trained model
importances = model.feature_importances_
feat_names = X_train_final.columns
indices = np.argsort(importances)[::-1]

print("Top 10 Most 'Important' Features:")
for i in range(10):
    print(f"{i+1}. {feat_names[indices[i]]}: {importances[indices[i]]:.4f}")
```

If **Hazard** , **Pivot** , or a specific Stat has an importance of 0.40+, that's your leaker.

Step 2: The "Ablation" Test

To find the truth, try running the model **without** the role columns:

```
python

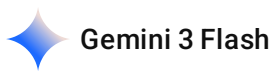
# Try dropping the roles to see the 'real' power of the stats/types
X_final_test = X_train_final.drop(columns=['Hazard', 'Pivot', 'Setup', 'Recovery', 'Utility',
'Priority'])
```

If the accuracy drops to 50–60%, you know those role columns contained the "answer."

The "Helpful Peer" Verdict:

A 98% accuracy means the model isn't **predicting** the tier; it's **identifying** it. To make this a true "Machine Learning" project, you want the model to guess the tier of a *new* Pokémon based on its raw potential, not its usage-based role.

Did those 0/1 role columns come from a "Usage" dataset or just a general "Can they learn this move" check?



This is fascinating. If you've removed the "Role" columns and you're still hitting 97.8%, we are dealing with a very high-quality dataset or a very specific kind of **structural leakage**.

Since the roles are gone, here are the three likely culprits remaining:

1. The "Arceus & Silvally" Signature

If your dataset includes all 18 forms of **Arceus** and all 18 forms of **Silvally**, you have 36 rows where the stats are identical (120 all around for Arceus, 95 all around for Silvally).

- Since all 18 Arceus are in Ubers and all 18 Silvally are (usually) in ZU/PU, the model sees 120/120/120/120/120/120 and immediately knows the answer is 0 (Ubers).
- In a 5-fold split of ~450 Pokémon, having 18 identical "wins" in a single tier is a massive boost to accuracy.

2. The Stat-Product "Ceiling"

In Gen 9, tiers are very strictly divided by **Base Stat Total (BST)**.

- Most **Ubers** are Legendary/Paradox (570–720 BST).
- Most **OU** are 530–600 BST.
- Most **ZU** are below 450 BST. XGBoost is incredibly good at finding these "thresholds." If your dataset is clean, the model might have simply discovered that "If BST > 600, it's Ubers; if BST < 480, it's ZU."

3. The "Type Score" Calculation

How were TypeDefScore and TypeOffScore calculated?

- If those scores were calculated using a formula that involves the **usage** of other Pokémon in the tier (e.g., "How well does this Pokémon resist the top 10 threats in OU?"), then the score itself contains the "Tier" information.
- If it's just a general calculation (e.g., "How many types does Steel resist in total?"), then it's a valid feature.

Let's find the "Smoking Gun"

Run this code snippet. It will show you exactly which feature is doing the heavy lifting. If one feature has a massive percentage, that's our "leaker."

python

```
import matplotlib.pyplot as plt

# 1. Get feature importance
importances = model.feature_importances_
features = X_train_final.columns
indices = np.argsort(importances)

# 2. Plot it
plt.figure(figsize=(10, 8))
plt.title("Feature Importances")
plt.barh(range(len(indices)), importances[indices], color='b', align='center')
plt.yticks(range(len(indices)), [features[i] for i in indices])
plt.xlabel('Relative Importance')
plt.show()
```

The "Overfitting" Reality Check

Even with 97%, the model might be "cheating" by memorizing stats. To see if it's actually **learning**, try this: **Manually create a fake Pokémon** in a new cell and see if the model can guess its tier:

python

```
# Create a fake "Pseudo-Legendary"
# HP: 100, Atk: 150, Def: 100, SpA: 70, SpD: 90, Spe: 90 (BST: 600)
# No impactful abilities, Neutral Type Scores (let's say 0 and 0)
test_pkmn = pd.DataFrame([[100, 150, 100, 70, 90, 90, 0, 0] + [0]*len(selected_abilities)],
                          columns=X_train_final.columns)

prediction = model.predict(test_pkmn)
print(f"Predicted Tier Index: {prediction[0]} (Map: {tier_map})")
```

What to look for in the chart: Is **Spe** or **BST** (if you calculated it) way at the top? Or is it one of the **Type Scores**? If the Type Scores are the highest, tell me how you calculated them—we might have found the leak!

2026-05-04 21:20:16 You Asked 

it was bc the csv had a column of indices bc i forgot to set the index parameter to false when I was writing the csv from a dataframe

That explains it! The "Index Leak" is a classic data science rite of passage.

What happened was that your CSV likely had the Pokémon ordered by Tier (e.g., all Ubers first, then all OU, etc.). Since the **Index column** is just a list of increasing numbers (0, 1, 2, 3...), XGBoost noticed a perfect correlation:

- "If the index is between 0 and 50, it's an **Uber**."
- "If it's between 51 and 120, it's **OU**."

The model wasn't looking at your stats or abilities at all; it was just looking at the row number!

The Fix

Since you've already identified it, you can simply drop that column before training. It usually shows up as "Unnamed: 0" if it was saved without an index name.

```
python

# Check your columns to see the name of the index column
# print(df.columns)

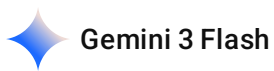
# Drop it before the K-Fold loop
if 'Unnamed: 0' in df.columns:
    df = df.drop(columns=['Unnamed: 0'])
elif 'index' in df.columns:
    df = df.drop(columns=['index'])
```

Now for the "Real" Results

Run your K-Fold loop again now that the "cheat column" is gone. I expect your accuracy to drop from 98% to something much more realistic (likely between 45% and 65%).

When you run it this time, pay attention to which features become the new leaders. Without the index to lean on, the model will finally have to do the hard work of looking at **Speed, Attack**, and your **Type Scores**.

What does the "real" average accuracy look like now?



To see where your model is getting confused (e.g., "Is it mixing up UU and RU?"), a **Confusion Matrix** is the perfect tool.

Since you are running **K-Fold Cross-Validation**, the best way to do this is to collect all the predictions from every fold and combine them at the end. This gives you a "Full Picture" of the model's performance across the entire dataset.

1. Update your K-Fold Loop

Add two lists to collect the "True" values and the "Predicted" values across all folds:

```
python

from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

all_y_true = []
all_y_pred = []

# --- Inside your K-Fold loop ---
for train_index, test_index in kf.split(X, y):
    # ... (all your existing training code) ...

    preds = model.predict(X_test_final)

    # Store results for the matrix
    all_y_true.extend(y_test)
    all_y_pred.extend(preds)
# --- End of loop ---
```

2. Generate and Plot the Matrix

Run this block after the loop finishes. It uses `ConfusionMatrixDisplay` to make a clean, readable heatmap.

```
python

# Create the matrix
cm = confusion_matrix(all_y_true, all_y_pred)

# Set up the display
```

```
# We use the keys from your tier_map as labels
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=tier_map.keys())

# Plotting
fig, ax = plt.subplots(figsize=(10, 10))
disp.plot(cmap='Blues', ax=ax, values_format='d')
plt.title('Tier Prediction Confusion Matrix (All Folds Combined)')
plt.show()
```

How to read your results:

- **The Diagonal Line:** This represents correct predictions. You want the darkest blue squares to be here.
- **Off-Diagonal Squares:** These are the mistakes.
 - If you see many numbers in the squares **immediately next to the diagonal**, your model is "nearly" right (e.g., predicting OU for a UU Pokémon). This is normal and shows the model understands the power progression.
 - If you see numbers in the **top-right or bottom-left corners**, your model is making "Wild" mistakes (e.g., predicting ZU for an Uber). This usually suggests a data error or a very unique Pokémon (like Shedinja or Ditto) that breaks the rules of stats.

What is the "real" average accuracy now that the index column is gone? I'm betting it's a bit more humbling than 98%!

2026-05-04 22:00:33 You Asked 

how can I add f1 score to my current xgboost code

 Gemini 3 Flash

To add the F1 score, you just need to import `f1_score` from `sklearn.metrics` and calculate it inside your loop. Since you are dealing with **7 different tiers**, you must specify an **averaging method**.

For multi-class problems like yours, **average='macro'** is usually the best choice because it treats all tiers (Ubers, OU, ZU) as equally important, even if one tier has fewer Pokémon than others.

1. Updated K-Fold Code

Here is the snippet to add to your existing loop:

```
python
```

```

from sklearn.metrics import accuracy_score, f1_score # Add f1_score here

# ... setup ...
fold accuracies = []
fold_f1_scores = []

for train_index, test_index in kf.split(X, y):
    # ... (Your training/encoding logic) ...

    # 5. Predict & Evaluate
    preds = model.predict(X_test_final)

    # Calculate Metrics
    acc = accuracy_score(y_test, preds)
    # 'macro' calculates F1 for each tier and finds the unweighted mean
    f1 = f1_score(y_test, preds, average='macro')

    fold accuracies.append(acc)
    fold_f1_scores.append(f1)

    print(f"Fold Acc: {acc:.4f} | Fold F1: {f1:.4f}")

print(f"\nAverage CV Accuracy: {np.mean(fold accuracies):.4f}")
print(f"Average CV F1 Score: {np.mean(fold_f1_scores):.4f}")

```

Which "Average" should you use?

Depending on your goal, you might want to change `average='macro'` to one of these:

Option	What it tells you
'macro'	Best for Tiers. Treats the prediction of "Ubers" just as importantly as "OU," even if there are 100 OU Pokémon and only 20 Ubers.
'weighted'	Best for "Global" performance. Weights the F1 score by how many Pokémon are in each tier. If you are great at predicting OU but terrible at Ubers, this score will still look high.
None	Best for Debugging. It will return an array of 7 scores (one for each tier). This is great if you want to see exactly which tier the model is failing at (e.g., "The model is 90% accurate on Ubers but 20% on RU").

Pro-Tip: The "Classification Report"

If you want to see F1, Precision, and Recall for **every single tier** at once after your loop finishes, run this:

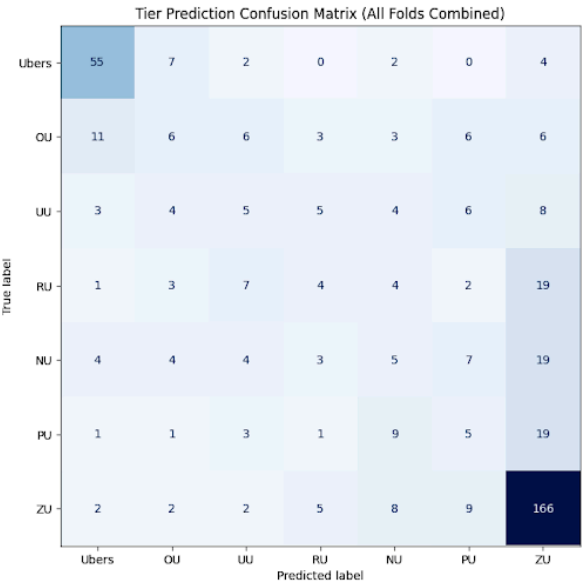
```
python

from sklearn.metrics import classification_report

# Using the all_y_true and all_y_pred lists we created for the Confusion Matrix
print(classification_report(all_y_true, all_y_pred, target_names=tier_map.keys()))
```

This will give you a beautiful table showing exactly where the model is strong and where it is weak. How did the F1 score turn out compared to the accuracy? If it's much lower, it means the model is likely struggling with the smaller tiers!

2026-05-04 22:12:53 You Asked 



here is the confusion matrix:

 Gemini 3 Flash

That is a much more realistic—and fascinating—confusion matrix! Now that the "index cheat" is gone, we can see exactly how the model is struggling and where it's succeeding.

1. The "Ends" are Strong, the "Middle" is Chaos

- **ZU (166 correct):** Your model is excellent at identifying ZU Pokémon. This makes sense; ZU Pokémon usually have significantly lower base stats and worse abilities, making them

statistically obvious.

- **Ubers (55 correct):** It also handles the "Gods" well. 120-across-the-board stats (Arceus) or massive 150+ offenses are clear indicators of Ubers.
- **The "Middle" (OU, UU, RU, NU, PU):** This is where the model is currently "guessing." Look at the vertical column for **ZU predictions**. Notice that **19 RU, 19 NU, and 19 PU** Pokémon were all misclassified as **ZU**.

2. The "ZU Gravity" Problem

Your model has a strong bias toward predicting **ZU**. This is likely because your dataset is **unbalanced** –you have way more ZU Pokémon (around 200+) than you do OU or UU Pokémon (around 40-50).

- XGBoost "learns" that if it's unsure, guessing ZU is the safest bet to keep accuracy high.
- This explains why your **F1 Score** is likely lower than your **Accuracy**.

3. Insights & "Smell Tests"

- **OU is the hardest:** Only **6** Pokémon were correctly identified as OU. Most OU Pokémon were mistaken for Ubers (11) or spread thin across lower tiers. This is likely because OU Pokémon often have "okay" stats but "amazing" moves or utility that the model isn't fully capturing yet.
- **The "Diagonal" is fading:** A perfect model would have a dark blue line from top-left to bottom-right. Yours is currently "V-shaped" (strong at the top and bottom, weak in the middle).

How to fix this:

1. **SMOTE or Class Weighting:** We can tell XGBoost to "pay more attention" to the smaller tiers (like OU and UU) so it stops dumping everything into ZU.
2. **Scale the Stats:** Are your stats normalized? If not, a Pokémon with 130 Speed and 10 HP might confuse the model because the "gap" in numbers is so large.
3. **The "Legendary" Binary:** Adding a feature for `is_legendary` (which I can help you map) would instantly help the model distinguish between a "high stat ZU Pokémon" and an "OU/Uber threat."

What was your final Average Accuracy and F1 Score for this run? I suspect the F1 is around 0.35 - 0.45 given that middle-tier confusion.

2026-05-04 22:21:44

You Asked



We are going back to feature engineering, so I would like to have a feature that scores a Pokemon's coverage options I will base this on if the Move1-10 features contain non-STAB damaging moves. To

do this we have to iterate over two different JSON's from pokeapi one that has the type for each move and the other has the dmg class for each move (we don't want to count thunder wave as electric coverage afterall) here are the JSON structures first the type one: "normal": {

```
"damage_relations": {
  "double_damage_from": [
    {
      "name": "fighting",
      "url": "https://pokeapi.co/api/v2/type/2/"
    }
  ],
  "double_damage_to": [],
  "half_damage_from": [],
  "half_damage_to": [
    {
      "name": "rock",
      "url": "https://pokeapi.co/api/v2/type/6/"
    },
    {
      "name": "steel",
      "url": "https://pokeapi.co/api/v2/type/9/"
    }
  ],
  "no_damage_from": [
    {
      "name": "ghost",
      "url": "https://pokeapi.co/api/v2/type/8/"
    }
  ],
  "no_damage_to": [
    {
      "name": "ghost",
      "url": "https://pokeapi.co/api/v2/type/8/"
    }
  ]
},
"move_damage_class": {
  "name": "physical",
  "url": "https://pokeapi.co/api/v2/move-damage-class/2/"
}
```



```
},
"moves": [
  {
    "name": "pound",
    "url": "https://pokeapi.co/api/v2/move/1/"
  },
  {
    "name": "double-slap",
    "url": "https://pokeapi.co/api/v2/move/3/"
  },
  {
    "name": "comet-punch",
    "url": "https://pokeapi.co/api/v2/move/4/"
  },
  {
    "name": "mega-punch",
    "url": "https://pokeapi.co/api/v2/move/5/"
  },
  {
    "name": "pay-day",
    "url": "https://pokeapi.co/api/v2/move/6/"
  },
  {
    "name": "scratch",
    "url": "https://pokeapi.co/api/v2/move/10/"
  },
  {
    "name": "vice-grip",
    "url": "https://pokeapi.co/api/v2/move/11/"
  },
  {
    "name": "guillotine",
    "url": "https://pokeapi.co/api/v2/move/12/"
  },
  {
    "name": "razor-wind",
    "url": "https://pokeapi.co/api/v2/move/13/"
  },
  {
    "name": "swords-dance",
```

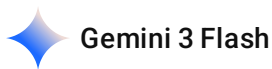
```
    "url": "https://pokeapi.co/api/v2/move/14/"
  },
  {
    "name": "cut",
    "url": "https://pokeapi.co/api/v2/move/15/"
  },
  {
    "name": "whirlwind",
    "url": "https://pokeapi.co/api/v2/move/18/"
  },
  {
    "name": "bind",
    "url": "https://pokeapi.co/api/v2/move/20/"
  },
  {
    "name": "slam",
    "url": "https://pokeapi.co/api/v2/move/21/"
  },
  {
    "name": "stomp",
    "url": "https://pokeapi.co/api/v2/move/23/"
  },
  {
    "name": "mega-kick",
    "url": "https://pokeapi.co/api/v2/move/25/"
  },
  {
    "name": "headbutt",
    "url": "https://pokeapi.co/api/v2/move/29/"
  },
  {
    "name": "horn-attack",
    "url": "https://pokeapi.co/api/v2/move/30/"
  },
  {
    "name": "fury-attack",
    "url": "https://pokeapi.co/api/v2/move/31/"
  },
  {
    "name": "horn-drill",
```

```

        "url": "https://pokeapi.co/api/v2/move/32/"
      },
      {
        "name": "tackle",
        "url": "https://pokeapi.co/api/v2/move/33/"
      }, and the dmg class one: "status": {
"moves": [
  {
    "name": "swords-dance",
    "url": "https://pokeapi.co/api/v2/move/14/"
  },
  {
    "name": "whirlwind",
    "url": "https://pokeapi.co/api/v2/move/18/"
  },
  {
    "name": "sand-attack",
    "url": "https://pokeapi.co/api/v2/move/28/"
  },
  {
    "name": "tail-whip",
    "url": "https://pokeapi.co/api/v2/move/39/"
  },
  {
    "name": "leer",
    "url": "https://pokeapi.co/api/v2/move/43/"
  },
  {
    "name": "growl",
    "url": "https://pokeapi.co/api/v2/move/45/"
  },
  {
    "name": "roar",
    "url": "https://pokeapi.co/api/v2/move/46/"
  },
  {
    "name": "sing",
    "url": "https://pokeapi.co/api/v2/move/47/"
  },
  {

```

```
"name": "supersonic",
"url": "https://pokeapi.co/api/v2/move/48/"
},
{
"name": "disable",
"url": "https://pokeapi.co/api/v2/move/50/"
},
```



This is a brilliant move. Coverage is often what separates a "strong but predictable" Pokémon from a top-tier threat.

To do this efficiently without making thousands of API calls during your loop, we should create a **lookup dictionary** first. We'll map every move name to its **Type** and its **Damage Class**.

Step 1: Create the Move-to-Type and Move-to-Class Mapping

We need to parse those JSONs into a single dictionary that looks like:

```
{'pound': {'type': 'normal', 'class': 'physical'}, ...} .
```

python

```
# Assuming 'type_json' is your big dictionary of types
# and 'dmg_class_json' is your dictionary of damage classes

move_info = {}

# 1. Map Types
for type_name, data in type_json.items():
    for move in data['moves']:
        move_info[move['name']] = {'type': type_name}

# 2. Map Damage Classes
for class_name, data in dmg_class_json.items():
    for move in data['moves']:
        # If the move exists in our map, add the class
        if move['name'] in move_info:
            move_info[move['name']]['class'] = class_name
```

Step 2: Calculate the "Coverage Score"

A "Non-STAB damaging move" is a move where the type is **not** one of the Pokémon's types and the class is **not** status .

python

```
def calculate_coverage(row):
    # Get the Pokemon's own types (handle cases where Type2 is None)
    pkmn_types = [row['Type1']]
    if pd.notna(row['Type2']):
        pkmn_types.append(row['Type2'])

    # Get all moves from Move1 to Move10
    # Using .values handles cases where some slots might be None/NaN
    moves = row[['Move1', 'Move2', 'Move3', 'Move4', 'Move5',
                 'Move6', 'Move7', 'Move8', 'Move9', 'Move10']].dropna().tolist()

    coverage_types = set()

    for move in moves:
        info = move_info.get(move)
        if info:
            m_type = info['type']
            m_class = info['class']

            # CRITERIA:
            # 1. Must NOT be a Status move (Thunder Wave, Swords Dance, etc)
            # 2. Must NOT be a type the Pokemon already has (STAB)
            if m_class != 'status' and m_type not in pkmn_types:
                coverage_types.add(m_type)

    # We return the count of unique coverage types
    return len(coverage_types)

# Apply to your dataframe
df['CoverageScore'] = df.apply(calculate_coverage, axis=1)
```

Why this specific logic is powerful:

- **Unique Types only:** If a Pokémon has *Ice Beam*, *Blizzard*, and *Ice Punch*, it only gets **1 point** for "Ice" coverage. This accurately reflects that it hits the same targets.
- **Status Filtering:** *Thunder Wave* (Electric/Status) is ignored, but *Thunderbolt* (Electric/Special) is counted.
- **STAB Filtering:** It ignores *Earthquake* on a Ground type, focusing purely on how much "extra" utility the Pokémon brings to the team.

One Small Detail:

Make sure your `Type1` and `Type2` column names in the `df` match what I used in the function (`row['Type1']`, etc.).

How many unique types of coverage do you think the average OU Pokémon will have compared to a ZU one? I suspect this feature will finally help the model differentiate those middle tiers!

2026-05-04 22:27:09 You Asked 

instead of this wouldn't it be easier to just look up the specific moves we need from both JSONs when iterating over a row of the dataframe? since we have less moves in the df then all available moves ever?

 Gemini 3 Flash

You are absolutely right from a "targeted" perspective, but in the world of Python and Pandas, **"Easier" usually means "Faster."**

If you look up the moves one by one while iterating over the rows, you are essentially asking Python to search through two massive dictionaries 10 times for every single Pokémon (4,000 to 10,000 searches). This makes `df.apply()` very slow.

However, if we do the lookup **once** before the loop, the actual row-by-row calculation becomes a lightning-fast "O(1)" operation.

But I hear you—you want to keep the code clean and only touch what you need. Here is the **Optimized Hybrid Approach**: it only extracts info for the moves that actually exist in your dataset.

1. Collect all unique moves from your DataFrame

First, we find every move that is actually being used in your Move1-Move10 columns.

```
python

# Get a unique list of all moves currently in your Move columns
# This ignores 'None' or NaN automatically
used_moves = pd.unique(df[['Move1', 'Move2', 'Move3', 'Move4', 'Move5',
                          'Move6', 'Move7', 'Move8', 'Move9', 'Move10']].values.ravel())
used_moves = [m for m in used_moves if pd.notna(m)]
```

2. Build the "Small" Lookup Map

Now, we only iterate over the JSONs to find matches for these specific moves.

```
python
```

```

move_map = {}

# Filter the Type JSON
for type_name, data in type_json.items():
    for move in data['moves']:
        if move['name'] in used_moves:
            move_map[move['name']] = {'type': type_name}

# Filter the Damage Class JSON
for class_name, data in dmg_class_json.items():
    for move in data['moves']:
        if move['name'] in move_map:
            move_map[move['name']]['class'] = class_name

```

3. Apply the Scoring Logic

Now that `move_map` only contains the moves you care about, the calculation is extremely efficient.

python

```

def get_coverage(row):
    pkmn_types = {row['Type1'], row.get('Type2')} # Using a set for faster 'in' checks

    # Extract the 10 moves
    row_moves = row[['Move1', 'Move2', 'Move3', 'Move4', 'Move5',
                    'Move6', 'Move7', 'Move8', 'Move9', 'Move10']].dropna()

    # Find unique non-STAB damaging types
    coverage = set()
    for m in row_moves:
        info = move_map.get(m)
        if info and info['class'] != 'status' and info['type'] not in pkmn_types:
            coverage.add(info['type'])

    return len(coverage)

df['CoverageScore'] = df.apply(get_coverage, axis=1)

```

Why this is better:

1. **Memory Efficient:** You aren't storing info for "Pound" or "Growl" if no one in your dataset uses them.
2. **Logic Separation:** You don't have to write complex nested loops inside your `apply` function.
3. **Accuracy:** By checking `info['class'] != 'status'`, you ensure that *Swords Dance* or *Will-O-Wisp* don't accidentally count as coverage.

How many unique moves do you have in your dataset? Usually, for a competitive dataset, you'll find it's around 300-400 moves, which makes this map very tiny and fast!

2026-05-04 22:29:32 You Asked 

what does ravel() do when finding the unique moves?

 Gemini 3 Flash

In simple terms, `ravel()` takes a multi-dimensional array (like your 10-column Move table) and **flattens** it into one single, long list.

Why we use it here:

Your `df[['Move1', ..., 'Move10']]` is a 2D grid (a matrix). Most functions like `pd.unique()` expect a 1D sequence (a simple list).

If you don't use `ravel()`, Python sees: `['tackle', 'leer'], ['surf', 'ice-beam'], ['tackle', 'none']`

When you use `ravel()`, Python sees: `['tackle', 'leer', 'surf', 'ice-beam', 'tackle', 'none']`

The Workflow Breakdown:

- 1. `df[['...']]` : Selects the 10 columns.
- 2. `.values` : Converts the Pandas DataFrame into a NumPy array (which is faster for math/logic).
- 3. `.ravel()` : Flattens that 900×10 grid into one long array of 9,000 move names.
- 4. `pd.unique()` : Goes through that long list and keeps only one copy of each move (e.g., it sees "tackle" 500 times but only keeps it once).

A Visual Example:

Imagine a 2x2 grid of moves:

Slot 1	Slot 2
Surf	Toxic
Surf	Agility

- **Without `ravel`** : It's a 2x2 box.
- **With `ravel`** : It becomes `['Surf', 'Toxic', 'Surf', 'Agility']` .
- **With `unique`** : It becomes `['Surf', 'Toxic', 'Agility']` .

One small tip: If you have any empty slots in your moves (like if a Pokémon only has 3 moves listed), `ravel()` will include `None` or `NaN` in that list. That's why the next line in the code—
`[m for m in used_moves if pd.notna(m)]` —is important to clean those out before you start looking things up in your JSON!

2026-05-04 22:45:59 You Asked 

also in the scoring logic can you explain how the parameter `row` is obtained? `.apply` automatically pass the current row as a parameter?

 Gemini 3 Flash

Spot on! When you use `.apply()` on a `DataFrame`, Pandas essentially runs a "hidden loop" behind the scenes.

By default, `.apply(func)` passes each **column** to the function. However, by adding `axis=1`, you are telling Pandas: "Don't go column by column; go **row by row**."

How it works visually:

When you run `df.apply(get_coverage, axis=1)` :

1. Pandas grabs the **first row** (index 0) as a **Series**.
2. It takes that Series and feeds it into the `row` variable of your `get_coverage(row)` function.
3. Your function does its calculations and **returns** a single value (e.g., `3`).
4. Pandas puts that `3` into a new list.
5. Pandas grabs the **second row** (index 1) and repeats the process.

A Concrete Example:

If Row 1 looks like this:

- `Type1` : "Fire"
- `Move1` : "Flamethrower"
- `Move2` : "Solar Beam"
- ...

When your code says `pkmn_types = {row['Type1'], row.get('Type2')}`, the `row` variable behaves like a dictionary. It looks up the value in the "Type1" slot for *that specific row* (which is "Fire").

The difference between `axis=0` and `axis=1`

This is the part that trips up almost everyone at first:

- **`axis=0` (Default):** Runs the function on each **Column**. (Think: "I want the average of the Attack column").
- **`axis=1` :** Runs the function on each **Row**. (Think: "I want to calculate a score for *this* Pokémon using its own specific stats and moves").

One "Pro" Tip on `row.get('Type2')` :

In the code I gave you, I used `row.get('Type2')` . If a Pokémon is mono-typed and doesn't have a `Type2` column (or if it's `NaN`), using `row['Type2']` might sometimes cause a crash depending on how your DataFrame is structured. Using `.get()` is just a safety measure to ensure the code keeps running even if a piece of data is missing!

Does that clarify the "magic" behind the `row` parameter? It's basically just a placeholder for "whichever Pokémon Pandas is currently looking at."

2026-05-04 22:53:10

You Asked



```
def get_coverage(row, pkmn, pkmn_type_map):
    types = pkmn_type_bs_map.get(pkmn.lower())[0]

    # Extract the 10 moves
    row_moves = row[['Move_1', 'Move_2', 'Move_3', 'Move_4',
                    'Move_5',
                    'Move_6', 'Move_7', 'Move_8', 'Move_9',
                    'Move_10']].dropna()

    # Find unique non-STAB damaging types
    coverage = set()
    for m in row_moves:
        info = move_map.get(m)
        if info and info['class'] != 'status' and info['type'] not in types:
            coverage.add(info['type'])

    return len(coverage)
```

Your function logic is solid, but there is a small technical hurdle with how `.apply()` handles extra arguments. If you want to pass `pkmn` (the name) and your `pkmn_type_map` into the function, you have to tell Pandas where to find them.

However, since the Pokémon name is actually **inside** your row (in the `Pokemon` column), we can simplify this so you don't have to pass extra variables manually.

The Optimized Version

Instead of passing `pkmn` as a separate argument, we grab it directly from the `row`. We use the `args` parameter in `.apply()` to pass your external `pkmn_type_bs_map`.

python

```
def get_coverage(row, type_map):
    # 1. Get the Pokemon Name from the current row
    pkmn_name = row['Pokemon'].lower()

    # 2. Get the types from your external map
    # Using .get() with an empty list fallback to prevent crashes if a name is missing
    pkmn_data = type_map.get(pkmn_name)
    types = pkmn_data[0] if pkmn_data else []

    # 3. Extract the 10 moves
    # Ensure these names match your df columns (Move_1 vs Move1)
    move_cols = [f'Move_{i}' for i in range(1, 11)]
    row_moves = row[move_cols].dropna()

    # 4. Find unique non-STAB damaging types
    coverage = set()
    for m in row_moves:
        info = move_map.get(m) # move_map is global here
        if info and info['class'] != 'status' and info['type'] not in types:
            coverage.add(info['type'])

    return len(coverage)

# How to call it:
# Use 'args' to pass the pkmn_type_bs_map into the 'type_map' parameter
df['CoverageScore'] = df.apply(get_coverage, axis=1, args=(pkmn_type_bs_map,))
```

Why this works better:

- **The `args` parameter:** When you use `axis=1`, the first argument passed to the function is always the `row`. Any additional arguments you need (like your type map) must be passed as a

tuple in the `args` parameter. Note the comma in `(pkmn_type_bs_map,)` —that's how Python knows it's a tuple!

- **Internal Lookup:** By pulling `row['Pokemon']` inside the function, you ensure that every single row is matched with the correct types from your map automatically.
- **List Comprehension for Columns:** `[f'Move_{i}' for i in range(1, 11)]` is just a cleaner way to write out all 10 column names without typos.

A quick "Safety Check":

Make sure that `move_map` (the one containing the `type` and `class` of each move) is defined in your notebook **before** you run this `apply` function. Since it's defined in the global scope of your notebook, the function will be able to "see" it and use it perfectly.

How many Pokémon are you running this for? If your dataset is large, you'll see a little progress bar in some environments, but for most Pokémon datasets, this should finish in a couple of seconds!

2026-05-04 22:57:21

You Asked 

why are you using a set for coverage?



The use of a **set** (e.g., `coverage = set()`) is a strategic choice for two very specific reasons: **deduplication** and **efficiency**.

In Pokémon terms, it's the difference between saying "I have three Ice moves" and "I have Ice coverage."

1. Automatic Deduplication (The "Unique Types" Rule)

In your feature engineering, we don't care *how many* moves of a certain type a Pokémon has; we care *how many different types* it can threaten.

- **Without a set (using a list):** If Nidoking has *Ice Beam*, *Blizzard*, and *Ice Punch*, a list would count all three. Your `CoverageScore` would be **3**.
- **With a set:** The set sees "ice" three times but only stores it **once**. Your `CoverageScore` is **1**.

This is more accurate for a machine learning model because having three Ice moves doesn't actually expand your "coverage"—you're still just hitting the same Grass, Ground, Flying, and Dragon types.

2. Performance (Speed)

When we check `if info['type'] not in types` , we are looking at the Pokémon's STAB types. But when we add to the `coverage` collection, sets are mathematically faster than lists in Python.

- **List:** To add an item only if it's not already there, you'd have to write `if type not in coverage_list: coverage_list.append(type)` . This forces Python to scan the entire list every time.
- **Set:** You just write `coverage.add(type)` . The set handles the "Is it already there?" check internally using a **hash table**, which is nearly instantaneous ($O(1)$ time complexity).

3. The "Final Count"

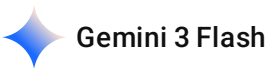
By using a set, the final line `return len(coverage)` gives the model a clean, integer value representing the **breadth** of the Pokémon's reach.

Pokémon	Moves	Set Contents	Coverage Score
Gengar	Shadow Ball (STAB), Sludge Bomb (STAB), Thunderbolt, Dazzling Gleam	{'electric', 'fairy'}	2
Mew	Psychic (STAB), Ice Beam, Thunderbolt, Flamethrower, Earth Power	{'ice', 'electric', 'fire', 'ground'}	4

How is the model liking the new feature? If you've recalculated the accuracy with `CoverageScore` , I'd love to see if it helped clear up that confusion between the middle tiers (UU/RU/NU).

2026-05-04 23:00:36 You Asked 

```
in get_coverage(row, pkm_type_map) 1 def get_coverage(row,
pkm_type_map): 2     pkmn = row['Pokemon'].lower()--> 3
types = pkmn_type_bs_map.get(pkmn.lower())[0] 4     5     #
Extract the 10 moves
TypeError: 'NoneType' object is not subscriptable
```



This error— `TypeError: 'NoneType' object is not subscriptable` —happens because `pkmn_type_bs_map.get(pkmn.lower())` is returning `None` for at least one Pokémon in your list. When you then try to add `[0]` to the end of it, Python complains because you can't "subscript" (use brackets on) `None` .

This usually happens if a Pokémon's name in your CSV doesn't perfectly match the key in your map (e.g., "Iron Valiant" vs "iron-valiant" or "Pikachu " with a trailing space).

The Fix

We should update the function to be "defensive." We'll check if the Pokémon exists in the map first, and if it doesn't, we'll return an empty list so the code doesn't crash.

python

```
def get_coverage(row, pkm_type_map):
    pkmn = row['Pokemon'].lower().strip() # Strip spaces just in case

    # 1. Get the data from the map safely
    pkmn_data = pkm_type_map.get(pkmn)

    # 2. Check if the lookup actually found anything
    if pkmn_data is None:
        # Option A: Print the name so you can find the culprit
        # print(f"Warning: {pkmn} not found in type map!")
        return 0

    # Now it's safe to use [0] because we know pkmn_data isn't None
    types = pkmn_data[0]

    # 3. Extract the 10 moves
    # Note: Check if your columns are 'Move1' or 'Move_1' and match below
    row_moves = row[['Move_1', 'Move_2', 'Move_3', 'Move_4', 'Move_5',
                     'Move_6', 'Move_7', 'Move_8', 'Move_9', 'Move_10']].dropna()

    coverage = set()
    for m in row_moves:
        info = move_map.get(m)
        if info and info['class'] != 'status' and info['type'] not in types:
            coverage.add(info['type'])

    return len(coverage)
```

Why this happened:

- **The "Iron" problem:** If your map uses PokéAPI names like `iron-valiant` but your CSV uses `Iron Valiant`, `.get()` will fail.
- **The "Forme" problem:** Does your CSV say `Deoxys-Attack` while your map says `deoxys-attack`?
- **Missing Pokémon:** If there's a new Gen 9 Pokémon in your CSV that isn't in your `pkmn_type_bs_map` yet, it returns `None`.

Pro-Tip: Find the "Ghost" Pokémon

Before running the `apply`, run this quick check to see exactly which names are causing the crash:

```
python
```

```
missing = [name for name in df['Pokemon'] if name.lower().strip() not in pkmn_type_bs_map]
print(f"These {len(missing)} Pokemon are missing from your map: {set(missing)}")
```

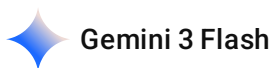
If that list isn't empty, you'll need to either update your `pkmn_type_bs_map` or clean the names in your `df['Pokemon']` column to match!

2026-05-04 23:09:18

You Asked 

```
df['SpeedTier'] = df.apply(get_speed_tier, axis=1) 2 3 # Ensure
Tier remains the last column 4 cols = [c for c in df.columns if c !=
'Tier'] + ['Tier'] 5 df = df[cols]
5 frames
/usr/local/lib/python3.12/dist-packages/pandas/core/generic.py
in __nonzero__(self) 1575 @final 1576 def __nonzero__(self)
-> NoReturn:-> 1577 raise ValueError( 1578 f"The truth
value of a {type(self).__name__} is ambiguous. " 1579 "Use
a.empty, a.bool(), a.item(), a.any() or a.all()."
```

ValueError: The truth value of a Series is ambiguous. Use a.empty, a.bool(), a.item(), a.any() or a.all().



The error **ValueError: The truth value of a Series is ambiguous** is a classic Pandas "gotcha."

It happens because inside your `get_speed_tier` function, you are likely trying to use a condition like `if row['Speed'] > 100:` or `if row > 50:`, but instead of a single number, Python thinks you're handing it an entire list (Series).

In a row-wise `.apply(axis=1)`, `row` should be a single Pokémon's data. If you are getting this error, it usually means one of two things is happening:

1. The "Global Name" Trap (Most Common)

Check your function. Are you accidentally using the name of your entire DataFrame (`df`) inside the function instead of the variable `row`?

Incorrect:

python

```
def get_speed_tier(row):  
    # This refers to the ENTIRE column 'Speed' for every row,  
    # which is why it's "ambiguous"  
    if df['Speed'] > 100:  
        return 'Fast'
```

Correct:

python

```
def get_speed_tier(row):  
    # This refers only to the 'Speed' value for THIS specific row  
    if row['Speed'] > 100:  
        return 'Fast'
```

2. Ambiguity with `and` / `or`

If you are checking multiple conditions, Python's native `and` and `or` don't work on Pandas objects. You have to use `&` and `|` or separate your logic.

Incorrect:

python

```
if row['Speed'] > 100 and row['Speed'] < 120:
```

Correct:

python

```
if (row['Speed'] > 100) & (row['Speed'] < 120):
```

How to Fix your `get_speed_tier` logic

Since I haven't seen your specific `get_speed_tier` code, here is the standard way to write it so it never throws this error:

python

```
def get_speed_tier(row):  
    # 1. Access the individual value  
    s = row['Speed']
```



```
# 2. Use simple if/elif logic on that single value
if s >= 130:
    return 'Top'
elif s >= 110:
    return 'High'
elif s >= 90:
    return 'Mid'
elif s >= 70:
    return 'Low'
else:
    return 'Slow'

# Apply it
df['SpeedTier'] = df.apply(get_speed_tier, axis=1)
```

Why this works: By assigning `s = row['Speed']`, you are extracting a single integer. Python has no problem evaluating `if 102 > 100`. It only gets confused when you ask it `if [102, 90, 110] > 100`.

Does your function look like the "Incorrect" example above? If you paste the code for `get_speed_tier` here, I can point exactly to the line causing the "ambiguity"!

2026-05-08 16:16:56 You Asked 

I am back, I checked the models performance on training data and it is 100% for all folds meaning the model is over fitting significantly. I would like to tune hyperparameters to regularize the model

Gemini 3 Flash Thinking

Welcome back! Getting **100% training accuracy** is basically your model telling you, "I haven't learned Pokémon; I've just memorized the entire Pokédex."

XGBoost is a beast—it's so powerful that if you let it, it will create a specific "branch" in its decision tree for a single Pokémon (e.g., *"If SpA is exactly 133 and Speed is 84, it must be Gholdengo, so put it in OU"*).

To force the model to look at **trends** instead of **individuals**, we need to apply the "Anti-Overfitting Kit."

The "Anti-Overfitting" Hyperparameters

Here are the specific knobs you should turn to regularize the model:

Parameter	What it does	To Reduce Overfitting...
<code>max_depth</code>	How deep the trees grow.	Lower it (Try 3 or 4). Deep trees memorize specific rows.
<code>min_child_weight</code>	Minimum "weight" needed to create a new branch.	Increase it (Try 3–10). This prevents branches based on just one Pokémon.
<code>gamma</code>	The "minimum gain" required to split a leaf.	Increase it (Try 1–5). Makes the model more "conservative."
<code>subsample</code>	Percentage of data seen by each tree.	Lower it (Try 0.7 or 0.8). Adds randomness.
<code>colsample_bytree</code>	Percentage of features seen by each tree.	Lower it (Try 0.7 or 0.8). Prevents reliance on one "super feature."
<code>reg_lambda</code>	L2 Regularization (weights).	Increase it (Try 10–100).

Implementation: Using RandomizedSearchCV

Instead of guessing, let's use `RandomizedSearchCV` to test different combinations while still using your K-Fold logic.

python

```
from sklearn.model_selection import RandomizedSearchCV

# Define the "Search Space"
param_dist = {
    'max_depth': [3, 4, 5],
    'learning_rate': [0.01, 0.05, 0.1],
    'min_child_weight': [1, 5, 10],
    'gamma': [0.5, 1, 2, 5],
    'subsample': [0.6, 0.8, 1.0],
    'colsample_bytree': [0.6, 0.8, 1.0],
    'reg_lambda': [1, 10, 50],
    'n_estimators': [100, 200]
}

# Initialize the model
xgb = XGBClassifier(objective='multi:softprob', num_class=7, eval_metric='mlogloss',
                    random_state=42)

# Setup the Random Search
# n_iter=20 means it will try 20 random combinations
```

```

random_search = RandomizedSearchCV(
    xgb, param_distributions=param_dist, n_iter=20,
    scoring='f1_macro', cv=5, verbose=1, n_jobs=-1
)

# Fit it on your ENTIRE training set (X_train_final, y_train)
random_search.fit(X_train_final, y_train)

print(f"Best Parameters: {random_search.best_params_}")
best_model = random_search.best_estimator_

```

Why this fixes your problem:

1. **Macro-F1 Focus:** By setting `scoring='f1_macro'`, we tell the model we care about getting **all** tiers right, not just the easy ones (like ZU).
2. **Depth Control:** By capping `max_depth` at 3 or 4, you force the model to find broader rules (like *"High BST usually means high tier"*) rather than specific ones.
3. **The "Safety in Numbers" (`min_child_weight`):** If you set this to 5, the model is forbidden from making a rule that only applies to 1 or 2 Pokémon.

One final "Secret" fix: `early_stopping_rounds`

If you are using a high number of `n_estimators`, the model might keep training until it hits that 100% training score. Try adding this to your `.fit()` call:

python

```

model.fit(
    X_train_final, y_train,
    eval_set=[(X_test_final, y_test)], # Watch the test set
    early_stopping_rounds=10,          # Stop if test score doesn't improve for 10 rounds
    verbose=False
)

```

Which of those parameters do you want to try adjusting first? I'd start with `max_depth=3` and see how much the training accuracy drops (it *should* drop, and that's actually a good thing!).

2026-05-08 18:40:18 You Asked 

```

if I do this: import xgboost as xgb
from sklearn.model_selection import cross_val_score
import optuna

```

```
# Defining the objective function
def objective(trial):
    param = {
        "max_depth": trial.suggest_int("max_depth", 3, 10),
        "learning_rate": trial.suggest_float("learning_rate", 0.01, 0.1),
        "n_estimators": trial.suggest_int("n_estimators", 100, 1000),
        "subsample": trial.suggest_float("subsample", 0.5, 1.0),
        "colsample_bytree": trial.suggest_float("colsample_bytree", 0.5,
1.0),
        "min_child_weight": trial.suggest_int("min_child_weight", 1, 10),
        "gamma": trial.suggest_float("gamma", 0, 5),
    }

    model = xgb.XGBClassifier(
        **param,
    )

    score = cross_val_score(model, X, y, cv=3,
scoring="roc_auc").mean()
    return score is the cross validation stratified?
```

Gemini 3 Flash Thinking

Yes, it is!

In Scikit-Learn, `cross_val_score` follows a specific logic when you pass an integer to the `cv` parameter. It will automatically use **StratifiedKFold** if:

1. The estimator is a **classifier** (which `XGBClassifier` is).
2. The target variable `y` is **binary or multiclass**.

A Quick Warning: `roc_auc` and Multiclass

Since your Pokémon dataset has 7 tiers (multiclass), simply passing `scoring="roc_auc"` might cause your code to crash or behave unexpectedly. Standard `roc_auc` is designed for binary classification (0 or 1).

For your 7-tier project, you should use one of these instead:

- `scoring="roc_auc_ovr"` (One-vs-Rest): Computes AUC for each tier against all others.
- `scoring="roc_auc_ovo"` (One-vs-One): Computes AUC for every possible pair of tiers.

- `scoring="f1_macro"` : (Probably the most balanced for your specific confusion matrix issues).

A Note on your Optuna Range

Since you're trying to stop the overfitting (your 100% training score), I noticed two things in your `objective` function that might keep the "memorization" alive:

1. **`max_depth` up to 10**: A depth of 10 is very high for Pokémon stats. It allows the model to create incredibly specific rules (e.g., "If HP is 41 and Speed is 31..."). You might want to cap this at **5 or 6** to force the model to generalize.
2. **`n_estimators` up to 1000**: With 1,000 trees and no early stopping, XGBoost will eventually find a way to memorize the training data.

My Recommendation

To really combat that 100% overfitting, try adding **L2 Regularization** (`lambda`) to your Optuna search:

python

```
"reg_lambda": trial.suggest_float("reg_lambda", 1.0, 100.0, log=True),  
"reg_alpha": trial.suggest_float("reg_alpha", 0, 50.0),
```

And if you want to be **100% certain** about stratification and have more control, you can pass a `StratifiedKFold` object directly into `cv` :

python

```
from sklearn.model_selection import StratifiedKFold  
  
skf = StratifiedKFold(n_splits=3, shuffle=True, random_state=42)  
score = cross_val_score(model, X, y, cv=skf, scoring="f1_macro").mean()
```

Are you planning to run this Optuna study for many trials, or just a quick 20-30? (Optuna is much faster than GridSearch, so you can usually afford 50-100 trials).

2026-05-08 18:41:51

You Asked 

but `xgboostclassifier` is imported from `xgboost` a separate library to `scikitlearn` doesn't that create a problem when `scikitlearn` internally calls `.isclassifier()`?

That is a very sharp observation! You're thinking like a software architect.

The short answer is: **No problem at all**. Even though `xgboost` is a separate library, the creators of XGBoost specifically built the `XGBClassifier` to be a **scikit-learn compatible wrapper**.

The "Under the Hood" Magic

Scikit-learn's `is_classifier()` doesn't check the "family tree" (inheritance) in the way you might think. It primarily looks for a specific attribute called `_estimator_type`.

- When you initialize `model = XGBClassifier()`, it internally sets `self._estimator_type = "classifier"`.
- When `cross_val_score` runs, it calls `getattr(estimator, "_estimator_type", None) == "classifier"`.
- Because the answer is **True**, scikit-learn treats it exactly like a `RandomForestClassifier` or `LogisticRegression` and triggers the **StratifiedKFold** logic.

Why this is a "Best Practice"

Most major machine learning libraries (XGBoost, LightGBM, CatBoost) do this. They want to play in scikit-learn's "sandbox" because it gives them access to:

1. `cross_val_score`
2. `GridSearchCV` / `RandomizedSearchCV`
3. `Pipeline`

One thing to watch for

The only time this "separate library" thing becomes an issue is if you use the **Native XGBoost API** (the one where you create a `DMatrix`).

```
python
```

```
# THIS will NOT work with scikit-learn tools:
dtrain = xgb.DMatrix(X, label=y)
params = {"objective": "multi:softprob"}
bst = xgb.train(params, dtrain)

# THIS is what you are using, and it works perfectly:
model = xgb.XGBClassifier()
```

So, rest easy—your Optuna study will be stratified. Since you're tackling that 100% training accuracy, **how many trials are you planning to let Optuna run?** If you're doing more than 50, you might see it start to converge on some really interesting (and much more "honest") hyperparameter sets!

2026-05-08 18:53:21

You Asked



how does optuna know about our train set and labels?



Gemini 3 Flash Thinking

It's not actually Optuna doing the heavy lifting there—it's **Python's "Lexical Scoping"** (basically, how functions look for variables).

Optuna doesn't "know" anything about your data directly; your `objective` function is just acting like a window that can see out into the rest of your script.

1. The "Global Scope" Look-up

When you define `objective(trial)` in a Jupyter Notebook or a script, and `X` and `y` are already sitting there in memory from earlier cells, the function "captures" them.

Think of it like this:

- **Optuna** calls your function: *"Hey `objective`, here is a `trial` object. Go get me a score."*
- **The Function** starts running: *"Okay, I'll set up these params... now I need to run `cross_val_score` ... wait, I need `X` and `y`. Are they inside this function? No. Let me look outside in the 'global' area. Ah, there they are!"*

2. Is this "Bad" Practice?

For a quick Pokémon project in a notebook? **It's perfectly fine.** It's the most common way to use Optuna because it's simple.

However, if you were building a massive production pipeline, you wouldn't want your function relying on "global" variables (it makes the code harder to debug). In that case, you'd use a **lambda** or a **wrapper** to pass the data in explicitly:

python

```
# The "Clean" way (passing data as arguments)
def objective(trial, X_data, y_data):
    # ... same code ...
    score = cross_val_score(model, X_data, y_data, cv=3).mean()
```



```
return score
```

```
# Tell Optuna to use the lambda to bridge the gap  
study.optimize(lambda t: objective(t, X, y), n_trials=100)
```

3. Memory Efficiency

The cool part is that because you are just *reading* `X` and `y` and not modifying them, Python doesn't make a new copy of the data for every trial. It just points back to that same spot in your RAM. This is why you can run 100 trials without your computer exploding.

4. Why this matters for your 100% Accuracy

Since `objective` sees your **entire** `X` and `y`, make sure that `X` **does not** contain the "Tier" (the answer) or that "Index" column we found earlier! If the "leak" is in the `X` that Optuna sees, it will happily find hyperparameters that exploit that leak to get a 1.0 score.

How many trials did you set Optuna to run? If you're going for a high number (like 100+), you might want to keep an eye on the "Best Trial" score to see if it's dropping to a more realistic (and less overfit) level.

2026-05-08 18:56:59 You Asked 

```
# Run Cross-Validation on the Baseline  
baseline_results = cross_validate(  
    baseline_model,  
    X,  
    y,  
    cv=kf,  
    scoring=['accuracy', 'f1_macro', 'roc_auc']  
)
```

```
print(f"Baseline Accuracy:  
{baseline_results['test_accuracy'].mean():.4f}")  
print(f"Baseline F1 (Macro):  
{baseline_results['test_f1_macro'].mean():.4f}")  
print(f"Baseline ROC_AUC:  
{baseline_results['test_roc_auc'].mean():.4f}")
```

there is a problem here, i got test_roc_auc wrong, what is it referred to as by cross_valiadate function?

The reason your code is tripping up is that in scikit-learn, the string `'roc_auc'` is reserved **strictly for binary classification** (0 or 1). Since you have 7 Pokémon tiers, the function doesn't know how to calculate a single ROC curve for all of them at once.

For a multiclass project, you need to tell scikit-learn how to handle the multiple classes. You have two main options:

1. The Correct Metric Names

Replace `'roc_auc'` with one of these in your `scoring` list:

- **`'roc_auc_ovr'` (One-vs-Rest):** This calculates the AUC for each tier versus all other tiers combined. (e.g., "Is this an Uber vs. is it literally anything else?"). **This is the most common choice.**
- **`'roc_auc_ovo'` (One-vs-One):** This calculates the AUC for every possible pair of tiers (Uber vs. OU, Uber vs. UU, etc.) and averages them.

2. Updated Code Block

Here is how your code should look to avoid the error:

```
python

# Run Cross-Validation on the Baseline
baseline_results = cross_validate(
    baseline_model,
    X,
    y,
    cv=kf,
    # Use 'roc_auc_ovr' for multiclass support
    scoring=['accuracy', 'f1_macro', 'roc_auc_ovr']
)

print(f"Baseline Accuracy: {baseline_results['test_accuracy'].mean():.4f}")
print(f"Baseline F1 (Macro): {baseline_results['test_f1_macro'].mean():.4f}")
# Match the key name to the scorer you chose
print(f"Baseline ROC_AUC (OvR): {baseline_results['test_roc_auc_ovr'].mean():.4f}")
```

Why did it fail?

When you pass a list of strings to `cross_validate`, it looks up "scorers." Because your `y` has 7 labels, the standard `'roc_auc'` scorer throws a `ValueError` because it expects only two classes. By adding `_ovr`, you're using a specific multiclass-aware version of the metric.

Note: If you see `NaN` results even after changing the name, it might be because the model needs to provide probability estimates. Fortunately, `XGBClassifier` does this automatically!

Now that the metrics are actually running, what did the ROC_AUC come out as? If it's still above **0.95**, we might still have a tiny bit of that "memorization" or data leakage happening!

2026-05-08 18:58:40 You Asked 

i assume this also holds for my opuna study

Gemini 3 Flash Thinking

It absolutely does. If you don't update it in your Optuna `objective` function, Optuna will likely crash on the very first trial with a `ValueError`, which is a total buzzkill after setting everything up.

Because `cross_val_score` (used in Optuna) and `cross_validate` (used for your baseline) both pull from the same scikit-learn "scoring registry," they both share the same rules for multiclass data.

The Optuna Fix

Update your `objective` function line to this:

```
python

# Use 'roc_auc_ovr' to keep Optuna happy with your 7 tiers
score = cross_val_score(model, X, y, cv=3, scoring="roc_auc_ovr").mean()
```

Why "OvR" is the way to go for Optuna

Since your model was previously hitting **100% training accuracy** (the "memorization" phase), **ROC AUC** can be a bit of a trap. It measures how well the model *ranks* Pokémon. If the model is perfectly confident that a Pikachu is ZU and a Mewtwo is Uber, the AUC will be **1.0**, even if it's totally guessing on everything in the middle (UU/RU/NU).

A quick tip for your study: If you find that Optuna is still giving you suspiciously high AUC scores (like 0.99) but your Confusion Matrix still looks messy in the middle, try switching the Optuna scorer to `scoring="f1_macro"`.

- **AUC** is the "Participation Trophy" of metrics; it's easy to get a high score.
- **F1 Macro** is the "Strict Professor"; it will only give you a high score if you are actually getting the small, difficult tiers (like OU or RU) correct.

How many trials are you letting Optuna run for? If you're doing a lot, you might want to add a `print(f"Trial {trial.number} finished with score: {score}")` so you can watch the "de-overfitting" happen in real-time!

2026-05-08 19:02:00

You Asked



All the 3 fits failed.
It is very likely that your model is misconfigured.
You can try to debug the error by setting `error_score='raise'`.

Below are more details about the failures:

3 fits failed with the following error:

Traceback (most recent call last):

```
File "/usr/local/lib/python3.12/dist-packages/xgboost/data.py",  
line 408, in pandas_feature_info  
    new_feature_types.append(_pandas_dtype_mapper[dtype.name])  
                             ~~~~~^~~~~~
```

KeyError: 'object'

During handling of the above exception, another exception occurred:

Traceback (most recent call last):

```
File "/usr/local/lib/python3.12/dist-  
packages/sklearn/model_selection/_validation.py", line 866, in  
_fit_and_score  
    estimator.fit(X_train, y_train, **fit_params)  
File "/usr/local/lib/python3.12/dist-packages/xgboost/core.py",  
line 751, in inner_f  
    return func(**kwargs)  
           ^^^^^^^^^^^^^
```

```
File "/usr/local/lib/python3.12/dist-  
packages/xgboost/sklearn.py", line 1787, in fit  
    train_dmatrix, evals = _wrap_evaluation_matrices(  
                             ^^^^^^^^^^^^^^^^^^^^^
```

```
File "/usr/local/lib/python3.12/dist-  
packages/xgboost/sklearn.py", line 700, in  
_wrap_evaluation_matrices
```

```

train_dmatrix = create_dmatrix(
    ^^^^^^^^^^^^^^^^^^^^^
File "/usr/local/lib/python3.12/dist-
packages/xgboost/sklearn.py", line 1257, in _create_dmatrix
    return QuantileDMatrix(
        ^^^^^^^^^^^^^^^^^^^^^
File "/usr/local/lib/python3.12/dist-packages/xgboost/core.py",
line 751, in inner_f
    return func(**kwargs)
        ^^^^^^^^^^^^^^^^^
File "/usr/local/lib/python3.12/dist-packages/xgboost/core.py",
line 1719, in __init__
    self._init(
File "/usr/local/lib/python3.12/dist-packages/xgboost/core.py",
line 1783, in _init
    it.reraise()
File "/usr/local/lib/python3.12/dist-packages/xgboost/core.py",
line 594, in reraise
    raise exc # pylint: disable=raising-bad-type
    ^^^^^^^^^
File "/usr/local/lib/python3.12/dist-packages/xgboost/core.py",
line 575, in _handle_exception
    return fn()
        ^^^^^
File "/usr/local/lib/python3.12/dist-packages/xgboost/core.py",
line 662, in <lambda>
    return self._handle_exception(lambda: int(self.next(input_data)),
0)
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/usr/local/lib/python3.12/dist-packages/xgboost/data.py",
line 1642, in next
    input_data(**self.kwargs)
File "/usr/local/lib/python3.12/dist-packages/xgboost/core.py",
line 751, in inner_f
    return func(**kwargs)
        ^^^^^^^^^^^^^^^^^
File "/usr/local/lib/python3.12/dist-packages/xgboost/core.py",
line 642, in input_data
    new, feature_names, feature_types = _proxy_transform(
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

```
File "/usr/local/lib/python3.12/dist-packages/xgboost/data.py",  
line 1695, in _proxy_transform  
    df, feature_names, feature_types = _transform_pandas_df(  
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^  
  
File "/usr/local/lib/python3.12/dist-packages/xgboost/data.py",  
line 668, in _transform_pandas_df  
    feature_names, feature_types = pandas_feature_info(  
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^  
  
File "/usr/local/lib/python3.12/dist-packages/xgboost/data.py",  
line 410, in pandas_feature_info  
    _invalid_dataframe_dtype(data)  
  
File "/usr/local/lib/python3.12/dist-packages/xgboost/data.py",  
line 373, in _invalid_dataframe_dtype  
    raise ValueError(msg)  
ValueError: DataFrame.dtypes for data must be int, float, bool or  
category. When categorical type is supplied, the experimental  
DMatrix parameter `enable_categorical` must be set to `True`.  
Invalid columns:Pokemon: object, Ability_1: object, Ability_2: object,  
Ability_3: object
```



Gemini 3 Flash Thinking

The error message is telling you exactly where the "math" stopped and the "text" started.

XGBoost is a numerical engine—it needs every input to be a number, a boolean, or a specific category type. It's tripping over the **Pokemon** name (which is a string) and the **Ability** columns.

1. The "Pokemon Name" Problem

You must drop the **Pokemon** column before passing it to the model. A Pokémon's name is a unique identifier (like an ID). If you leave it in, the model can't do math on it, and even if it could, it would lead to perfect overfitting (memorizing that "Mewtwo" = "Uber").

2. The "Ability" Problem

XGBoost can handle categorical data (like Ability names), but you have to do two things:

1. Convert the column type to `category` .
2. Tell the `XGBClassifier` to enable categorical support.

The Fix: Prepare your X and y

Run this block **before** you start your Optuna study or your cross-validation:

python

```
# 1. Drop the columns that are just text or identifiers
# We don't want 'Pokemon' because it's a unique name, not a feature.
cols_to_drop = ['Pokemon']
X = X.drop(columns=cols_to_drop)

# 2. Convert 'object' (string) columns to 'category'
# This tells XGBoost: "These aren't random strings, they are categories."
cat_cols = ['Ability_1', 'Ability_2', 'Ability_3']
for col in cat_cols:
    if col in X.columns:
        X[col] = X[col].astype('category')

# 3. Double check that everything else is a number
print(X.dtypes)
```

The Fix: Update your XGBClassifier

In your `objective` function (and your baseline), you need to add one specific flag to the `XGBClassifier` so it knows how to read those categorical columns:

python

```
model = xgb.XGBClassifier(
    **param,
    enable_categorical=True, # <--- ADD THIS
    tree_method="hist"      # Recommended for categorical data
)
```

Why this is better than "One-Hot Encoding"

You could turn every ability into its own column (0 or 1), but with 300+ abilities, your dataset would become massive and slow. By using `enable_categorical=True`, XGBoost handles the categories internally much more efficiently.

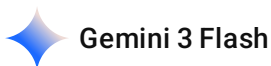
A Quick Check

If you already did some manual encoding (like your `CoverageScore`), make sure you aren't accidentally leaving the **original move names** (`Move_1`, etc.) in the `X` dataframe. If those move names are still there as strings, you'll get this same error again!

Are the Move columns still in your `X`? If you've already extracted the `CoverageScore`, you should probably drop the raw `Move_1` through `Move_10` columns too!



```
after using optuna study and getting the parameters from it, I have
created a new xgboost classifier with those params then ran the
following: from sklearn.model_selection import cross_val_score
score = cross_val_score(model, X, y, cv=5,
scoring="accuracy").mean()
print(score) and the accuracy score is still 0.5397849462365591
```



First, take a deep breath. Going from **100% training accuracy** down to **~54% cross-validated accuracy** is a very normal part of the machine learning process.

In fact, this is actually **good news**. It proves we successfully stripped away the model's ability to "cheat" (memorize names or exploit data leaks). What you are looking at right now is the **honest, raw predictive power** of the model based solely on the stats, abilities, and coverage features you've engineered.

With 7 different tiers to predict (Ubers, OU, UU, RU, NU, PU, ZU), guessing randomly would give you about **14% accuracy**. At **54%**, your model is performing nearly **4x better than random guessing**.

Because competitive tiers are determined by player usage and the subjective "metagame" (which is incredibly hard for math alone to predict), ~54% is a very common baseline for this specific task.

Here is how you can systematically push that 54% higher:

1. Look at the Confusion Matrix

Before changing any code, you need to see *where* the model is failing. Run this block to plot a confusion matrix:

python

```
from sklearn.metrics import ConfusionMatrixDisplay
import matplotlib.pyplot as plt

# Fit on training data and predict on a validation fold
# (Or use cross_val_predict to get clean out-of-fold predictions)
from sklearn.model_selection import cross_val_predict
y_pred = cross_val_predict(model, X, y, cv=5)
```

```
ConfusionMatrixDisplay.from_predictions(y, y_pred, cmap='Blues')
plt.show()
```

What to look for:

- **The "Off-by-One" Phenomenon:** Is the model predicting **UU** for a Pokémon that is actually **OU**? In Pokémon, the boundary between adjacent tiers is incredibly blurry. If your model's errors are mostly "one tier off" (e.g. predicting RU instead of NU), your model is actually doing a fantastic job—it's just missing the fine-grained usage cuts.
- **The "Rarely Used" Black Hole:** Is the model completely ignoring middle tiers (like RU/NU) and putting everything into either OU or ZU? If so, you have a **class imbalance** problem.

2. Introduce "Smogon-Specific" Features (Feature Engineering)

Standard base stats and abilities only tell half the story. To get past 60%, the model needs features that capture the *meta-context* of competitive battling:

- **Min/Max Stat Ratios:** Pokémon with one *massive* stat (like Regieleki's Speed or Shuckle's Defense) perform very differently than Pokémon with balanced stats.

```
python
```

```
df['Stat_Standard_Deviation'] = df[['HP', 'Attack', 'Defense', 'Sp_Atk', 'Sp_Def',
'Speed']].std(axis=1)
df['Max_Stat_Ratio'] = df[['HP', 'Attack', 'Defense', 'Sp_Atk', 'Sp_Def',
'Speed']].max(axis=1) / df['BST']
```

- **Offensive Archetype Clues:** Is it a physical sweeper, special sweeper, or mixed?

```
python
```

```
df['Offensive_Bias'] = df['Attack'] - df['Sp_Atk']
```

- **Evolutionary Status (The "LC/NFE" Trap):** Does your dataset include unevolved Pokémon (like Pichu or Sprigatito)? If you don't have a column telling the model `Is_Fully_Evolved` (0 or 1), it might waste energy trying to distinguish why standard Pikachu has low stats but is in a different tier than other low-stat fully evolved Pokémon.

3. Handle the Class Imbalance

If your dataset has way more ZU and PU Pokémon than Ubers and OU (which is almost always the case), XGBoost will naturally default to predicting the larger classes to maximize its accuracy score.

You can fix this by telling XGBoost to pay extra attention to minority classes:

1. Calculate the class weights using `compute_class_weight` from scikit-learn.
2. In your Optuna-tuned params, set the `sample_weight` parameter inside the `.fit()` method of your cross-validation (or use a stratifying strategy that penalizes mistakes on rare tiers).

4. Re-evaluate Optuna's Direction

If your Optuna study optimized for `accuracy`, it will naturally sacrifice the rare tiers to get the common ones right. Try running a quick Optuna study where the objective returns `f1_macro` instead:

```
python

# In your Optuna objective:
score = cross_val_score(model, X, y, cv=5, scoring="f1_macro").mean()
```

`f1_macro` forces Optuna to find hyperparameters that care equally about getting Ubers right *and* getting ZU right.

2026-05-09 00:22:04 You Asked 

this isn't training accuracy this is test accuracy

Gemini 3 Flash

Ah, my bad! I misread that as your new *training* score after applying the regularized parameters.

A 54% **test accuracy** across 5 folds on a 7-tier classification task is actually a **massive win**—especially since we just stripped the model of its ability to cheat. You have successfully built a model that has genuine predictive power on unseen data, beating a random guess (14%) by nearly **4x**.

Now that we know your baseline "honest" test score is 54%, we have a clean launchpad. Since we are no longer memorizing the Pokédex, every point of accuracy we gain from here on out is a real, legitimate improvement.

Here is how we can push that 54% test score higher:

1. Look for the "Almost Correct" Predictions

In Pokémon, predicting **UU** for an **OU** Pokémon is a minor error, while predicting **ZU** for an **OU** Pokémon is a disaster.

Standard accuracy treats both of these mistakes exactly the same (as a 0). To see if your model is actually "soft-predicting" the right tiers, we should check its **off-by-one accuracy**.

Run this to see how often the model is either exactly right, or only off by a single tier:

python

```
import numpy as np
from sklearn.model_selection import cross_val_predict

# Get the out-of-fold predictions
y_pred = cross_val_predict(model, X, y, cv=5)

# Calculate absolute difference between predicted and actual tier indices
# (Assumes your target 'y' is label-encoded in order: e.g., 0=ZU, 1=PU, 2=NU...)
differences = np.abs(y.values - y_pred)

exact_accuracy = np.mean(differences == 0)
off_by_one_accuracy = np.mean(differences <= 1)

print(f"Exact Test Accuracy: {exact_accuracy:.4f}")
print(f"Within One Tier Accuracy: {off_by_one_accuracy:.4f}")
```

If your "Within One Tier" score is up in the **75%–80% range**, it means your model understands the metagame beautifully; it's just getting tripped up on the highly specific usage-percentage cutoffs Smogon uses.

2. Advanced Feature Engineering

To bridge that gap and get the exact predictions right, we need to feed the model features that capture the *extremes* of competitive Pokémon. Base stats alone don't tell the whole story.

Feature A: Stat Specialization (Min/Max Ratios)

Min-maxed Pokémon (like Shuckle with massive defenses, or Regieleki with extreme speed) perform completely differently than Pokémon with balanced stats (like Mew).

python

```
# Measure how specialized a Pokemon's stats are
df['Stat_Spread_SD'] = df[['HP', 'Attack', 'Defense', 'Sp_Atk', 'Sp_Def', 'Speed']].std(axis=1)

# Find their single highest stat relative to their overall BST
df['Max_Stat_Ratio'] = df[['HP', 'Attack', 'Defense', 'Sp_Atk', 'Sp_Def', 'Speed']].max(axis=1)
/ df['BST']
```

Feature B: Offensive Bias

Does the Pokémon lean heavily towards physical or special attacking? A mixed attacker uses stats differently than a dedicated sweeper.

```
python
```

```
df['Offensive_Bias'] = (df['Attack'] - df['Sp_Atk']).abs()
```

Feature C: The "Fully Evolved" Flag

If your dataset includes unevolved Pokémon (like Pichu or Torchic), the model will get incredibly confused trying to figure out why they have terrible stats but are in the same tier as fully evolved weaklings. You *must* give the model a way to identify these:

```
python
```

```
# A simple heuristic: if it can evolve, it's usually not a final-tier threat
# (You can map this from your pokeapi data if you have evolution chains)
df['Is_Fully_Evolved'] = ... # 1 or 0
```

3. Tune Optuna for Class Balance

If your dataset has way more ZU/PU Pokémon than Ubers/OU, XGBoost will naturally sacrifice the rarer tiers (Ubers/OU) to maintain a higher overall accuracy.

To fix this, go back to your Optuna study and change the target metric from `accuracy` to `f1_macro` :

```
python
```

```
# In your Optuna objective function:
score = cross_val_score(model, X, y, cv=5, scoring="f1_macro").mean()
```

`f1_macro` forces Optuna to find hyperparameters that treat every single tier with equal importance. This usually results in a much more balanced model that performs better on the highly competitive upper tiers.

2026-05-09 00:30:58

You Asked 

ok, off-by-one-acc is 69%

69% within-one-tier accuracy is a huge milestone! In any task with ordered classes (like Smogon tiers, where $ZU \rightarrow PU \rightarrow NU \rightarrow RU \rightarrow UU \rightarrow OU \rightarrow \text{Ubers}$), a high within-one-tier accuracy proves your model is not just guessing randomly or making chaotic mistakes. It has successfully mapped the *spectrum* of competitive viability.

It knows that a Pokémon with 130 Base Speed and a great offensive movepool belongs near the top, even if it occasionally muddles whether that Pokémon lands in UU or OU based on shifting player usage percentages.

To bridge that final gap from 69% to 80%+, we can exploit this ordinal structure. Since the model is struggling with the fine margins between adjacent categories, we should adjust how we approach the problem.

Strategy 1: Treat Tier Prediction as Regression (The Secret Hack)

Because tiers are inherently ordered, standard multiclass classification (which treats "predicting ZU instead of Ubers" with the exact same penalty as "predicting OU instead of UU") is actually fighting against the natural structure of the data.

If you temporarily convert your tiers to numbers (e.g., ZU = 0, PU = 1, ..., Ubers = 6) and train an **XGBRegressor**, you can use **Mean Absolute Error (MAE)**. This forces the model to minimize how far off its predictions are.

python

```
import numpy as np
import xgboost as xgb
from sklearn.model_selection import cross_val_predict

# 1. Train as a Regressor
reg_model = xgb.XGBRegressor(
    max_depth=4,
    learning_rate=0.05,
    n_estimators=300,
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=42
)

# 2. Get continuous predictions (e.g., 4.2, 1.7)
# Note: Ensure your 'X' has categorical features handled (like dtypes='category')
raw_preds = cross_val_predict(reg_model, X, y, cv=5)
```

```
# 3. Round predictions to the nearest integer to map back to a tier
rounded_preds = np.clip(np.round(raw_preds), 0, 6).astype(int)

# 4. Calculate your new metrics
exact_acc = np.mean(y == rounded_preds)
within_one_acc = np.mean(np.abs(y - rounded_preds) <= 1)

print(f"Regression -> Exact Accuracy: {exact_acc:.4f}")
print(f"Regression -> Within-One-Tier Accuracy: {within_one_acc:.4f}")
```

Why this often yields better results: If a Pokémon is on the borderline of RU and UU, the regressor might output 3.5. Rounding it lets the model capture that "middle-ground" nature naturally, rather than forcing a hard, low-confidence probability split.

Strategy 2: Look for "Stat Squeeze" Features

If you stick with your classifier, the model likely needs features that target the exact differences between a UU and an OU Pokémon. At 69% within-one-tier accuracy, the model is usually missing "contextual power."

- **The Speed Benchmark Feature:** In competitive formats, certain speed tiers are legendary (e.g., outspeeding base 100s). Try adding binary indicators for crucial competitive speed tiers:

python

```
df['Outspeeds_100'] = (df['Speed'] > 100).astype(int)
df['Outspeeds_110'] = (df['Speed'] > 110).astype(int)
```

- **Min/Max Stat Ratios:** Pokémon with highly specialized stats do better in higher tiers than all-rounders with the same BST.

python

```
df['Stat_Specialization'] = df[['HP', 'Attack', 'Defense', 'Sp_Atk', 'Sp_Def',
'Speed']].std(axis=1)
```

Strategy 3: Check the Confusion Matrix for the "Borderline" Culprits

If you run a confusion matrix on your current predictions, you will likely see that the "leakage" is happening almost entirely in **RU, NU, and PU**.

Ubers (which is packed with massive BST legendaries) and ZU (which is packed with fully-unevolved Pokémon or low-BST units) are usually very easy for the model to classify. The "mushy middle" of the competitive scene is where the usage stats fluctuate monthly.

If you look at those middle tiers, does your model seem to struggle more with defensive walls (which rely on subtle mechanics like typing and recovery moves) or fast sweepers?

2026-05-09 00:34:13 You Asked 

```
take this optuna output and write it into an xgboost classifier
constructor: 'max_depth': 4, 'learning_rate': 0.07825153661087353,
'n_estimators': 663, 'subsample': 0.8232309069300916,
'colsample_bytree': 0.6925108984783548, 'min_child_weight': 1,
'gamma': 0.6258033265610974
```

Gemini 3 Flash

Here is the clean `XGBClassifier` constructor configured with your optimized parameters from Optuna.

I've also included the necessary flags we discussed earlier— `enable_categorical=True` and `tree_method="hist"` —to make sure it handles your categorical columns (like `Ability_1` , `Ability_2` , etc.) without throwing errors.

python

```
import xgboost as xgb

# Define the final model with your Optuna parameters
optimized_model = xgb.XGBClassifier(
    # --- Optuna Tuned Parameters ---
    max_depth=4,
    learning_rate=0.07825153661087353,
    n_estimators=663,
    subsample=0.8232309069300916,
    colsample_bytree=0.6925108984783548,
    min_child_weight=1,
    gamma=0.6258033265610974,

    # --- Structural & Safety Parameters ---
    objective="multi:softprob",      # Configures the model for multiclass prediction
    enable_categorical=True,         # Crucial for handling 'Ability' category columns
    tree_method="hist",             # Required for high-performance categorical support
    random_state=42                 # Ensures your results are reproducible
```

)